NetPulse

Complete Technical Reference

Concepts · every function · design decisions · every setting · export calculations · server, tests, deployment · line-by-line analytics and export path

Applies to: netpulse.html v67 (offline single-file build) and the server analytics port

Prepared: 28 October 2026

Contents

1. What NetPulse is

1.1 The offline invariant
1.2 Two code bases
1.3 How to read this document

2. Code map: block by block

2.1 The state object and render loop
2.2 Storage layout

3. The data pipeline: from a spreadsheet row to a number

3.1 Ingestion
3.2 buildSamples: one row becomes one sample
3.3 computeAnalytics: samples become link records
3.4 Risk score and verdict

4. Forecast mathematics

4.1 Building the daily series
4.2 Fitting the trend (forecastFrom)
4.3 Outputs

5. Design decisions and why

5.1 Bugs that changed the design

6. Every setting, tick and toggle

6.1 Admin panel
6.2 Link page controls
6.3 Multi-link report panel (Reports)
6.4 Other controls

7. Exports: calculations and data manipulation

7.1 The shared definition of a hit (hitCalc)
7.2 Upgrade Policy Hits workbook (genUpgradePolicyXlsx)
7.3 Regulatory Capacity Report (regReportData, genRegReport)
7.4 Threshold Report and link report
7.5 Processed-data export (exportProcessed)
7.6 Multi-link pack
7.7 Other outputs

8. Which chart plots what

9. Testing, limits and known gaps

9.1 Testing
9.2 Known limits
9.3 Adding a feature safely

10. Complete code reference — part A: shell, utilities, storage

10.1 File envelope (lines 1–18)
10.2 Stylesheet (lines 18–726)
10.3 Document body and boot markup (lines 728–752)
10.4 Utilities (lines 762–790)
10.5 Icons (lines 796–827)
10.6 IndexedDB wrapper IDB (lines 835–854)
10.7 State and defaults (lines 857–913)
10.8 Storage safety (lines 950–977)
10.9 Workspaces (lines 978–1031)

11. Complete code reference — part B: ingestion layer

11.1 Header understanding (lines 1033–1078)
11.2 Unit and timestamp parsing (lines 1080–1166)
11.3 resolveColumns(headers, rows) (lines 1169–1238)

11.4 buildSamples — see section 3.2; additional details
11.5 File readers (lines 1332–1421)
11.6 Validation validateStaged (lines 1424–1466)
11.7 Commit pipeline (lines 1467–1562)

12. Complete code reference — part C: analytics, forecast, charts

12.1 Classification and time helpers (lines 1565–1581)
12.2 Identity merge (lines 1583–1708)
12.3 inMaintWin and computeAnalytics (lines 1712–1892)
12.4 Forecast code (lines 1893–1966)
12.5 Scoring and narrative (lines 1967–2043)
12.6 Charting engine (lines 2045–2200)

13. Complete code reference — part D: navigation and views

13.1 Navigation (lines 2252–2264)
13.2 Executive (view_exec, infraTrend, kpi, topNTable)
13.3 NOC (view_noc, focusAnomaly, nocCard)
13.4 Digital Twin (view_twin, twinSite, twinDevice, led)
13.5 Link page (view_intf) — the largest view
13.6 Capacity, Trends, Anomalies, Historical
13.7 Link Manager (view_links, lmSet, lmSetTh, lmSetThOff)
13.8 Uploads (view_uploads)
13.9 Processed export, Maintenance, Upgrade Watch, Reports, Audit

14. Complete code reference — part E: Excel writer and Excel/regulatory exports

14.1 Minimal XLSX writer (lines 3216–3353)
14.2 genUpgradePolicyXlsx(scope) — step by step
14.3 Regulatory report code
14.4 RCA helpers and report launchers (lines 3601–3710)
14.5 linkReportHTML and exportLink
14.6 breachReportHTML — complete walk-through (lines 3757–3893)
14.7 Multi-link report layer — bulkWindow, hitSeries, bulkHit, bulkPrep
14.8 PDF writer (lines 3905–3993)
14.9 pdfChart, pdfLinkReport, pdfFooter, pdfCover, bulkPdfFiles
14.10 bulkXlsx, bulkPanel, bulkGenerate

15. Complete code reference — part F: handlers, glossary, boot

15.1 Interaction handlers (lines 4439–4545)
15.2 Demo data genDemo (lines 4544–4573)
15.3 Router and presentation helpers (lines 4575–4727)
15.4 Global events and boot (lines 4738–4781)

16. Complete code reference — part G: server port, tests, scripts, deployment

16.1 server/ — the C# analytics port
16.2 Scripts
16.3 Deployment files
16.4 The other application in the repository: neteng-toolkit.sh
16.5 Documents and generators in the repository
16.6 The specification set (docs 01–21)

17. Code-review observations, dead code and known inconsistencies

18. Line-by-line annotated reference: analytics and export path

18.1 Turning raw cells into numbers: unit, speed, time and rate parsing
18.2 buildSamples: one spreadsheet row becomes one sample
18.3 inMaintWin and computeAnalytics: samples become link records
18.4 Forecast, risk score and verdict
18.5 The shared hit definition and the multi-link preparation
18.6 Upgrade Policy Hits workbook: genUpgradePolicyXlsx
18.7 Regulatory Capacity Report: data and Excel sheets
18.8 Processed-data export: exportProcessed (the audit trail)
18.9 Multi-link Excel pack: bulkXlsx and its helpers

18.10 Worked example: one sample from file to report

1. What NetPulse is

NetPulse is a network analytics and capacity-planning tool delivered as **one HTML file** (`diff/netpulse.html`, version 67, about 425 KB). You open it in a browser, drop in exported polling data (Excel or CSV: device, interface, timestamp, transmit and receive rates, link capacity), and it produces utilisation charts, threshold-hit counts, capacity forecasts, upgrade recommendations and formal reports (PDF, Excel, HTML).

It answers four questions for each link: **How busy is it? When did it cross a threshold? When will it run out of headroom? Does policy say it needs an upgrade?**

1.1 The offline invariant

The file never touches the network. Nothing is loaded from a CDN, no fonts are downloaded, no data is sent anywhere. A Content-Security-Policy meta tag (`default-src 'none'; connect-src 'none'`) makes the browser itself refuse any network request, so even a coding mistake cannot leak data. This is why the product is acceptable inside a regulated network: the data never leaves the machine.

- Storage: the browser's **IndexedDB** (a database inside the browser profile on that computer).
- File reading: browser-native `DecompressionStream` (to unzip .xlsx), `DOMParser` (to read the XML inside).
- File writing: a hand-written .xlsx writer and a hand-written PDF writer, both inside the same file. Downloads are Blob URLs.
- Icons are inline SVG paths; fonts are system fonts.

1.2 Two code bases

Part	Location	Role
Browser app	`diff/netpulse.html`	The reference implementation. Every number in every report comes from here.
Server port	`server/` (ASP.NET Core 8, C#, SQL Server schema)	Analytics engine re-implemented in C# so a central server can compute the same numbers for many users. Only the statistics, forecast and risk code is built; ingestion, API and UI are not. 38 xUnit tests prove it matches golden outputs captured from the browser engine to 1e-9.
Regression harness	`scripts/regression.mjs`, `scripts/capture-golden.mjs`	Drives the real page in Chromium (Playwright); checks every view, export, and cross-checks hit counts between reports.
Deployment	`diff/deploy/web.config`, `IIS-Deployment.md`	Optional static hosting on Windows IIS. Hosting the file does not centralise data; each user's data still lives in their own browser.

1.3 How to read this document

- **Section 2** walks the code block by block.
- **Section 3** follows data from a spreadsheet row to a number on screen (ingestion and analytics).
- **Section 4** is the forecast mathematics.

- **Section 5** records the design decisions and the bugs that shaped them.
- **Section 6** lists every setting, checkbox, dropdown and toggle and what it changes.
- **Section 7** explains each export: what is selected, which formula is applied, and how it is laid out.
- **Section 8** says which chart plots which statistic. **Section 9** covers testing and known limits.
- **Sections 10–16** are the complete code reference: every block and function of `netpulse.html` in file order (styles, utilities, storage, ingestion, analytics, charts, views, Excel writer, exports, PDF writer, handlers, boot), then the C# server port, tests, scripts, deployment files, the second application in the repository, the documents and the specification set. **Section 17** lists dead code and known inconsistencies found while reading every line.
- **Section 18** is a line-by-line annotated reference of the analytics and export path: every source line (verbatim) beside its explanation, ending with a worked example.

2. Code map: block by block

The file is about 4,800 lines in a fixed order. Line numbers below are for v67 and drift as the file changes; the function names are stable and searchable.

Lines (approx.)	Block	What it does and how
1–760	`<head>`, CSP, CSS	Security policy, design tokens (colour, spacing, radius, shadow variables for dark and light), glass-style panels, tables, chips, print rules. All styling is inline; light mode is `:root.light` overriding variables.
760–830	Helpers and icons	`fmtBps`, `fmtPct`, `esc` (HTML escape), `percentile`, `mean`, `std`, `clamp`, `toast`; `intern()` string-pooling (links repeat their name on every sample, so strings are shared to save memory); `ic()` returns inline SVG icons.
830–1030	State, IndexedDB, workspaces	Default config object `S.cfg` (thresholds 60/75/90 %, business hours 09:00–17:00 Mon–Fri, upgrade rule 80 %/5 days/business-hours/90-day look-back, risk weights). `IDB` stores one key per dataset: `batches`, `cfg`, `spl:<batchId>` (a compressed sample chunk per upload). `loadAll` rebuilds `S.samples`. **Workspaces** are separate IndexedDB databases. `storageErr` / `requestPersist` / `storageInfo` handle quota: a write that would overflow is refused and rolled back so an upload can never half-save. `logAudit` appends to the audit log.
1075–1240	Column mapping and parsing	`FIELD_ALIASES` lists synonyms for each field. `resolveColumns` scores every header against the aliases (exact, then fuzzy via Levenshtein `levDist`/`simRatio`) and returns a map with a confidence. `headerUnit` reads units from headers such as "Transmit (Mbps)". `parseRate`/`parseSpeed` convert "25.41 Mbps", "1G", "10 Gbps" etc. to bits per second. `parseTs`, `xlSerialToTs`, `mkTs` read timestamps.
1240–1330	`buildSamples`	Turns spreadsheet rows into sample objects (section 3.2).
1330–1560	File readers and commit	`parseCSV`; `inflateRaw`/`parseZip`/`parseXLSX` (reads the .xlsx zip, shared strings, sheet XML); `findHeaderRow`; `ingestFile`; `validateStaged` (preview, warnings); `commitOneFile`/`commitStaged` (append-only de-duplication on `ifid@ts`, then write).
1565–1710	Classification and identity	`classify` guesses class and business impact from the interface name (Core, Trading, Internet, Backup, Storage, Voice, Management, Customer). `stateOfT` maps utilisation to normal/warning/high/critical against a threshold set. `isBH` tests market hours. `applyDeviceMerge`, `applyLinkMerge`, `mergeDeviceVariants` join renamed hostnames/interfaces into one history.

Lines (approx.)	Block	What it does and how
1712–1892	`inMaintWin`, `computeAnalytics`	The core. Groups samples per link, removes maintenance and invalid samples, computes statistics, distribution, policy days, baseline anomalies, forecast, risk, verdict (section 3.3).
1893–1966	Forecast engine	`rollingMedian`, `theilSen`, `forecastFrom` (section 4).
1967–2050	Scoring	`riskScore` (0–100), `verdictFor` (the recommendation text and level), `aiSummary` / `aiSummarySimple` (plain-English paragraphs generated from the numbers; no external AI is called).
2052–2260	Charts	`downsampleMM`, `svgLineArea`, `niceTop`, `chartSVG` (the Traffic & Utilisation chart), `chartHover`/`chartZoom`, `donut`, `heatmap`, `calHeat`, `sparkline`, `riskBar`. Pure SVG strings; no chart library.
2260–2830	Views	`view_exec` (executive), `view_noc`, `view_twin`, `view_intf` (single link page), `view_capacity`, `view_trends`, `view_anomalies`, `view_historical`, `view_links`, `view_uploads`.
2832–3030	Data and policy views	`exportProcessed` (audit export), `view_maintenance`, `view_upgrade`, `view_reports`, `view_audit`.
3033–3190	`view_admin`	All configuration panels (section 6.1).
3194–3355	Report helpers and xlsx writer	`genReport`; `crc32`, `zipStore`, `zipDeflate`, `sheetXML`, `chartXML`, `drawingXML`, `xlsxParts`, `xlsxBlob`/`xlsxBlobZ` (build a real .xlsx with styles and native Excel charts).
3356–3600	Excel and regulatory exports	`genUpgradePolicyXlsx`, `regReportData`, `genRegReport`, `exportTableCSV`, `dl`.
3634–3910	Link and threshold reports	`linkReportHTML`/`exportLink`, `activeTF`, `hitCalc`, `breachReportHTML`.
3913–4440	Multi-link pack (between `/*BULK-START*/` and `/*BULK-END*/`)	`pdfBytes`/`PdfDoc` (PDF file writer), `bulkWindow`, `hitSeries`, `bulkHit`, `bulkPrep`, `pdfChart`, `pdfLinkReport`, `pdfCover`, `bulkPdfFiles`, `bulkXlsx`, `bulkPanel` (UI), `bulkGenerate` (button handler).
4439–4545	Action handlers	`saveIfMeta`, `addMaint`, `saveThresholds`, `savePlanning`, `saveTimezone`, `saveForecastCfg`, `setForecastHours`, `saveClassThresholds`, `saveCompliance`, `setMarketHours`, `saveUpgrade`, `addCustomField`, upload handling.
4544–4784	Demo, routing, boot	`genDemo` (synthetic data), `go`/`navTo` routing, `render`, `wireDynamic`, theme, boot IIFE that opens IndexedDB, loads data and paints.

2.1 The state object and render loop

One global object `S` holds everything: `S.samples` (every sample in the workspace), `S.cfg` (settings), `S.ifmeta` (per-link overrides), `S.maintenance`, `S.comments`, `S.analytics` (cached result), `S.scope` (current view and time filter). Views are functions that return an HTML string. `render()` calls the current view, inserts the string, then `wireDynamic()` attaches events.

Any setting that changes a number sets `S.analytics=null` and calls `render()`; the next view asks `A()`, which sees the cache is empty and runs `computeAnalytics()` again. This is why changing a threshold instantly re-colours every chart and verdict without re-uploading.

2.2 Storage layout

- One IndexedDB per workspace (default workspace plus any you create).
- Samples are stored in chunks, one per upload batch, compressed with `CompressionStream`. Roughly 18 MB per million samples.
- Config, per-link overrides, maintenance windows, comments, audit log, and the batch list are small JSON values.
- Every write goes through one guarded function; on a quota error it rolls back the in-memory state so the screen never shows data that is not saved.

3. The data pipeline: from a spreadsheet row to a number

3.1 Ingestion

1. **Read the file.** .xlsx is unzipped and its XML parsed; CSV is parsed directly. `findHeaderRow` skips title rows above the real header.
2. **Map columns.** `resolveColumns` matches each header to a field (device, interface, timestamp, transmit, receive, peak transmit/receive, capacity or per-direction capacity, percentage columns, site, IP, link name, and any custom field). The preview screen shows the mapping and confidence; you can override any field with a dropdown (this sets `manual` and confidence 1).
3. **Read units.** A unit in the header ("Transmit (Mbps)") or in the cell ("25.41 Mbps") fixes the scale. Capacity like "1G" or "10 Gbps" is converted to bits per second.
4. **Build samples** (3.2).
5. **Validate and preview.** Warnings count rows with no timestamp, no capacity, utilisation recalculated differently from a supplied utilisation column (more than 15 percentage points apart), and over-capacity samples.
6. **Commit.** Samples are appended; a sample whose `ifid@ts` (link identity plus timestamp) already exists is skipped as a duplicate, so re-uploading the same file adds nothing. Uploads are batches and can be removed individually in Uploads.

3.2 buildSamples: one row becomes one sample

Step	Rule
Identity	Rows without device or interface are skipped. Link identity `ifid = device|interface`.
Link attributes	Site, IP, link name and custom fields are link attributes, not time series: the last non-empty value per link wins.
Timestamp	Rows without a parseable timestamp are counted (`noTs`) and skipped. Timestamps are kept **as written** (wall-clock) when Admin → Time zone → "As written" is selected; with "UTC" they are converted to your zone. Excel serial numbers are converted without applying any hidden offset.
Rates	`parseRate` returns bits per second. Percentage columns can stand in: if rate is missing but "Transmit %" exists, rate = % ÷ 100 × capacity of that direction.
Capacity	Per-direction capacity (transmit and receive bandwidth) if present, else the single speed column. Sample capacity `speed = max(transmit capacity, receive capacity)`.
Unit-less magnitude guess	If at least 10 positive values in a column carry no unit anywhere, the median of value ÷ capacity decides: median above 1.5 (up to 3000) means the values are far larger than the capacity and are scaled ×1e-6; above 3000, ×1e-9; a positive median below 0.0005 (typical of a file that reports Mbps) is scaled up ×1e6. **Values with an explicit unit are never rescaled.** A note tells the user what was guessed.
Utilisation	Each direction against **its own** capacity: `uT = tx ÷ capacityTx`, `uR = rx ÷ capacityRx`. **Sample utilisation = max(uT, uR).** If neither can be computed, a supplied utilisation column is used.
Peak utilisation	Same formula on the peak-rate columns (catches microbursts hiding inside an averaged poll). Stored as `pu`.

Step	Rule
Invalid vs over-capacity	Only a **negative** utilisation is marked `bad` (a physically impossible reading). A value above 100 % (or the Admin "flag above" level) is **kept** and marked `over`; it usually means the capacity in the source is stale after an upgrade.
Output	`{ifid, dev, intf, ts, tx, rx, ptx, prx, pu, speed, speedTx, speedRx, util, bad, over, batch}`

3.3 computeAnalytics: samples become link records

For each link, in this order:

#	Stage	What happens
1	Group and sort	Samples grouped by `ifid`, sorted by time. The count and span before any exclusion are remembered (`rawCount`, `rawStart`, `rawEnd`).
2	Maintenance exclusion (live)	`inMaintWin` is evaluated now against the current maintenance windows (scope: all links, one device, or one link). Samples inside a window are removed from analytics. Because this is evaluated every time, adding or deleting a window changes charts immediately in both directions.
3	Classification and overrides	`classify` guesses class and impact from the interface name; a per-link override (Link settings) always wins. Site comes from override, file, then hostname.
4	Thresholds per link	Business-hours bands = global thresholds, overlaid by class thresholds, overlaid by per-link thresholds. Off-hours bands (if set globally or per link) overlay those for samples outside market hours.
5	Speed override	If the link has a speed override, utilisation and peak are recomputed for every sample against that single capacity.
6	Capacity change detection	If more than one distinct capacity appears in the history, `capChange` records the values, the time of the first change and whether it was an upgrade. Utilisation is already per-sample, so nothing is distorted; this is surfaced for transparency.
7	Remove invalid	Negative-utilisation samples are removed and counted (`badCount`).
8	Exclusion summary	`excluded` records how many samples were removed for maintenance or invalid reading, with first/last time, and whether the analysed data ends before the ingested data ends (`tailGap`/`headGap`). Used for the on-screen note and the "Excluded from this analysis" line in reports.
9	Statistics	`p95`, `p99`, `avg`, `mx` over per-sample utilisation. `pkP95`, `pkMax` over peak-column utilisation.
10	Provisioning peak	`provisionPeak` = the chosen percentile (p99.9 default, p99, or max) of peak utilisation over the trailing projection window (default 180 days before the link's last sample). Falls back to average utilisation if no peak column exists.

#	Stage	What happens
11	Time in state	Each sample is classified against its own band set (market vs off-hours); fractions of normal/warning/high/critical are stored in `dist`.
12	Business-hour counts	`breachesBH` = samples at or above Critical during market hours. `bhDays` = distinct calendar days with a market-hours sample at or above High.
13	Upgrade policy	Within the trailing look-back window (default 90 days, ending at the link's last sample): count distinct days with a sample at or above the policy percentage (market hours only if ticked). `upgradeNeeded = days ≥ required days`.
14	Baseline and anomalies	Mean and standard deviation per weekday-and-hour slot. A sample is a **Traffic spike** if the slot has ≥5 history points, sd > 0.001, z > 3.2, jump above the slot mean ≥ 8 percentage points, and utilisation > 15 %. Also: **Weekend traffic on business link** (class Core/Trading/Internet/Customer; weekend median > 20 % and ≥ 60 % of the weekday median) and **RX/TX imbalance** (median of max÷min > 6 over more than 10 samples).
15	Daily series and forecast	Section 4.
16	Risk and verdict	`riskScore` and `verdictFor` (3.4).
17	Aggregation	Per device and per site. Aggregate risk = 60 % impact-weighted average + 40 % worst link. Impact weights: Critical 1.0, High 0.7, Medium 0.4, Low 0.2.

> **Windowed analytics.** `computeAnalytics({from,to})` runs the identical pipeline on a narrower input and returns the result without replacing the cache. There is no second set of formulas.

3.4 Risk score and verdict

Risk score (0–100)

Six factors, each scaled 0–1, multiplied by a weight, summed, divided by the sum of weights, multiplied by an impact multiplier, clamped, and shown as a percentage.

Factor	Scaling	Weight	Notes
Peak util	`clamp(peak,0,1)^1.4` (peak = p95 of peak samples, else p95)	0.9	Non-linear: 50 % is much safer than 90 %.
Avg util	`clamp(avg,0,1)^1.4`	0.7	
Trend	`clamp(slope×60/0.5, 0, 1)`	0.8	Slope per day; 60 days of growth reaching +50 points = full marks.
Growth	`clamp(growthMoM/0.3, 0, 1)`	0.7	30 % monthly growth = full marks.
Violations	`clamp(bhDays/14, 0, 1)`	1.0	Business days at High or above.
Anomalies	`clamp(count/4, 0, 1)`	0.6	

Impact multiplier: Critical 1.25, High 1.05, Medium 0.85, Low 0.65.

Verdict (first matching rule wins)

#	Title / level	Condition
1	Data Quality Issue (warning)	Invalid samples ≥ 30 % of the link's samples.
2	Insufficient Data (warning)	Fewer than 8 samples.
3	Link Upgrade Recommended (critical)	`upgradeNeeded` (policy days met in the look-back window).
4	Recurring Business-Hour Congestion (critical)	≥ 5 business days at High or above and (Critical time > 2 % or High time > 10 %).
5	Critical Congestion (critical)	Time in Critical band > 3 %.
6	Capacity Planning Required (high)	Forecast reaches 90 % within the planning horizon and the link is growing (> 5 % per month with positive slope).
7	Weekend Backup Traffic (warning)	Weekend anomaly present.
8	RX/TX Imbalance (warning)	Imbalance anomaly present.
9	Bursty Traffic (warning)	Peak p95 ≥ Critical band while average p95 < High band.
10	Traffic Pattern Changed (warning)	Any spike anomaly.
11	Idle / Under-utilised (normal)	p95 < 3 %.
12	Monitor (warning)	High + Warning time > 15 % or growing.
13	Healthy (normal)	None of the above.

Verdict confidence = `clamp(0.4 + min(count/300,1)×0.35 + forecastR2×0.2 + min(bhDays/7,0.1), 0, 0.97)`.

4. Forecast mathematics

The aim is a defensible "days until this link reaches 80 / 90 / 95 %" with an honest range, not a single hopeful date. Every choice below exists to stop one noisy day or one weekly cycle from steering the answer.

4.1 Building the daily series

- Take only samples inside the **projection window** (default trailing 180 days).
- Optionally keep only market-hours samples (the All hours / Market hours toggle on forecast views).
- Bucket by calendar day. Each day's value is the **forecast metric** percentile (p95 by default; p99 or max selectable) of that day's utilisation.
- Drop days with fewer samples than the **minimum samples per day** (default 3), unless that leaves fewer than 5 days, in which case all days are kept.
- Direction: **Combined** uses the worse of TX/RX per sample. **Split** builds separate TX and RX series (each against its own capacity), forecasts both, and reports the direction that reaches 90 % first (`TX-bound` / `RX-bound`).

4.2 Fitting the trend (forecastFrom)

1. Fewer than 5 daily points: no trend; projection stays at current p95 and the note says "not enough history".
2. Clamp each daily value to 0–150 %.
3. **Deseasonalise**: replace each value by the median of a centred 7-day window. A 5-day working week with a 2-day weekend would otherwise draw a saw-tooth that biases the slope.
4. **Theil–Sen regression**: slope = median of all pairwise slopes (sampled with stride `max(1, floor(n/45))` to bound cost); intercept = median of `y − slope×x`. x is real elapsed days, so polling gaps do not distort it. A single spike day cannot bend a median.
5. **Confidence range**: slope low/high = the (1−CI)/2 and (1+CI)/2 percentiles of the pairwise slopes (CI default 80 %).
6. **Two candidate fits**: the full window and the recent half (at least 7 days). If there are 8 or more points, hold out the last 20 % (at least 3 points), refit each candidate on the rest, and choose the one with the lower mean absolute error on the held-out days. That backtest error also sets the relative error used for confidence.
7. **Plausibility**: slope clamped to ±4 % per day.
8. **Current level** = `slope × lastX + intercept`, clamped 0–`FC_MAX`. **`FC_MAX = 1.5` (150 %) is used for both level and every projection**; v65 fixed a bug where the level used 1.5 but projections used 1.2, producing a fake decline on links already above 120 %.

4.3 Outputs

Output	Formula
Days to threshold L (80/90/95 %)	0 if current ≥ L; else `(L − current) ÷ slope` if slope > 0.00001 per day; otherwise "no trend" (null).
Range for days to L	Earliest = using the high slope; latest = using the low slope. A low slope ≤ 0 means the latest date is unbounded.
Projections at 30/60/90/180 days	`clamp(current + slope×d, 0, 1.5)`, plus low and high bands from the CI slopes.
Growth per month	`clamp(slope×30 ÷ max(current, 0.03), −3, 3)`. Comes from the same slope, so growth and trend cannot disagree.

Output	Formula
Confidence	`0.15 + min(span/45 days,1)×0.25 + (1−relativeError)×0.3 + (0.1 if both fits agree on direction) + tightness×0.15`, capped 0.95. Tightness = `1 − (width of the 90 % range in days ÷ 120)`, 0.35 if unknown.
R^2-like score	`1 − relativeError` (backtest-based, not a textbook R^2).
Upgrade window	If days-to-90 % is positive: start = `max(0, d90 − lead time)`, end = `max(start+15, d90)`. Lead time default 42 days.
Model note	Which fit won, the CI percentage, plus metric/direction tags. Shown under the chart.

> **Provisioning vs forecast.** "Recommended capacity" in the regulatory report is not the forecast. It is `provisionPeak × N × capacity` where N is the Admin "Provision to N× peak" factor (default 1.5) and peak is the p99.9 of peak samples over the trailing window.

5. Design decisions and why

Each decision below is either a hard product requirement or the fix for a real defect found during the project. The "lesson" column is the reason the rule must not be quietly undone.

Decision	What the code does	Reason / lesson
Single offline file	Everything inline; CSP blocks all network.	Regulated environment; data must never leave the machine. A browser-enforced policy beats a promise.
Never drop data	Only negative utilisation is invalid. Above-100 % samples are kept and flagged. Maintenance and invalid rows are excluded from analytics but retained in storage and listed in the audit export.	An early version silently discarded >100 % readings and hid whole days. Now every exclusion is counted and stated in the report.
Timestamps as written	Wall-clock kept exactly; a setting says whether the file is already local or UTC.	Browsers disagree on time zones; shifting data made market-hours flags and report times disagree. One clock everywhere.
Utilisation per sample, against that sample's own capacity	`util = max(tx÷capTx, rx÷capRx)` stored per sample.	A 1G→10G upgrade must not rewrite last year's percentages. Asymmetric links exist.
Maintenance evaluated live	`inMaintWin` runs inside `computeAnalytics`.	v60 bug: exclusion stamped at upload never refreshed, so deleted windows left data hidden forever.
Append-only dedupe by link+time	`ifid@ts` key.	Re-uploading overlapping files is safe and idempotent.
A "hit" is one sample	`hitCalc`: util at/above the band threshold (or policy %) on a single sample.	Reports count exceedances a human can verify row by row. One shared function serves the Threshold Report and the multi-link pack so they cannot disagree (regression test cross-checks them).
Hit charts plot peak, not p95	Threshold Report and link report charts use bucketed **peak**; trend views use p95.	v66/v67 bug: a chart of daily p95 stayed under the line while the report counted hits above it, so the chart "contradicted" the numbers. A hit is a single sample; a p95 line cannot show one.
p95 for trend and capacity views	Exec fleet trend, Trends, sparklines, calendar strip, forecast use p95 of the day.	p95 is robust to single glitches and is the sustained-busy measure capacity planning needs. Deliberately different from hit counting; labels say which is shown.
Median-based forecasting	Rolling median + Theil–Sen + backtest choice + clamps.	Robust to spikes and weekly cycles; the backtest decides full-vs-recent window by evidence not assumption.

Decision	What the code does	Reason / lesson
One projection ceiling (150 %)	`FC_MAX` for level and projections.	v65: mismatched caps (150 % vs 120 %) made over-capacity links appear to decline.
Hit chart resolution follows window length	Hourly ≤ 16 days, 6-hourly ≤ 100 days, daily beyond; each point keeps its peak and average.	Readable at any span without hiding a hit by averaging.
Link identity merge	Short hostname ↔ FQDN auto-merge (optional); manual device and link merges.	Renamed hardware split one link's history in two and halved the forecast's data.
Workspaces as separate databases	One IndexedDB per workspace; downloads prefixed with the workspace name.	Two estates (e.g. Internet vs Corporate core) must never mix thresholds, maintenance, or reports.
Guarded storage	Quota errors roll back; storage meter; persistence request.	A half-saved upload is worse than a refused one.
Hand-written xlsx and PDF writers	`xlsxParts`, `PdfDoc`.	No libraries are allowed offline. Native Excel charts and a real PDF with bookmarks are generated directly.
Separate PDF downloads by default	Up to 25 files individually; more than 25 falls back to a zip.	Windows blocked extraction of zips from network shares as "potentially harmful". Separate files avoid the problem; the panel explains Unblock.
Report windows bound to the screen filter	`activeTF` feeds the Threshold Report, Excel hits and link report.	The document and the screen must describe the same period.
Server port with golden tests	C# engine asserted against browser outputs.	A report that says 88.5 % in one place and 88.7 % in another is worse than none. Mutation testing confirmed the tests fail when the port is deliberately broken.

5.1 Bugs that changed the design

Ver.	Symptom	Fix
v60	Chart stopped after 18 July; export still had rows.	Maintenance moved from stamp-at-upload to live evaluation; exclusion summary added.
v61	Same link split by hostname + interface renames.	Link merge (hostname and interface), manual link merge.
v62	Internet and Corporate estates mixed.	Workspaces.
v63	Storage fragility with many files.	Compressed chunks, rollback, meter, redesign.
v65	Forecast chart showed decline on already-saturated link.	Single 150 % ceiling. PDFs flagged as harmful on Windows: separate downloads.
v66	Threshold Report peak and counts did not match the chart.	Chart switched to daily peak.

Ver.	Symptom	Fix
v67	Same issue checked on other charts.	Per-link report chart now peak + average; Traffic & Utilisation chart confirmed per-sample; PDF chart confirmed peak-based.

6. Every setting, tick and toggle

"Tick" here means every checkbox, radio button, dropdown or toggle that changes behaviour. For each: where it is, what it changes, how the code uses it, and what it does **not** do. Saving a setting clears the analytics cache, so the whole app recomputes immediately unless stated otherwise.

6.1 Admin panel

Global thresholds

Control	Stored as	Effect
Warning / High / Critical ≥ (%)	`cfg.thresholds` (60 / 75 / 90 default)	Bands for every link without a class or link override. A sample is Warning if util ≥ warn, High if ≥ high, Critical if ≥ crit. Drives chart band lines, time-in-state, hits, verdicts.
Off-hours Warn / High / Crit	`cfg.thresholdsOff` (blank = same as business)	Applied to samples **outside** market hours. Lets a backup link run hot at night without alarms while market hours stay strict. Blank fields fall back to the business value.
Flag utilisation above (%)	`cfg.flagUtilAbove` (100 %, clamp 50–1000 %)	Samples above this are marked "over" and the link is flagged for capacity review in the Upgrade tab. **Never removes data.** Analytics derive it live, so it applies to existing data; the per-sample `over` flag stored in the file only updates on re-upload.

Business hours and planning

Control	Stored as	Effect
Business start / end hour	`cfg.businessHours`	Defines "market hours" (`isBH`) used by: off-hours bands, hits "Market vs Non-market", business-hours upgrade counting, forecast market-hours basis, reports. Days default Mon–Fri. Minutes come from the preset.
Planning horizon (days)	`planHorizonDays` (90)	A link whose days-to-90 % is within this many days and is growing gets "Capacity Planning Required".
Upgrade lead time (days)	`upgradeLeadDays` (42)	Subtracted from days-to-90 % to produce the recommended upgrade window.
Preset: 09:15–15:30	`setMarketHours`	Sets market hours to 09:15–15:30 Mon–Fri and also saves any compliance fields currently typed.

Time zone

Control	Stored as	Effect
Your time zone	`cfg.tz`	Only a **label** on reports (IST, UTC, …). Does not move data.

Control	Stored as	Effect
Times in the input files are…	`cfg.inputTz`: `aswritten` or `utc`	As written: no shift. UTC: converted to your zone **when files are parsed**; applies only to newly uploaded files (re-upload to re-apply).

Link upgrade policy

Control	Stored as	Effect
Utilisation threshold (%)	`upgradeRule.utilPct` (80)	A sample at or above this counts as a policy hit.
Sustained for (days)	`upgradeRule.days` (5)	Number of **distinct calendar days** with at least one hit needed to trigger.
Look-back window (days, 0 = all)	`lookbackDays` (90)	Trigger is evaluated over this many days ending at the link's last sample, so an old breach cannot latch the verdict. The Excel policy report instead covers the full loaded history and says so.
Count business-hours only (checkbox)	`businessHoursOnly` (ticked)	Ticked: only market-hours hits count toward the trigger; non-market hits are shown but do not trigger. Unticked: all hours count.
Action	`mode`: recommend / auto	Wording of the verdict ("Upgrade recommended" vs "Auto-flagged for upgrade"). Does not change the maths.

Forecast and provisioning

Control	Stored as	Effect
Forecast metric	`forecastMetric` p95 / p99 / max	Which daily percentile the trend is fitted to. p95 = sustained busy; p99 near-peak; max = worst sample.
Direction	`forecastDirection` combined / split	Combined: worse of TX/RX per sample. Split: forecast each direction and report the one that saturates first.
Provision to N× peak	`provisionFactor` (1.5, min 1)	Regulatory sheet: recommended capacity = peak × N × capacity; "Meets N× headroom" = peak×N ≤ 100 %.
Peak basis for sizing	`peakBasis` p999 / p99 / max	Percentile of peak samples used for `provisionPeak`.
Confidence band (%)	`forecastCI` (80 %, 50–99)	Width of the slope range, hence the days-to-threshold range and bands drawn on the chart.
Projection window (days, 0 = all)	`forecastWindowDays` (180)	History used for provisioning peak **and** the forecast series.
Min samples per day	`forecastMinDaySamples` (3)	Thin days are dropped from the series (unless fewer than 5 days would remain).

Class-specific policies

A table with Warn/High/Critical per class (Core, Trading, Internet, WAN, Storage, Backup, Voice, Replication, Management, Customer). Blank means "use global". Stored in `classThresholds`. Priority for a link: **per-link override > class > global**. The default impact (Critical/High/Medium/Low) shown beside each class comes from `classify` and feeds risk weighting.

Custom fields

Add a named attribute (e.g. ISP, Purpose). Effects: it becomes a mappable column on import (matched by name), an editable box on each link, and a column in every Excel export and PDF report. Removing a field hides it but keeps the stored values. Names that clash with built-in fields are refused.

Compliance and regulatory

Control	Stored as	Effect
Institution / entity name	`institution`	Printed on every report cover and header, and in the multi-link panel title.
Regulatory capacity threshold (%)	`regThreshold` (75 %, clamp 10–100)	The ceiling used throughout the Regulatory Capacity Report.
Prepared / Reviewed / Approved by	`reportPreparedBy` etc.	Signature block of the regulatory report.
Log retention basis (days)	`regRetentionDays` (180)	Stated in the data-integrity section. It is a statement, not an automatic deletion.

Device and link identity

- **Merge N host(s) now** (`doMergeVariants`): rewrites stored samples so a short hostname and its FQDN become one device (shortest name kept). Safe rule: one name must be a dotted prefix of the other. Not auto-reversible (re-upload to undo).
- **Automatically merge new variants after each upload (checkbox)** (`mergeHostnames`): applies the same rule after every commit.
- **Manual merge, devices**: two chosen devices, no similarity guard; same-named interfaces combine.
- **Manual merge, links**: joins two (hostname + interface) identities; duplicates by timestamp are skipped and counted.

Workspaces, storage and data

- **Workspace picker (top bar)**: switches the whole app to another database instantly. **New / Rename / Delete**: the default workspace cannot be deleted.
- **Storage meter**: usage versus browser quota; about 18 MB per million samples.
- **Export all data (JSON)**: dumps batches, samples, configuration and audit log of the current workspace. **Reset all data**: deletes every sample slice and clears samples, batches, maintenance windows and comments of the current workspace. **It does not reset configuration, link overrides or the audit log**, even though its confirmation text says it does (a known inconsistency).

6.2 Link page controls

Control	Effect
Filter From / To / Hours (Apply, Reset)	Narrows the charts, statistics on that page and the report windows to the range (`S.scope.tf`). Hours: all / business / off-hours. The **forecast always uses the full history** (stated on screen). The Threshold PDF, the single-link Excel policy report and the link export adopt the From/To window through `activeTF` (dates only; the Hours choice is separate — the Threshold PDF's own Hours select simply starts at the filter's Hours value).
Capacity forecast: All hours / Market hours	`forecastHours`. Market hours: the daily series uses only market-hours samples. Global setting, shown on forecast views.
AI summary: Technical / Simple	Switches between `aiSummary` and `aiSummarySimple` wording. Presentation only.
Threshold PDF: Hours	All / Market / Non-market. Samples outside the choice are not counted as hits.
Threshold PDF: Severity	Warning and above / High and above / Critical only, using that link's own thresholds (market vs off-hours bands honoured).
Link settings (Class, Impact, Tags, Speed, Site, Warn/High/Crit, off-hours bands, upgrade overrides, custom fields)	Per-link overrides stored in `ifmeta`. They beat any automatic or global value. **Speed override recomputes utilisation for every sample** against that one capacity. "Revert" deletes the overrides.
Comments	Notes with author role and timestamp, stored per link; audit-logged.
Maintenance form (scope, start, end, reason)	Adds a window (all links / one device / one link). Takes effect immediately in analytics and is audit-logged; deleting it brings samples back.

6.3 Multi-link report panel (Reports)

Control	Effect
Filter box, All, ⚠ Upgrade, None	Search the link list; select all visible; select only links whose upgrade trigger is met; clear.
Period	Last 7 / 30 / 90 days, 6 months (default 183), 12 months, All, or Custom From/To. Anchored on the **newest sample among the selected links** (`bulkWindow`) so links are compared on the same dates.
Hours	All / Market only / Non-market only.
Count a hit as	Warning and above, High and above, Critical only (each link's own bands), or the link's Upgrade policy percentage.
Include: Utilisation graph (checkbox)	Adds the period chart to each PDF and a chart sheet per link plus a "Top Links Chart" to the Excel file.
Include: Hits by date (checkbox)	One row per day per link.
Include: Hits by date-time (checkbox)	Every hit. PDF lists the first 10,000 per link; Excel has all (split into sheets of up to 200,000 rows).
Untick both tables	Produces a summary-and-graph report without tables.

Control	Effect
PDF layout (radio)	Separate PDF files (one download per link, up to 25) / one .zip of PDFs / one combined PDF with cover, contents and bookmarks. More than 25 separate files automatically become a zip.
Generate PDF / Generate Excel	Runs `bulkGenerate`; shows progress; warns when a selected link has no data in the period ("no data", not zero hits). Audit-logged with all options.

6.4 Other controls

Control	Effect
Role selector (top bar)	Admin, Architect, NOC Lead, NOC Eng, Read-Only, Viewer. Labels audit entries and comments; the offline file has no login, so this is a convenience, not security.
Theme toggle ◐	Dark / light via `applyTheme`; remembered.
NOC wall mode ⛶	Full-screen display for a wall monitor.
Demo data	Generates synthetic links for trying the tool.
Upload column mapping dropdowns	Override any auto-detected column.
Remove upload batch	Deletes that batch's samples; the rest remain.
Processed-data export: link, From, To, CSV or Excel	Scope and window for the audit export (7.5).
Reports: link selector, All links	Scope for the policy and regulatory exports.

7. Exports: calculations and data manipulation

Every export starts from the same per-link records produced by `computeAnalytics` (section 3), applies a scope and a window, then formats. The table shows the family at a glance; subsections give the details.

Export	Format	Scope / window	Counts based on
Upgrade Policy Hits	.xlsx (3 sheets)	All links or one; screen filter if set; else full history	Samples ≥ policy %
Regulatory Capacity Report	.xlsx, HTML, PDF (print)	All links or one; full history	p95, peak, forecast, ≥ regulatory %
Threshold Report (per link)	PDF via print	Link; screen filter or last 183 days	Samples in selected bands
Link report	HTML/print	Link; screen filter	Peak + average, hits
Processed-data export	.csv / .xlsx	Link or all; From/To	Every sample incl. excluded
Multi-link PDF	.pdf (files, zip or combined)	Selected links; common window	Samples in selected bands or policy %
Multi-link Excel	.xlsx with charts	Same	Same
Table CSV buttons	.csv	The visible table	As displayed
Export all data	.json	Workspace	Raw

7.1 The shared definition of a hit (hitCalc)

One function decides what a hit is, for the Threshold Report and the multi-link pack alike.

1. Candidate samples = the link's analysed samples (maintenance and invalid already removed) with utilisation present, inside [window start, window end], and passing the **Hours** filter (all / market / non-market).
2. For each sample pick the band set: market hours use `r.th`; other times use `r.thOff` if the link has off-hours bands, else `r.th`.
3. **Severity mode** (Warning / High / Critical): classify the sample into normal/warning/high/critical. It is a hit if its band is in the selected set (warning+high+critical; high+critical; critical).
4. **Policy mode**: it is a hit if `util ≥ upgradeRule.utilPct/100`, whatever band that is. Hits are still tallied by band for the breakdown ("Below warning" appears if policy % is below the warning line).
5. Tallies: counts per band; distinct hit days (all, and Critical days); market vs non-market split of all hits and of Critical hits; per-day record {warnings, highs, criticals, total, market count, peak, peak time, first, last}; and the list of hit samples.
6. **Peak** = highest utilisation among all candidate samples (not only hits) with its time.

7.2 Upgrade Policy Hits workbook (genUpgradePolicyXlsx)

Selection

- Threshold = link's `upgradeRule.utilPct`. `allCross` = every analysed sample ≥ threshold inside the optional screen filter (so the file shows every crossing, market and non-market).
- `policyHits` = market-hours crossings only if the policy is "business hours only", else all crossings. `daysHit` = distinct calendar days in `policyHits`; `met = daysHit ≥ required days`.
- Links are sorted by number of crossings (then p95), so links that did cross appear at the top.

Sheets

Sheet	Columns and meaning
Summary	Header block (title, generated time, scope, coverage and days, a note that counts cover full loaded history while the live verdict uses the rolling look-back, policy wording, capacity-change note, "wrong link selected" warning). Then one row per link: device, interface, link name, site, custom fields, class, impact, capacity (Mbps), policy threshold %, policy days, policy counts (Market hours only / All hours), days threshold hit, market-hour hits, non-market hits, total crossings, last hit, data-through, trigger met (YES/no), busy p95 %, verdict title.
Hits by Day	One row per link per calendar day with crossings: date, weekday, market hits, non-market hits, total, peak %, first and last hit time, policy %. Built by rolling `allCross` into days and splitting by `isBH`.
Hits by Date-Time	One row per crossing: timestamp, Market/Non-market, utilisation %, transmit and receive (Mbps), capacity (Mbps), "Counts toward policy" (Yes/No), policy %. Truncated at 50,000 rows with a note.

Data manipulation: Mbps = bits per second ÷ 1,000,000, rounded to 1 decimal; percentages rounded to 2 decimals; timestamps rendered as wall-clock with the zone label. If no crossings exist the sheets say so explicitly (with the coverage days) rather than appearing blank. If a larger device-name twin exists for the chosen link, a warning is written atop every sheet.

7.3 Regulatory Capacity Report (regReportData, genRegReport)

Per link, with T = regulatory threshold (default 75 %):

Column	Calculation
Peak %	`provisionPeak` (p99.9 / p99 / max of peak samples in the trailing window); fallback to the larger of peak-p95 and max.
p95 %, Avg %	Statistics of per-sample utilisation over the analysed history (maintenance excluded).
% time > T (all hours)	Samples with util ≥ T ÷ all samples of the link.
% time > T (market hrs)	Market-hours samples with util ≥ T ÷ market-hours samples.
Days > T	Distinct calendar days with a sample ≥ T.
Headroom to T (pp)	`max(0, T − p95)`.
Headroom to 100 % (pp)	`max(0, 1 − p95)`.
Days to T	0 if forecast current ≥ T; else `(T − current) ÷ slope`; "no trend" if slope ≤ 0.00001 per day.
Days to T (range)	Same using the high-slope (earliest) and low-slope (latest) of the confidence interval.
Recommended capacity (N×peak, Mbps)	`peak × capacity × N ÷ 1,000,000`.
Meets N× headroom	YES if `peak × N ≤ 1`; else "NO — below N×".
Augmentation required	YES if `p95 ≥ T` **or** days-to-T is between 0 and 180.
Verdict	Verdict title from 3.4.

Sheets: **Cover** (institution, period, generated, scope, threshold, provisioning target, projection basis, market hours, retention, total samples, links at/above threshold, links requiring augmentation, signatories, classification), **Capacity Summary** (above), **Threshold Breaches** (every sample ≥ T, 50,000 cap, with market-hours flag), **Definitions** (methodology, indicative regulatory alignment, disclaimer). The HTML/PDF version holds the same executive position, a ranked table with flagged rows, data-integrity and retention statements, methodology, alignment and the signature block. The threshold and format must be validated by the institution's compliance team; the tool supplies data and structure.

7.4 Threshold Report and link report

Threshold Report (breachReportHTML, "Export PDF report")

- **Window**: the screen filter if set; otherwise the last 183 days ending at the link's last sample.
- **Counts**: from `hitCalc` with the chosen hours and severity: warning/high/critical hits, distinct days, market vs non-market split, peak and peak time.
- **Chart** (v66): bucketed series from `hitSeries` plotting the **peak** per bucket (hourly / 6-hourly / daily by window length) with threshold guide lines, labelled axes, the peak marked and a legend. Because the line is the maximum in each bucket, every hit visibly rises above its threshold line.
- **Tables**: hits by day, and the timestamped detail — limited to the first **3,000** hits, with a note pointing to the Excel exports for the full log. Exclusions (maintenance, invalid) are stated in an "Excluded from this analysis" line.

Link report (linkReportHTML, "Export link")

Summary statistics for one link, the same chart style (peak plus average since v67), forecast results and verdict. Uses the screen filter for its window.

7.5 Processed-data export (exportProcessed)

The audit trail: **one row per sample, nothing removed**, so any chart or report number can be reconciled to source.

Column	Meaning
Device, Interface, custom fields	Identity and link attributes.
Timestamp (local), Epoch ms	Wall-clock with zone label (to the second) and the numeric value.
Transmit / Received / Peak Tx / Peak Rx (Mbps)	bps ÷ 1e6, 3 decimals.
Capacity Tx / Rx (Mbps)	Capacity in force at that sample.
Transmit % / Receive %	`rate ÷ capacity` per direction.
Utilization % (max), Peak %	Stored per-sample values.
Period	Market / Non-market hours.
Policy threshold %, Counts as policy hit	Yes if not invalid, util ≥ policy %, and (market hours if the policy is business-hours-only).
Capacity review	Warning text when util exceeds the flag level; sample kept.
Excluded from analytics	"Yes — maintenance window", "Yes — invalid (negative) reading" or "No — included". Evaluated **now** against current maintenance windows.

Limits: 200,000 rows for Excel, 1,000,000 for CSV; a note row is added if truncated. Sorted by link then time.

7.6 Multi-link pack

Preparation (bulkPrep, per link)

1. Window from `bulkWindow` (common to all links).
2. `H = hitCalc(...)` with the chosen hours and bands. `total` = number of hits.
3. Coverage: the number and first/last time of the link's samples that fall in the window. A link with zero is reported as **"no data"**, not "0 hits".
4. If graph is ticked, `hitSeries` produces the bucketed points {time, peak, average, sample count}.
5. `bulkHit` describes the hit definition in words (e.g. "Warning and above, utilisation at or above 60 % (per-link thresholds)", or the policy %).

PDF (pdfLinkReport, pdfChart, pdfCover)

- A real PDF is written by hand: Helvetica text, vector chart paths, FlateDecode streams, outline bookmarks, and an Info dictionary. Non-Latin-1 characters print as "?".
- Each link report: identity header (including custom fields), reporting period and hit definition, summary counts (total, Warning/High/Critical, market vs non-market, distinct days, peak and when, samples in period, coverage), then, according to the ticks: the **graph** (peak line, average line, threshold lines, hits marked), **hits by date**, **hits by date-time** (first 10,000 hits, with a note of how many were omitted).
- Layouts: separate files, zip, or one combined PDF with a cover page (`pdfCover`: institution, period, hours, hit definition, totals, links ranked by hits) and a contents page with bookmarks.

Excel (bulkXlsx)

Sheet	Content
Summary	Header block; one row per link sorted by hits (busiest first): identity, custom fields, class, impact, capacity, data from/to, samples in period, total, warning, high, critical, market-hour hits, non-market hits, distinct hit days, peak and time, first and last hit, policy %, policy days hit, upgrade trigger met. Filter and frozen header.
Top Links Chart	Stacked column chart (native Excel chart) of hits by band for the top 25 links with hits (plus "Below warning" in policy mode).
Hits by Date	Per link per day: warning, high, critical, total, market, non-market, peak, peak time, first, last.
Hits by Date-Time	Every hit, split into sheets of up to 200,000 rows each (Excel limit is 1,048,576).
One sheet per link	Time, Peak, Average, "Peak at/above hit threshold" (markers), Warning/High/Critical (and Policy) lines, sample count, with a native line chart. Sheet names are made unique, ≤31 characters, with forbidden characters replaced.

Excel timestamps are real Excel date-time serial numbers (formatted cells), so they sort and filter correctly; percentages are stored as fractions with a percent format.

Consistency guarantee

The pack and the Threshold Report both call `hitCalc`; the regression test generates both and asserts they print identical counts for the same link, window, hours and band.

7.7 Other outputs

- **Weekly/Executive/Capacity reports** (`genReport`): printable HTML tables from the same analytics (critical links; links forecast to reach 90 % within 90 days; etc.).
- **CSV buttons** on tables export exactly the displayed cells.
- **Audit log**: append-only record of uploads, config changes, merges, exports (what, scope, options, counts).

8. Which chart plots what

Chart	Statistic	Why
Traffic & Utilisation (link page)	Per-sample values (min/max envelope when zoomed out)	Shows real samples so every spike is visible. Downsampling keeps min and max per bucket, never the mean.
Threshold Report chart (v66)	Daily/6-hourly/hourly **peak**	Counts are single samples, so the plot must show maxima.
Link report chart (v67)	Peak + average	Same reason; average adds context.
Multi-link PDF and Excel link charts	Peak, average, hit markers, threshold lines	Same definition as the counts.
Capacity forecast chart	Daily forecast-metric (p95 default), fitted projection, confidence band	Trend of sustained load; deliberately smoothed.
Exec fleet trend, Trends, sparklines, calendar strip	Daily p95	Fleet/trend views show sustained load; a day can look green while containing a hit (documented limitation).
Heatmap	Mean by weekday × hour	Shows when a link is busy, not how bad the worst moment was.

9. Testing, limits and known gaps

9.1 Testing

- `node scripts/regression.mjs` drives the page: renders every view, builds every export, tests maintenance exclusion both ways, link merge, workspace isolation, storage rollback, download modes, over-capacity forecast, and a cross-check that the multi-link pack matches the Threshold Report. Zero console errors required (CSP violations show as errors, so this also guards the offline rule).
- Regression tests were verified to **fail** under the old bugs (e.g. the over-capacity test fails with the old 1.2 ceiling).
- Server: 38 xUnit parity tests against golden outputs; mutation-tested.

9.2 Known limits

Limit	Detail
PDF font	Helvetica only; characters outside Latin-1 become "?".
PDF hit list	First 10,000 hits per link (Excel has all).
Policy workbook cap	50,000 hit rows.
Multi-link window	Anchored on the newest sample among selected links; older-ending links show "no data".
Per-browser data	Data lives in one browser profile; clearing site data deletes it. Export JSON for backup. Hosting on IIS does not centralise data.
No authentication	Role selector is a label only.
Unconfirmed	Cause of the Windows "potentially harmful" zip block was mitigated (separate files, Unblock guidance) but not confirmed on the user's machine.
Server port	Analytics built; ingestion, API, UI not; SQL schema written but never executed.
Heatmap vs business hours	Observed hot 15:00–21:00 against business 09:00–16:00 on sample data; noted, not changed.
Time zone change	Applies to newly uploaded files only.

9.3 Adding a feature safely

1. Keep it offline: no network calls, no external assets; re-run the offline grep from `CLAUDE.md`.
2. If it changes a number, change it in one shared function and make reports call that function.
3. Prefer a setting stored in `S.cfg`; clear `S.analytics` on save.
4. Run `node scripts/regression.mjs`. If forecast or risk maths changed, regenerate goldens (`capture-golden.mjs`) and run `dotnet test`; read any fixture diff as a published-number change.

10. Complete code reference — part A: shell, utilities, storage

Sections 10–16 document every function and block in `netpulse.html` in file order (and then the server, scripts and deployment files). Where earlier sections explain *concepts*, these explain *code*: inputs, outputs, side effects and the reason for each unusual line. Line numbers are v67.

10.1 File envelope (lines 1–18)

- `<!doctype html>` and a long comment block restating the **offline invariant** for any future editor.
- **CSP meta tag**: `default-src 'none'; script-src 'unsafe-inline'; style-src 'unsafe-inline'; img-src 'self' data: blob:; media-src 'self' data: blob:; font-src 'self' data:; connect-src 'none'; base-uri 'none'; form-action 'none'; frame-src 'none'; object-src 'none'`. Only inline script/style can run; images/media only from the page itself, `data:` or `blob:` (needed for downloads); every network route (`fetch`, XHR, WebSocket, forms, frames, plugins, `<base>`) is blocked. A report preview opened with `window.open` + `document.write` stays within these rules because it is not a navigation to a remote origin.
- Viewport meta (responsive) and `<title>`.

10.2 Stylesheet (lines 18–726)

The CSS is layered. Later layers deliberately override earlier ones (no deletion, so each design pass is auditable), which is why the same selector appears several times.

Lines	Layer and contents
19–70	**Base tokens and reset.** `:root` custom properties: background, panel/panel2 (translucent), stroke/stroke2 (hairlines), text/muted, accent/accent2, state colours `--normal/--warning/--high/--critical`, `--down`, `--nodata`, glow, shadow, radius, chrome, table-header background, gridline, AI-text, `--pagebg` gradients. `:root.light` redefines them for light mode. Body uses a system font stack with antialiasing; `.mono` uses tabular numerals.
71–100	**Layout.** `#app` is a CSS grid: 236 px sidebar column, 56 px top bar row, areas `logo top / nav main`. `#main` scrolls vertically only. `.navgrp`, `.navitem` (hover, active), `.badge` (critical-count pill). Topbar search, `.topbtn`, `.role` selector. Breadcrumb styles.
102–142	**Panels and KPIs.** `.panel`, `.pad`, `.grid` with `.g2/.g3/.g4/.g-main` (collapse to one column below 1150 px), `.sec-title`, `.kpi` (+ glow), `.up/.down/.flat` text colours. **Verdict banner**: `.verdict` with a 4 px state-coloured left bar, `.lvl` badge, reason and action text. `.pill` and `.chip` (+ impact variants Critical/High/Medium/Low).
143–200	**Tables, digital twin, charts.** Sticky blurred table headers, hover rows, `.riskbar` (inline `.fill` made `display:block` to fix the empty-bar bug). `.tile`, `.leds`/`.led` (critical LEDs pulse), `.frontpanel`/`.port` (switch front-panel metaphor). Chart container, tooltip (`.tip`), tool buttons (`.ctools`), crosshair, zoom hint, legend, **heatmap grid** (`.heat`: 34 px label column + 24 hour columns), **calendar strip** (`.cal .d`).
202–250	**Forms and misc.** Dropzone `.dz` (+ `.drag`), buttons `.btn/.ghost/.sm`, inputs (shared style for text, number, datetime-local, select, textarea), `label.fld`, severity blocks, column-mapping grid `.map-row`, confidence bar, timeline `.tl`, toast, empty state, AI box, distribution legend, info glyph `.info` and `#gtip` tooltip, `.tabs` segmented control, flex helpers.

Lines	Layer and contents
252–300	**Motion layer (v-early).** Keyframes: fadeUp, fadeIn, popIn, drawLine (stroke-dashoffset line draw-on), fadeArea, growBar, floatBlob, breathe, ledBreathe, pulse. Chart lines animate on first draw (`pathLength=1` trick). Bars grow with `scaleX`. A `prefers-reduced-motion` block collapses all animation durations to ~0.
302–345	**v38 visual refresh.** Deeper surfaces, crisper borders, `--panel-hi` top highlight, unified transitions, focus-visible outlines, scrollbar styling.
347–540	**v56 Apple Liquid Glass.** Dark = true black with elevated grey surfaces; light = #f5f5f7 ground, white panels. Colours reserved for state. Radius tiers (`--r` 18, `--r-panel` 22, pill 999, chip 6, thumb 12). Solid surfaces (no blur) except where layers overlap: top bar, brand, nav, sticky table headers, tooltips use `saturate(180%) blur(20px)`. Hairline row dividers; pill primary buttons; Apple-style sidebar selection; quiet status tints; attention-queue verdicts merged into one list panel.
541–660	**v59 refinement.** Breadcrumb's last item becomes a large negative-tracked page title; more whitespace; KPI number as hero; segmented control with raised white thumb; calmer sidebar and pill search; primary button gradient (the one permitted gradient); composed empty state; chart scaffolding recedes (area fill alpha .085); responsive collapse and 44 px touch targets under 760 px; storage meter (`.stgbar`).
667–725	**v63 design pass.** Line-icon styling (`.ic` inherits colour), unified KPI row (`.kpis` — a 1 px grid gap is the hairline), state dot `.sdot` instead of ▲/▼, date input polish, **accessibility corrections** (light-mode muted greys darkened for 4.5:1 contrast; critical badge colour `#c9261c` for white-text contrast; 24 px minimum target size), and the multi-link picker styles (`.mllist`, `.mlrow`, `.mlck`).

10.3 Document body and boot markup (lines 728–752)

- `#app` contains `#brand` (dot, "NetPulse", a chip showing the version `v67`), `#topbar` (search `#q`, workspace selector `#wsSel`, Demo data, theme ◐, NOC wall ⛶, role selector), `#nav`, `#main`. `#toast` is the notification mount.
- One `<script>` in strict mode contains everything.

10.4 Utilities (lines 762–790)

Function	Behaviour
`$`, `$$`	`querySelector` / `querySelectorAll` as array.
`clamp(v,a=0,b=1)`	Bounds a number.
`fmtBps(v)`	bps → "x.xx Mbps" etc. in steps of 1000; decimals depend on magnitude (≥100: 0, ≥10: 1, else 2).
`fmtPct`, `pct1`	Fraction → percent string; `fmtPct` uses 1 decimal under 10 %, otherwise none; `pct1` always 1 decimal. Null/NaN → "–".
`esc(s)`	HTML-escapes `& < > "` — used on every user-supplied string put into HTML.
`fmtDate`, `fmtDay`, `dtLocal`	Locale date-time; day only; `yyyy-mm-ddThh:mm` for `datetime-local` inputs.
`days(a,b)`	Elapsed days. `DOW` = Sun..Sat.
`toast(msg)`	Shows a message for 3.2 s (HTML allowed, so only trusted text is passed).

Function	Behaviour
`stateColor(s)`	Reads `--normal` etc. from computed style so SVG strings pick up the active theme.
`levDist`, `simRatio`	Levenshtein distance and 1 − distance/maxLength similarity, used for fuzzy header matching.
`percentile(arr,p)`	Linear-interpolated percentile on a sorted copy (index = p×(n−1)); empty → 0.
`mean`, `std`	Mean; **population** standard deviation (0 if fewer than 2 values).
`amax`, `amin`	Loop-based min/max so arrays of 100k+ items do not overflow `Math.max(...arr)`; empty returns 0.
`intern(s)`	String pool (a `Map`) so repeated device/interface/ifid strings share one object; saves hundreds of MB at million-row scale.

10.5 Icons (lines 796–827)

`ICONS` maps names (search, file, download, printer, grid, bars, building, server, folder, user, spark, clipboard, clip, trend, message, upload, check, gauge, list, layers, capacity, arrowup, history, link, gear, clock, sliders, alert, bulb) to SVG path markup on a 24-unit grid. `ic(name,size)` wraps one in an `<svg class="ic">` using `currentColor`, stroke width 1.7, round caps. They inherit text colour so they follow the theme and stay monochrome.

10.6 IndexedDB wrapper IDB (lines 835–854)

Member	Behaviour
`db`, `root`, `ws`	`root` = the default database `netpulse` (always open; holds the workspace registry and active-workspace pointer). `db` = the database of the current workspace (same object as `root` for the default workspace).
`dbName(id)`	`netpulse` for default, `netpulse::<id>` otherwise.
`_open(name)`	Opens version 1 with a single object store `kv` (out-of-line keys).
`open(id)`	Opens root if needed, closes the previous workspace db, opens the target.
`_get/_put/_del`	Promise wrappers. `_get` resolves `undefined` on error; `_put` **rejects** on error (so quota errors surface); `_del` always resolves.
`get/put/del`	Operate on the current workspace.
`rget/rput`	Operate on the root registry only.
`drop(id)`	`indexedDB.deleteDatabase`, resolving on success/error/blocked.

10.7 State and defaults (lines 857–913)

- `TZLIST` — 13 curated IANA zones offered in Admin.
- `DEFAULT_CFG` — the complete default configuration (every key is documented in section 6). Notable defaults: class thresholds for Trading (45/60/80 %) and Backup (75/88/96 %); `peakBasis p999`; `forecastCI` 0.8; `forecastMinDaySamples` 3; `forecastWindowDays` 180; risk weights `{peak .9, avg .7, trend .8, growth .7, violations 1.0, anomaly .6, impact 1.2}` (the `impact` weight is stored but impact is applied as a multiplier in `riskScore`);

`regThreshold` .75; `regRetentionDays` 180; `tz` Asia/Kolkata; `inputTz` aswritten; `mergeHostnames` false.

- `cfList()`, `cfKey(label)` (`cf_` + normalised label, max 24 chars), `cfVal(r,key)` (per-link override beats the value from the file).
- `S` — global state: `cfg, samples, batches, audit, maintenance, comments, ifmeta, ifinfo, rca, watch, watchWin, role, view, scope, analytics, staged, aiMode`.
- `resetState()` — clears every workspace-owned value and restores default config so nothing can leak from the previous workspace.
- `loadAll(wsId)` — reset; open workspace db; read registry; read `batches`; **migrate legacy** single-blob `samples` into per-batch `spl:<id>` keys; otherwise concatenate every batch slice; re-intern strings (structured clone makes copies); load `audit, maintenance, comments, ifmeta, ifinfo, rca, watch`; read `watchWin` and `aiMode` from `localStorage` (wrapped in try/catch); merge saved `cfg` over defaults (`Object.assign`, so new keys added in later versions get defaults); `wsStamp()`.

10.8 Storage safety (lines 950–977)

Function	Behaviour
`storageErr(e)`	Quota errors → "Browser storage is full — nothing was saved. Free space in Admin → Storage..."; otherwise a generic write error.
`save(k)`	`IDB.put(k,S[k])` with a catch that toasts at most once per 4 s and logs to console. (The ternary for `cfg` is redundant; both branches read `S[k]`.)
`requestPersist()`	Best-effort `navigator.storage.persist()`; silent on refusal.
`storageInfo()`	`{usage, quota, persisted, pct}` from `navigator.storage.estimate()`.

10.9 Workspaces (lines 978–1031)

Function	Behaviour
`wsList/wsName/wsCur`	Registry accessors (default always exists).
`wsTag()`	Filename prefix (`Name_`) for non-default workspaces; used by `dl()`.
`wsSave(list)`	Persist registry to root DB.
`wsStamp()`	Records sample count, batch count, time span and last-seen for the current workspace in the registry, so the Storage panel can show every workspace without opening each DB.
`wsSwitch(id)`	Persist `activeWs`, `loadAll`, route to Executive (or Uploads when empty), render, toast.
`wsCreate()`	`prompt` for a name; id `ws<base36 time>`; add to registry; switch; audit.
`wsRename()`	Prompt; updates registry (re-adds default if missing); audit.
`wsDelete()`	Refuses the default; `confirm`; remove from registry; set active to default; close and drop DB; reload default.
`logAudit(action,target,detail)`	Prepends `{ts, who (role), action, target, detail}`, keeps **500** most recent, saves.

11. Complete code reference — part B: ingestion layer

11.1 Header understanding (lines 1033–1078)

- `FIELD_ALIASES` — for each canonical field, a list of normalised header synonyms: `device, interface, ts, tx, rx, ptx, prx, speed, speedtx, speedrx, util, txpct, rxpct, ptxpct, prxpct, site, ip, linkname`. Examples: `tx` ← out, egress, transmit, averagetransmit, outoctets, txbps…; `speedtx` ← transmitbandwidth, txcapacity…; `linkname` ← circuit, ifalias, alias….
- Meaning of fields: **speedtx/speedrx** = capacity per direction ("Transmit/Received Bandwidth"); **tx/rx** = average traffic rate; **ptx/prx** = peak traffic rate in the interval; **txpct…prxpct** = the same as percentages; **util** = overall utilisation %.
- `FIELD_LABELS` — human labels shown in the mapping UI; `fieldLabels()` adds custom fields with "(custom)".
- `REQUIRED_FIELDS = device, interface, ts`. `norm(s)` lowercases and strips everything but a–z, 0–9. `UNIT_RE` matches a number plus optional T/G/M/K plus a bit/byte token.

11.2 Unit and timestamp parsing (lines 1080–1166)

Function	Behaviour
`unitMultiplierFromToken(tok)`	Returns `{mult, byte}`: scale 1e12/1e9/1e6/1e3 by T/G/M/K, ×8 if the token is bytes/octets. Null if no b/octet/bps in the token.
`headerUnit(h)`	Finds an attached-prefix unit in a header (`mbps`, `gbit/s`, `bytes`). Returns null for percentage/util headers. **A bare "bps"/"bits" with no prefix is treated as ambiguous** (vendors label Mbps columns just "bps") and returns null so the magnitude check decides the scale; prefixed units and byte units stay exact.
`parseSpeed(raw)`	Capacity → bits per second. Numbers: ≥1e6 = already bps; 1000–1e6 = Mbps (×1e6); 1–1000 = **Gbps** (×1e9) — tuned for data-centre links (1, 10, 40, 100 → Gbps). Strings with a unit use it. A bare "100" meaning 100 Mbps would over-scale ×1000; `validateStaged` warns when nearly everything reads <0.5 %, and a Speed override fixes it.
`mkTs(y,mo,d,h,mi,s)`	Builds a timestamp from calendar fields: `Date.UTC` when `inputTz==="utc"`, else local `new Date(...)` (as written).
`xlSerialToTs(serial)`	Excel serial → instant. Rounds to the nearest second (Excel float error), computes the naive wall clock; UTC mode returns it as a UTC instant; as-written mode rebuilds the same fields in local time so 00:00 stays 00:00.
`parseTs(raw)`	Order: numbers 20000–80000 = Excel serial; >1e11 = ms epoch; >1e9 = seconds epoch. Day-first strings `dd-mm-yyyy[ hh:mm[:ss]]` (disambiguates by whichever part >12; otherwise treats first as day); ISO `yyyy-mm-dd[ T]hh:mm[:ss]` (trailing Z/offset ignored, fields taken as written); fall back to `Date.parse`.
`parseRate(raw,headerInfo)`	→ `{val (bps), viaCellUnit, unit}`. Numeric + header unit → scaled and `unit:true`. Strings: thousands commas removed, then a per-cell unit ("1,700.00 Mbps") wins; else a plain number scaled by the header unit if any.

11.3 resolveColumns(headers, rows) (lines 1169–1238)

1. Build `H[i] = {i, raw, n (normalised), unit}` for every header.

2. For each field, score every column: exact alias = 1; substring either way = 0.82; else `similarity × 0.75`.
3. Compute **value shape** from the first 40 non-empty cells per column: share that parse as dates, look like percentages, are numeric, are large (>1e5), and whether the column is monotonic integers.
4. Custom fields claim an **exact** header-name match first, so a field called "ISP" is not stolen by built-in fuzzy "ip".
5. Adjust scores: timestamp +0.6×date-share; percentage fields +0.35×pct-share, +0.3 if header looks like %; rate fields +0.05×numeric-share and −0.45 if header looks like % (so "Average Transmit %" never becomes a rate); `speed` +0.15×big-share, −0.5 if the header mentions transmit/receive/egress/ingress or %; `speedtx/speedrx` +0.1×big, −0.5 if %.
6. Sort candidates by raw score (so exact-alias wins stay ahead of boosted partial matches) and assign greedily: each field and each column used once; accept if score ≥ 0.42 (required fields) or 0.5.
7. Fallback: if no timestamp matched but a column is >60 % dates, use it.
8. Remaining custom fields: fuzzy-match to an unused column, accept if ≥ 0.6.

Returns `{map, conf, headers, shape}`. `map[field]` is a column index; `conf[field]` its score.

11.4 buildSamples — see section 3.2; additional details

- Rows are first collected into `raw` with per-direction capacity (`spTx`, `spRx`), explicit-unit flags (`tU`…), and percentage columns. Only then is **magnitude inference** run per rate column over values lacking any explicit unit, requiring ≥10 positive values and known capacity; factor = ×1e-6 if median(value ÷ capacity) is in (1.5, 3000], ×1e-9 if > 3000, ×1e6 if in (0, 0.0005) (the last is the usual Mbps-reporting file). A note (`scaleGuessNote`) is returned.
- `issues` counters returned: `corrupt` (negative), `noTs`, `noSpeed`, `utilRecalc`, `cellUnit`, `overCap`.
- `extremes` = the five highest-utilisation samples (shown in the preview so a human can sanity-check).
- `info` = per-link `{site, ip, linkname, cf:{…}}` gathered from the file.

11.5 File readers (lines 1332–1421)

Function	Behaviour
`parseCSV(text)`	Small state machine: quoted fields with `""` escapes, commas, `\n` row ends, ignores `\r`; drops rows that are entirely empty.
`inflateRaw(u8)`	`DecompressionStream("deflate-raw")` over a Blob stream; throws if unavailable (very old browsers).
`parseZip(buf)`	Finds the End-Of-Central-Directory record (scan from the end for `0x06054b50`), walks the central directory (`0x02014b50`) collecting `{method, data}` per entry; reads the local header to find the real data start. Supports stored (0) and deflated (8) entries; does not handle ZIP64.
`entryText(e)`	Inflates (unless stored) and decodes UTF-8.
`colToNum(ref)`	"AB12" → zero-based column index (base-26).
`parseXLSX(buf)`	Reads `xl/sharedStrings.xml` (`<si>` text), picks the first `xl/worksheets/sheetN.xml` (sorted), parses rows/cells with `DOMParser`; cell types: shared string (`s`), inline string, formula string (`str`), numeric (kept numeric when parseable). Fills gaps with null so columns align.
`findHeaderRow(rows)`	Scans the first 15 rows; score = 2×text cells + non-empty − numeric cells; needs ≥3 non-empty cells. Skips title rows.

Function	Behaviour
`ingestFile(file)`	CSV or XLSX → matrix → header row → `resolveColumns` → `{name, headers, rows, resolved, headerRow, rowCount}`.

11.6 Validation validateStaged (lines 1424–1466)

Produces three lists — **block** (must fix), **warn**, **info** — shown before commit:

- Block: missing device/interface/timestamp mapping; no traffic column (rate, %, or util); zero valid rows.
- Warn: no capacity column when needed; any mapping with confidence < 0.62; polling gaps (interval > 2.5× the median cadence); invalid (negative) values excluded; "almost all traffic < 0.5 % of capacity" when at least 50 samples have median utilisation < 0.5 % (probable over-scaled capacity); possible duplicate upload (checksum = count|first ts|last ts matches an earlier batch); overlap with existing samples (checks the first 5,000 keys; capped at 4,999).
- Info: over-100 % samples kept; supplied utilisation disagreed with recomputed; per-cell units normalised; auto-scaling note; coverage intersects a maintenance window; number of new interfaces.
- Side effects: stores `checksum`, `coverage`, `intervalNote`, `overlap` on `built`.

11.7 Commit pipeline (lines 1467–1562)

Function	Behaviour
`coverageOf(samples)`	First/last timestamp, checksum and median polling interval text ("~5 min").
`commitOneFile(pf,map,existKeys)`	New batch id `b<time>_<rand>`. Skips samples whose `ifid@ts` is already in `existKeys` (the dedupe set), stamps `batch`, appends to `S.samples`, writes `spl:<batch>`. **On any write failure the in-memory append is rolled back**, the key is deleted, and a readable error is thrown. Merges link info into `S.ifinfo`; unshifts a batch record `{id, file, ts, who, rows, added, coverage, checksum, src, intervalNote, entities, map, headers}`. Maintenance is **not** stamped here (live evaluation).
`commitStaged()`	Commits the staged file, then **streams** every other queued file with an identical header signature one at a time (reusing the confirmed mapping; freeing rows after each) so memory stays at one file (30 files × 40k rows no longer reaches GBs). Files with a different layout are deferred and staged next. A failure part-way keeps what committed cleanly, audits `upload.failed`, and stops. After success: audit `upload.commit`; clear analytics; optionally auto-merge hostnames; toast; next staged or route to Uploads.
`cancelStaged()`	Drops the staged upload.
`buildGroupView`	Builds the preview object: built samples, parsed header info, findings, the file group, signature.
`stageNext()`	Reads the first queued file, builds samples, stages it for preview; skips unreadable files with a toast.

12. Complete code reference — part C: analytics, forecast, charts

12.1 Classification and time helpers (lines 1565–1581)

Function	Behaviour
`classify(ifid,intf)`	Regex on the lower-cased **interface name**, first match wins: Core/Critical (`po`, `port-channel`, `core`, `spine`, `leaf`, `uplink` — preceded by start or a non-letter); Trading/Critical (`trad|market|exch|fix|feed`); Internet/High (`inet|internet|isp|wan|transit|border|edge`); Backup/Low (`back|bkp|repl|dr`); Storage/High (`stor|san|nas|iscsi`); Voice/Medium; Management/Low (`mgmt|mgt|oob|console`); Customer/High; default **WAN/Medium**. Class and impact can be overridden per link. Because the regex is on the interface name, "dr" or "po" fragments can misclassify — hence overrides.
`siteOf(dev)`	Prefix before the first `-` or `_` (2–6 letters), upper-cased; default `SITE-A`.
`th(cls)`	Global thresholds overlaid by that class's thresholds.
`stateOfT(util,t)`	null → `nodata`; ≥crit → critical; ≥high → high; ≥warn → warning; else normal. All comparisons are **≥**.
`stateOf(util,cls)`	Same using `th(cls)`.
`isBH(ts)`	True if the weekday is in `businessHours.days` and minutes-of-day ∈ [start, end) — start inclusive, end exclusive, minutes honoured. Uses local `getDay/getHours` on the wall-clock timestamp.

12.2 Identity merge (lines 1583–1708)

Function	Behaviour
`splitIfid(k)`	Splits `device|interface` at the first `|`.
`deviceAliasMap(devs)`	Union-find over device names: A and B are one host when B starts with `A + "."` (dotted prefix). The **shortest** name becomes canonical. Returns `{variant → canonical}`.
`deviceVariantFamilies()`	Counts samples per device, groups by canonical, keeps families with more than one member, sorts by size.
`applyDeviceMerge(map)`	Rewrites `dev` and `ifid` on every affected sample; re-keys `ifmeta, ifinfo, rca, comments, watch` (canonical link's data wins on conflicts; comment threads are concatenated); rewrites the affected batch slices in IndexedDB; recomputes batch entity counts; saves; clears analytics. Returns number of names merged.
`applyLinkMerge(map)`	`{oldIfid → keptIfid}`. First collects `ifid@ts` keys of links **not** being moved (duplicate guard). Each moved sample becomes the kept identity unless its timestamp already exists there, in which case it is dropped and counted. Re-keys the same side stores; rewrites touched batches; returns `{links, moved, dupes}`.
`allLinkIds()`	Every identity with count and first/last time, sorted — used by the merge pickers.

Function	Behaviour
`mergeLinksManual()`	Reads the two selects, validates, **warns if histories overlap** in time, confirms, applies, audits, toasts.
`mergeDeviceVariants()`, `allDeviceNames()`, `doMergeVariants()`, `setMergeAuto(v)`, `mergeDevicesManual()`	Auto-merge all families; list devices; button handler with confirm; checkbox handler (saves cfg, runs a merge immediately when turned on); manual two-device merge with no similarity guard.

12.3 inMaintWin and computeAnalytics (lines 1712–1892)

Documented stage by stage in section 3.3. Additional implementation details:

- `inMaintWin(x)` is O(windows) per sample: a window matches when `start ≤ ts ≤ end` and `scope==="all"` or the target equals the sample's `ifid` or `dev`. It is only called when at least one window exists.
- Threshold merge order uses `Object.assign`: `S.cfg.thresholds` ← `classThresholds[cls]` ← `ov.th`. Off-hours set = business set ← `thresholdsOff` ← `ov.thOff`; `tForS(x)` picks business or off-hours bands per sample (only when any off-hours band exists).
- The speed override rewrites each sample's `util`, `pu`, `bad` and `over` using `max(tx,rx)/override` and `max(ptx,prx)/override`.
- `speed` for the link = override, else the **last non-null** speed in time order (the current capacity).
- `provisionPeak` fallback chain: peak-column samples in window → utilisation in window → peak-column all history → `mx`.
- `dist` fractions use only samples with non-negative utilisation; `breachesBH` is counted against the business band set (`tRes.crit`).
- Anomaly list entries: `{ts, type, z, util, exp, jump}` for spikes; `{ts, type, util (weekend median), exp (weekday median)}`; `{ts, type, ratio}`.
- Weekend rule picks the first weekend sample at or above the weekend median as the anchor time.
- `dseries(valFn)` implements the window, optional market-hours-only, bucket-by-day, percentile, and min-sample filter (section 4.1). In **split** mode the per-direction series use `tx/speedTx` and `rx/speedRx`; the binding direction is the one with the smaller positive `t90` (null/negative treated as 1e9); the other direction's current and t90 are attached (`otherCur`, `otherT90`) and the note becomes "TX-bound · metric".
- If neither direction has data, it falls back silently to the combined series.
- The link record `rec` holds: identity (`ifid, dev, intf, site, cls, impact, tags`), capacity (`speed, speedTx, speedRx, capChange`), flags (`overCap, badCount, excluded`), thresholds (`th, thOff`), policy (`upgradeDays, upgradeNeeded, upgradeRule`), data (`samples, count, start, end`), statistics (`avg, p95, p99, mx, pkP95, pkMax, provisionPeak, dist, breachesBH, bhDays, baseStat, heat`), `anomalies, daily, forecast, risk`, plus `verdict` and `ai` text.
- Aggregation: `byDev`, `bySite`, `devices[]` and `sites[]` each with `risk` (60/40 blend), `critical` count; `infraRisk` for the whole estate.

12.4 Forecast code (lines 1893–1966)

- `rollingMedian(ys,w)`: centred window of `w` (half = (w−1)>>1) truncated at the edges; median via `percentile(…,0.5)`.
- `theilSen(pts,ci)`: see 4.2. Returns `{slope, inter, slopeLo, slopeHi}`; the interval percentiles use `a=(1-ci)/2`.

- `maeOf(fit,pts)`: mean absolute error against held-out points (Infinity when empty).
- `forecastFrom(daily,p95,cls)`: the browser build keeps a **busy-weekday** branch (fit weekdays only if their median > 1.35× weekend median + 0.02). It is unreachable because the preceding 7-day rolling median equalises weekday and weekend values (proved in `server/ARCHITECTURE.md`); the C# port omits it with identical output. Everything else is as section 4.
- The `cls` parameter is accepted but not used by the maths.

12.5 Scoring and narrative (lines 1967–2043)

- `riskScore(f)` and `verdictFor(r)`: section 3.4. Note `verdictFor` reads `S.cfg.planHorizonDays` and the link's own `th`. Its first rule uses `badCount`, but bad samples are already removed before `count` is taken, so `r.count+r.badCount` is the ingested count.
- `aiSummary(r)`: deterministic sentences: busy-period p95 and absolute rate; business days at High/Critical; monthly growth (|growth| ≥ 3 % else "flat"); days to 90 % if 0 ≤ t90 < 365 with the upgrade window, or "already at or above 90 %", or "no near-term capacity risk" when slope ≤ 0; anomaly count; excluded-artifact count; forecast confidence and model note. **No AI model is involved.**
- `aiSummarySimple(r)`: plain-English paragraph for managers (no jargon): health phrase from verdict level, "uses about N % of capacity", months/days to run low, critical-link remark.
- `A()` returns the cached analytics or computes them.

12.6 Charting engine (lines 2045–2200)

Function	Behaviour
`CHARTS`, `chartSeq`	Registry of live chart configs keyed `ch<n>`, so events can find the chart under the cursor.
`downsampleMM(d,maxPts)`	If more points than `maxPts`, bucket and keep the **min and max point** of each bucket (ordered by time). Spikes survive; rendered points are bounded (1,400 per series).
`svgLineArea(series,opts)`	Entry point. Series = `[{data:[[x,y,rawBps?]…], color, area, dash, wide, name}]`. Computes full x-range, registers the config, returns the chart `<div>` (SVG, zoom buttons, tooltip, hint).
`niceTop(v)`	Smallest "nice" top so the four gridline steps are round percentages: steps 0.5 %…50 % ×4 (e.g. 25 % steps → 100 %).
`chartSVG(cfg,animate)`	Axis top `y1` = option or `niceTop(min(max-in-view×1.08, 2))`; draws gridlines and % labels; threshold band lines (only if ≤ y1); optional **policy** line (amber dashed, labelled, "business hrs only" note); maintenance rectangles; business-hours rectangles per day in view; series paths clipped to the plot area (keeps one extra point each side while zoomed); event markers (anomaly dots, selected one larger with a pink guide line); 3 x-axis labels (start/middle/end, anchored inward); transparent hit rectangle; `.hg` group for the hover crosshair.
`redrawChart`, `chartAt`, `chartDataX`, `chartZoom`	Re-render without animation; find chart under an event; map pixel → data x; zoom about an anchor with span clamped between max(60 s, fullSpan/400) and full span, view window clamped to data.
`chartHover(cfg,e)`	Nearest point per series to the cursor; tooltip rows with colour swatch, series name, % and absolute rate (from point's 3rd element, or `% × speed`); crosshair and marker circles; keeps tooltip inside the chart.

Function	Behaviour
Document listeners	`mousemove` (pan or hover), `wheel` (zoom 0.78× in / 1.3× out, `passive:false`), `mousedown`/`mouseup` (drag-pan with a grabbing cursor), `dblclick` (reset).
`donut(dist)`	Four stroked circle arcs (radius 52) using `stroke-dasharray`; centre shows normal %.
`toRGBA(col,a)`	Hex or `rgb()` → `rgba()` with alpha, so heat cells keep text crisp.
`heatmap(heat,t,tOffP,speed)`	7×24 grid of mean utilisation by weekday × hour. Header with hour ticks every 3 h and a strip marking business hours; each cell coloured by the band of its value against **business** limits (inside market hours) or **off-hours** limits; alpha = 0.4 + 0.6×value; numeric % inside; business cells outlined; click shows a breakdown text (day, hour, business/off-hours, average and absolute rate, band and limits).
`calHeat(daily,t)`	One 13 px square per day coloured by the band of that day's **p95**, opacity 0.35 + 0.65×p95.
`sparkline(daily,color)`	120×26 polyline of daily p95; y scaled to the max (note: uses `Math.max(...ys)`, safe for ≤ ~365 days).
`riskBar(score)`	Bar + number; colour critical ≥75, high ≥50, warning ≥30, else normal.

13. Complete code reference — part D: navigation and views

13.1 Navigation (lines 2252–2264)

- `NAV` defines three groups: **Overview** (Executive, NOC, Digital Twin), **Analyze** (Capacity Planning, Upgrade Watch, Trend Analysis, Anomalies, Historical), **Operate** (Link Manager, Uploads, Maintenance, Reports, Audit Log, Admin).
- `renderNav()` paints the sidebar with an icon per item, highlights `S.view`, and shows a red count badge on NOC equal to the number of links whose verdict level is `critical`.
- `crumb(parts)` builds the breadcrumb; the last item gets class `here` (rendered as the large page title by CSS). Earlier parts are links carrying `data-nav` JSON that `go()` interprets.
- `needData()` — empty state with "Generate demo data" and "Upload files". Every view returns it when `S.samples` is empty.

13.2 Executive (view_exec, infraTrend, kpi, topNTable)

Element	How it is computed
Posture banner	`infra = a.infraRisk` (60/40 blend of weighted mean and worst link). ≥60 = "Elevated risk", ≥35 = "Stable with emerging risks", else "Healthy". Narrative counts critical links, links forecast to reach 90 % within 90 days (`t90 ≤ 90`) and names the fastest-growing link.
KPI row (4)	Infrastructure risk; critical links; links growing >20 %/month (`growthMoM > 0.2`); forecast breach in 90 days. Each KPI has a coloured state dot, not an arrow.
Fleet trend chart	`infraTrend(ifs)`: for each day, the **p95 across the links' daily p95 values** — a fleet-level sustained-load line (daily p95 by design). Threshold bands drawn from global thresholds.
Site risk list	Sites sorted by risk with risk bar, device and interface counts; click navigates.
Top tables	`topNTable`: top 8 by risk score and top 8 by growth; columns rank, interface, class, p95/MoM, risk/forecast-at-90-days, verdict pill.
`kpi(l,v,trend,t,glow)`	Renders one KPI tile (label, mono value, footnote with state dot).

13.3 NOC (view_noc, focusAnomaly, nocCard)

- **Attention queue** = links whose verdict level is not normal, sorted by impact rank (Critical→Low) then risk score; first 40 shown as cards (`nocCard`: level pill, impact chip, class chip, risk bar, title, reason, action, confidence).
- **Anomalies** panel: all anomalies of all links sorted by impact then newest; stored in `S.nocAnomalies` (plain data) for the dropdown. `focusAnomaly(idx)` shows a detail card (type, link, time, z-score or ratio) with an "Open interface" button; it can also point at a chart if `S.nocChartId` is set (currently null — anomaly pointing lives on the link page).
- **Maintenance note**: counts windows active *now* (`start ≤ now ≤ end`).
- **Fleet health**: counts of links per verdict level.

13.4 Digital Twin (view_twin, twinSite, twinDevice, led)

Three drill levels using `S.scope.twinSite` / `twinDev`: site tiles (risk bar, device/interface counts, up to 40 LEDs) → device tiles for a site → a device "front panel" of ports (one coloured port per interface) plus

a table. LED colour = verdict level; critical LEDs pulse. Risk colour thresholds: ≥75 critical, ≥50 high, ≥30 warning.

13.5 Link page (view_intf) — the largest view

Region	Details
Time filter state	`flt = S.scope.tf` = `{from, to, hours}`. `hasFlt` true if any is set. `inFlt(x)` tests time bounds and the Hours mode (business/off-hours). `fsam` = filtered samples. The filter changes charts, distribution, heatmap, business-vs-off-hours stats and the anomaly list, **not** the forecast (always full history; stated on screen).
Chart series	Transmit and Received are plotted **per sample against that sample's own capacity** (`x.tx/(x.speedTx r.speed)), with raw bps as the third element for tooltips. Peak series (pu) is plotted when peak columns exist (dashed pink). bwSub` subtitle shows TX p95, RX p95 and the highest peak.
Chart options	Threshold bands, maintenance shading, `speed` for tooltip, `bhShade:true`, anomaly `events` markers, and the **policy line** (`upgradeRule.utilPct`, noting business-hours-only).
Verdict banner	Level pill, impact chip, class chip, tags, IP and speed, risk bar, title, reason, action, confidence and comparison text. Buttons: Explain (RCA), Add to report, Export HTML, Export PDF, Policy hits .xlsx.
Threshold PDF row	Hours and Severity selects → `genBreachReport`. Shows "filtered range" chip if a From/To is set, else "(last 6 mo)".
Filter panel	From/To `datetime-local` limited to the link's span, Hours select, Apply, Reset, and a chip "filtered · n of N samples" (or "no samples match").
Notices	`exclNote` (maintenance/invalid exclusion with dates and fix); capacity-change note; over-100 % note with the Speed-override hint.
Business vs off-hours	`bhStats` splits samples by `isBH`; for each: count, mean, p95, max, state distribution (off-hours judged against its own limits). `bhCol` renders three KPIs and a stacked bar. `bhInsight` writes the text: ratio p95(BH)/p95(off) ≥ 1.3 = business-driven; ≤ 0.75 = off-hours hotter (backup-like); else even; adds critical-time remarks and the High/Critical business-day count.
Heatmap and donut	`heatmap(heatV,…)` and `donut(distV)` use filtered values when a filter is active (recomputed from `fsam`). `distExplain` writes a sentence (critical share, warm share, busiest window via `peakWindow`, growth).
Forecast panel	All hours / Market hours toggle; chart of historical daily metric, high and low confidence-band lines, mid forecast at 0/30/60/90/180 days; y-axis top = `niceTop(min(2, max(1, 1.06×max of all plotted)))`. KPI cells for days to 80/90/95 % with range (`fcell`: "now" if ≤0, "—" if null or ≥3650, range text "lo–hi" or ">1yr") and the upgrade window with confidence.
Provisioning target	Peak = `provisionPeak` (fallback chain), recommended capacity = `peak × speed × N`; states whether current capacity already meets it and how much of the 1/N ceiling is used.
AI summary	Technical or Simple text.

Region	Details
Link settings	Class, Impact, tags, business and off-hours thresholds, Speed override, Site override, custom fields. `saveIfMeta` stores only non-empty values; Reset deletes all overrides.
Risk factors	One bar per factor of `risk.factors` (already weighted) plus the impact multiplier.
Upgrade watch panel	`upgradeHits(r,start,end)` counts hits and distinct days **across the whole dataset** (not the rolling look-back); shows "trigger MET"; warns when many crossings are outside business hours and therefore excluded; a **monthly mini bar chart** (`monthlyPolicy`) of policy hits per month with "no data" months greyed (confirms every month has data); add/remove watch button.
Anomaly list and timeline	Dropdown to locate an anomaly on the chart (sets `cfg.selectedTs`, scrolls into view, brings it into the zoom window); `anomalyWhy` explains each type in words. Timeline (`buildTimeline`): monitoring start, first four anomalies, first Critical breach, maintenance windows touching the link, comments — newest first. Comment box → `addComment`.

13.6 Capacity, Trends, Anomalies, Historical

- **`view_capacity`**: links with a non-null `t90`, sorted by soonest (negative treated as 0); banner with the number due within 90 days and the lead-time statement; table of up to 60 links: current p95, slope per month (`slope×30×100 %`), time to 80/90/95 % with range under each, upgrade window, confidence; rows with `t90 ≤ 90` are highlighted; the All/Market hours toggle; CSV button.
- **`view_trends`**: top 10 growing and top 10 declining (`growthMoM` sign), and a table of the first 40 by growth with a daily-p95 sparkline (red up / green down).
- **`view_anomalies`**: every anomaly sorted by impact then newest; detail column shows z-score and expected value, or imbalance ratio, or util; CSV export.
- **`view_historical`**: pick any link (default highest risk) and see its rhythm heatmap and a deviation summary (busiest window, p95 vs average, growth flag).

13.7 Link Manager (view_links, lmSet, lmSetTh, lmSetThOff)

An editable table of every link (filter box applies on Enter): verdict pill, Class and Impact selects, business Warn/High/Crit inputs, off-hours W/H/C inputs, Speed override text, Site override, Tags. Each control writes into `S.ifmeta[ifid]` immediately, deleting keys when emptied and removing the whole record if empty, then `save`, `logAudit`, clear analytics, re-render. Edited links show ↺ to reset. Thresholds are clamped 0–200 % (`clamp(n/100,0,2)`).

13.8 Uploads (view_uploads)

- **Dropzone** (hidden while a file is staged) with a file input (`accept=.xlsx,.csv`, multiple) wired in `wireDynamic`.
- **Validation Console** for the staged file: row count, median interval, coverage; note about extra same-layout files; **column mapping table** (one row per field including custom fields; select of all headers; "unit detected" chip; confidence % bar or "manual"; required fields marked * and shown red if unmapped); findings as Blocking/Warning/Info blocks; **Parse inspector** table of the five highest-utilisation parsed rows (time, link, transmit, received, capacity, util, note); Commit button (disabled while blocking issues exist) showing the number of samples that will be added (rows minus overlap); Cancel; progress text.
- **Download processed data** panel: link select, From/To, Excel and CSV buttons, explanatory text.

- **Upload history** table: file, uploaded time, by (role), coverage, rows, added, interfaces, median interval, delete (✕ → `removeBatch`). A warning appears above 800,000 in-memory samples (estimate ≈ samples/1200 MB).

13.9 Processed export, Maintenance, Upgrade Watch, Reports, Audit

Maintenance (view_maintenance, addMaint, delMaint)

A table of windows (scope "All" or target name, start, end, reason, who, delete) and a form: Scope select (All interfaces / each Device / each Interface), Start, End (`datetime-local`), Reason. `addMaint` converts the inputs with `new Date(value).getTime()` (browser-local wall clock, consistent with as-written data), stores `{scope:'all'|'dev'|'if', target, start, end, reason, who}`, saves, audits, clears analytics, re-renders. `delMaint(i)` splices and does the same.

Upgrade Watch (view_upgrade and helpers)

Item	Behaviour
`WATCH_WINS`	`all, 30, 60, 90, 120, 180` day windows. `setWatchWin` stores the choice in `localStorage` (`np-watchwin`, wrapped in try/catch).
`upgradeHits(r,winStart,winEnd)`	Counts samples with util ≥ policy % (market hours only if the policy says so) inside the window: hits, distinct days (`toDateString`), last hit time, peak, threshold, whether the day count reaches the required days (`met`). **Whole-window count, not the rolling look-back used by the verdict.**
`monthlyPolicy(r)`	Per calendar month (key `yyyy-mm`): samples, policy hits, distinct hit days, peak; **gap-filled** between first and last month so months with no data appear as empty stubs.
`watchAdd/AddAll/Remove/Clear`	Maintain `S.watch` (array of ifids), persist to IndexedDB `watch`, audit, render.
Capacity-review table	Links with `overCap` sorted by max utilisation: capacity in data, count over the flag level (and % of samples), max, when, and a "Review & set capacity" button that opens the link page (where the Speed override lives).
Watched table	Window tabs, add-link select, Add all, Clear; KPIs (links watched, threshold hits, meeting trigger, policy); a row per watched link with hit count bar (red if trigger met), days hit `n/required`, peak, last hit, trigger pill, remove. "Export all watched to Excel" calls `genUpgradePolicyXlsx('all')` (note: this exports **all links**, not just watched ones).
`winEnd_`	Identity helper (returns `dataEnd`).

Reports page (view_reports)

- Links are ranked for the scope pickers: upgrade-trigger-met first, then highest p95, then alphabetical; each option label includes the sample count and p95 so a tiny hostname-variant stub is obviously a stub.
- Panels: multi-link pack (`bulkPanel`), Regulatory Capacity Report (scope + Excel/PDF/HTML), Upgrade Policy Hits Excel (scope + Generate), and seven quick reports (Executive Summary, Weekly Capacity, Monthly Trend, Top Busy, Top Critical, Idle, Business Impact) each with a Generate button.

genReport(kind,id) and exportReport

Kind	Content
exec	Narrative (infrastructure risk, sites, interfaces, critical count, forecast-to-90 % count) + top 10 by risk.
capacity	Up to 15 links with positive t90 sorted soonest: p95 (+absolute), MoM, time to 90 %, upgrade window, confidence.
trend	Top 15 by growth: MoM, p95.
busy	Top 15 by p95: p95, p95 bandwidth, peak (`mx`).
critical	Critical-verdict links by risk.
idle	Links with p95 < 5 %.
impact	Non-normal links by impact then risk (top 20).
intf	Single-link AI summary.

The result is rendered into `#reportOut` with Export HTML (`exportReport`: a dark-themed standalone HTML) and PDF (`window.print()`). `view_audit` lists the audit events (when, who/role, action chip, target, detail) with CSV export.

14. Complete code reference — part E: Excel writer and Excel/regulatory exports

14.1 Minimal XLSX writer (lines 3216–3353)

A real `.xlsx` is a ZIP of XML parts. The writer builds each part as a string, then packs them with a hand-written ZIP encoder. No library.

Function / constant	Behaviour
`CRC_T`, `crc32(u8)`	Standard CRC-32 (polynomial 0xEDB88320) table and function, required by the ZIP format.
`zipStore(files,mime)`	Builds local file headers (signature 0x04034b50, version 20, method 0 or 8, DOS time/date from "now", CRC, compressed/uncompressed size, name), writes names and bodies, then the central directory (0x02014b50) and End-Of-Central-Directory (0x06054b50). Entries with a pre-deflated `z` buffer use method 8; others are stored. Returns a Blob of the requested MIME type (xlsx by default; `application/zip` for the PDF zip). Does not set the UTF-8 flag, so file names must be ASCII.
`zipDeflate(files,mime)`	Compresses each entry with `CompressionStream("deflate-raw")`, uses the compressed bytes only if smaller, falls back to stored on any error.
`xmlEsc(v)`	Strips illegal control characters then escapes `& < > "`.
`colL(i)`	Zero-based column index → letters (A…Z, AA…).
`XL_STY`, `XL_STYLES`	Style table: `hdr` (white bold on navy #0F2740, wrapped), `pct` (0.0 %), `dt` (yyyy-mm-dd hh:mm:ss), `title` (14 pt bold), `bold`, `date` (yyyy-mm-dd), `int` (#,##0 — format id 3), `note` (italic grey 10 pt), `num1` (0.0), `wrap`, `pct2` (0.00 %). A cell is a plain value or `{v, s:'pct'}`; unstyled sheets are written plainly.
`sheetXML(rows,sh)`	Rows → `<sheetData>` using **inline strings** (`t="inlineStr"`) and numeric `<v>`; empty/null cells skipped; optional frozen header pane (`freeze`), column widths (`cols`), `autoFilter` (`filter`), and a `<drawing>` reference when the sheet has charts. Non-finite numbers are written as text.
`xlQ`, `xlRels`	Quote a sheet name for formulas (`'Name'` with doubled quotes); build a relationships part.
`chartXML(c,n)`	Native DrawingML chart. `type:'line'` (optional dashed series, marker-only series with circle markers) or `'col'` (clustered or stacked, overlap 100). Series names and values are **cell references** (`strRef`/`numRef`), so Excel charts the sheet's own data. Category axis labels rotated −45° with optional `tickLblSkip`; value axis starts at 0 with a number format (`yFmt`, e.g. `0%`); legend at bottom; element order follows the schema.
`drawingXML(charts)`	One two-cell anchor per chart (from/to column-row) with a graphic frame referencing `rId<k>`.
`xlsxParts(sheets)`	Assembles parts: sheets, per-sheet rels, charts, drawings, drawing rels, `[Content_Types].xml`, `_rels/.rels`, `workbook.xml` (+ `_FilterDatabase` defined names for filtered sheets), workbook rels, `styles.xml`. Absolute-reference conversion for the filter uses a regex replace.

Function / constant	Behaviour
`xlsxBlob`, `xlsxBlobZ`	Stored vs deflated workbook. The pack uses the deflated version because sheets can be huge.

14.2 genUpgradePolicyXlsx(scope) — step by step

1. Guard: no data → toast. `list` = all links or the one matching `scope`.
2. **Sibling check** (single link only): look for another link with the same interface name whose device is a dotted-prefix variant and which holds more than max(8, 2×) as many samples; if found, build a "Likely wrong link selected" note, repeated atop every sheet and in the toast.
3. Time filter: `tfP = activeTF(scope)` for a single link — the on-screen From/To filter bounds both the rows and the stated coverage; `inTF(x)` tests it.
4. Header rows: title, generated time, scope, coverage and days ("filtered range — matches the From/To filter" or "entire loaded history"), a line explaining the counts-vs-verdict difference, the policy wording (Market-hours-only vs All-hours), optional capacity-change note, optional sibling note.
5. Per link: `allCross` = analysed samples with util ≥ policy % in the filter; `mkt`/`nonMkt` split by `isBH`; `policyHits` = `mkt` if business-hours-only else `allCross`; `daysHit` = distinct dates of `policyHits`; `met = daysHit ≥ ur.days`. Links sorted by crossings then p95.
6. Summary row per link (columns in section 7.2); `lastHitTs` = latest of `policyHits`; "Data through" = end of the link's data (or the filter end).
7. Detail rows up to **50,000** (`MAXH`), then a truncation note and stop; per-day rollup sheet from `allCross` grouped by local date (market/non-market counts, peak, first/last time).
8. Zero crossings: explicit "— No threshold crossings —" rows on all three sheets with the reason and coverage days; a single-link tip suggests "All links".
9. Output: `xlsxBlob` of three sheets (Summary, Hits by Day, Hits by Date-Time), named `NetPulse_Upgrade_Policy_<AllLinks|Dev_Intf>_<yyyy-mm-dd>.xlsx` via `dl`; audit `report.export.xlsx`; informative toast.

14.3 Regulatory report code

- `timeToLevel(f,L)`: 0 if forecast current ≥ L, else `(L−current)/slope` if slope > 1e-5, else null.
- `regReportData(scope)`: for each link computes market-hours samples, samples ≥ T overall and in market hours, days ≥ T, `tReg`, `augment = p95 ≥ T || (0 ≤ tReg ≤ 180)`, peak (`provisionPeak` or fallback), headroom to T and to 100 % (percentage points of p95), % of time over T (all and market). Sorts by p95 descending; aggregates counts of links at/above T and needing augmentation; coverage days.
- `tzLabel()`: "IST" for Asia/Kolkata, otherwise the short zone name via `Intl.DateTimeFormat` (falls back to the zone id).
- `fmtIST(ts)`: "d Mon yyyy, hh:mm TZ" from the **wall-clock** fields — the label is just attached, no conversion (data timestamps are wall-clock in the configured zone).
- `genRegReport(scope,fmt)`: for **xlsx**, four sheets (Cover, Capacity Summary, Threshold Breaches, Definitions); the summary range column uses the slope interval: earliest = from `slopeHi`, latest = from `slopeLo`, printed "lo–hi" with ">365" when the latest is ≥ 3650 days; "AT/OVER" when already at/over; "no trend" when no positive slope. Recommended capacity = `peak × speed × N` (Mbps); headroom check `peak×N ≤ 1`. Breaches sheet capped at 50,000. For **html/pdf**: a styled document with executive position call-out, ranked table (flagged rows shaded), data-integrity table, methodology, alignment, disclaimer and a three-column signature block; HTML is downloaded, PDF opens a new window and calls `print()` (so the user picks "Save as PDF"; pop-ups must be allowed).

- Helper exports: `fmtTsPlain`, `csvCell` (collapses whitespace; quotes if needed), `exportTableCSV(sel,name)` (reads a rendered table's cells), `csvBtn`, `dl(blob,name)` (object URL + temporary anchor, prefixed with the workspace tag), `exportAll` (JSON dump of the workspace).

14.4 RCA helpers and report launchers (lines 3601–3710)

Function	Behaviour
`rcaAnswers(r)`	Six auto-answers: **When did it start?** (first sample ≥ Critical band, else "No hard breach; elevated trend"); **Recurring?** (≥3 business days at High/Critical); **Business hours?** (`breachesBH` > 0); **Growth trend?** (monthly % and rising/flat); **Related interfaces?** (up to three other links on the same device with highest risk); **Maintenance correlation?** (any window targeting the link or device).
`rcaCause(r)`	Rule-based likely cause: growth > 10 %/month with ≥3 business days → "Organic business-hour demand growth"; weekend anomaly → "Newly scheduled off-hours job (backup/replication) or misclassification"; imbalance anomaly → "Asymmetric routing / unidirectional flow"; max util > 90 % → "Localized burst / elephant flow"; else "Elevated utilization consistent with steady load".
`askRCA(id)`	Renders an editable RCA box under the verdict: answers, a "Likely cause" textarea and a "Next steps" textarea (defaults to the verdict action). `saveRCA` stores `{cause, steps}` in `S.rca[ifid]` (IndexedDB `rca`); `resetRCA` deletes the override. Edited text is used in exports.
`applyTimeFilter()` / `clearTimeFilter()`	Store/delete `S.scope.tf = {from, to, hours}` (strings from the inputs, audited) and re-render.
`setAiMode(m)`	Technical/Simple, remembered in `localStorage`.
`genBreachReport(id, hours,bands)`	Builds `breachReportHTML` and opens it in a new window, then calls `print()` after load (350 ms). Pop-ups must be allowed.
`activeTF(ifid)`	Returns `{from,to,label}` only when the open link page matches `ifid` and a From/To is set; otherwise null. This is how exports inherit the on-screen filter.
`exclKv(r,kv)`	Builds the "Excluded from this analysis" row for reports from `r.excluded`.

14.5 linkReportHTML and exportLink

- `exportLink(id,fmt)`: if a time filter is active, **recomputes the link through the identical analytics pipeline** (`computeAnalytics({from,to})`) restricted to that window, so p95, forecast, risk and verdict all describe the filtered period. HTML downloads a file; PDF opens a print window.
- `linkReportHTML(r,tf)`: light-theme printable page: header (device·interface, link name, IP, site, timestamp); reporting-window box (states when the filter applies and that all figures are computed over it); chips (tags, class, impact, speed); verdict banner (tags stripped from the reason, next steps from RCA override or verdict action, confidence); custom-field table; plain-terms paragraph and technical summary; **key metrics chart** (v67: `hitSeries` daily **peak** and **average** — a p95 line would hide short bursts) and table (p95 with absolute rate, average, peak, time-in-state, business-day breaches); capacity forecast table (current, time to 80/90/95, 90 % range, upgrade window, growth, confidence/model note); RCA answers plus cause/next steps; risk-factor bars with the impact multiplier; footer.

14.6 breachReportHTML — complete walk-through (lines 3757–3893)

1. Window: `tf` or last **183 days** ending at the link's end (`winDays=183`). Hours label, severity label/colour/threshold chosen from `bands`.
2. `hitCalc(...)` produces `breaches, cW, cH, cC, dW, dC, peak, peakTs, byDay, mWarn/mCrit/nWarn/nCrit`. `warnPlus = cW+cH+cC`.
3. Chart (only if ≥2 points): `hitSeries` points mapped to `{d, p95:=peak, avg}`; y-max = `max(crit×1.12, max×1.12, 0.01)`; five horizontal grid lines with % labels; up to six x-axis date labels; threshold lines for Warning/High/Critical (only if ≤ y-max) labelled in the right margin; area under the peak line (7 % opacity); average line (light blue); peak line (dark blue); the **peak point marked** with its value; axis titles ("Date (tz) · n days plotted", "Utilization (daily peak)").
4. Parameter table: link details, reporting window (and whether it matches the filter), hours basis, severity bands included, market-hours window, thresholds (market, and non-market if present), samples evaluated, exclusions.
5. Summary cards: hits at/above the lowest selected band (with distinct days), at/above Critical (with distinct days), peak utilisation with time. Band legend counts. When Hours = All, a small table splits market vs non-market hits.
6. **Per-day exceedance table**: date, per-band counts (only selected bands), peak, peak time, first–last hit time.
7. **Detailed log**: first **3,000** hits (`MAXDET`) with a note that the full evidence is in the Excel exports; each row has index, timestamp (with seconds), band pill, utilisation (2 decimals), transmit/receive (formatted bps), market/non-market.
8. Signature block (prepared/reviewed/approved) and footer stating the window, timezone, and that maintenance is excluded. CSS `@page` bottom-centre footer shows institution, "Confidential", and page X of Y when printed.

14.7 Multi-link report layer — bulkWindow, hitSeries, bulkHit, bulkPrep

Covered conceptually in 7.6; implementation notes:

- `bulkWindow(list,period,cFrom,cTo)`: `end = max(r.end)`, `start0 = min(r.start)` over the selected links; `custom` uses the inputs (falling back to data bounds); `all` = full span; numeric = `max(start0, end − N days)`.
- `hitSeries(r,{winStart,winEnd,hours})`: `step = span ≤ 16 d ? hourly : ≤ 100 d ? 6-hourly : daily`. Buckets key on the local hour / 6-hour block / local midnight. Each bucket keeps `peak` (max util), `sum`, `n`; output points `{t, peak, avg, n}` sorted by time. The Hours filter is applied before bucketing.
- `bulkHit(r,o)`: returns `{name, thr, text}`. Policy: threshold = link's policy % (name "Policy"). Severity: the link's own `th[k]` (market-hours value) as the chart/marker threshold with descriptive text.
- `bulkPrep(r,o)`: runs `hitCalc`; counts samples in the window for coverage (first/last/n) — **independent of the hours filter**, so "no data" really means no data; builds the series if graphs are on.
- Formatters: `bT` (yyyy-mm-dd hh:mm:ss), `bD`, `bHM`, `bPc` (0–1 → "62.5%", trailing zeros trimmed), `bPk` (always one decimal). `HOURS_LBL` and `PB` (palette and band labels).

14.8 PDF writer (lines 3905–3993)

Part	Behaviour
`PDF_W`, `PDF_WB`	Helvetica regular/bold advance widths per 1000 units for ASCII 32–126 (metric-compatible with Liberation Sans) used for **text measurement**, wrapping, alignment and truncation without embedding fonts.

Part	Behaviour
`CP1252`, `PDF_ASCII`, `pdfBytes(s)`	Maps a JS string to WinAnsi bytes: printable ASCII and Latin-1 (160–255) pass through; Windows-1252 punctuation (€ … – — • ’ “ ” etc.) is remapped to its code; ≥ ≤ → ← − ✓ ⚠ ▲ ▼ become ASCII text (">=", "<=", "->", "-", "v", "!", "^", "v"); anything else becomes "?" (tabs/newlines become spaces).
`pdfEsc`, `pdfInfoStr`	Escape `\ ( )` in PDF strings; Info/bookmark strings are **ASCII-only** (PDFDocEncoding differs from WinAnsi — an em dash once rendered as "Š").
`pdfWidth(s,size,bold)`	Sum of glyph widths × size / 1000 (bullet 350, em dash 1000, other high codes 556).
`pdfDeflate(u8)`	`CompressionStream("deflate")` (zlib stream) for FlateDecode; returns null if unsupported (stream written uncompressed).
`class PdfDoc`	A4 (595.28×841.89 pt). Pages are arrays of drawing operators. Coordinates are **top-left origin, y down**; converted with `H − y`. Methods: `addPage`, `mark(title)` (bookmark), `rect` (fill/stroke/both), `line` (dash optional), `poly` (round joins/caps), `circle` (4 Bézier arcs, κ=0.5523), `text` (baseline y, left/right/centre, `maxW` truncates with ".."), `wrap`, `para`, `tw`. Colour hex → cached "r g b" strings.
`build()`	Applies the footer callback to every page (so "Page i of n" is final), numbers objects (1 Catalog, 2 Pages, 3 Helvetica, 4 Helvetica-Bold, 5 Info, 6… Outlines, then content/page pairs), writes the header with a binary comment, each page content stream deflated, the xref table with 10-digit offsets, trailer and `startxref`. Returns a `Blob` of `application/pdf`. Bookmarks use `/Dest [page /Fit]`; `/PageMode /UseOutlines` opens the bookmark panel.

14.9 pdfChart, pdfLinkReport, pdfFooter, pdfCover, bulkPdfFiles

- `pdfChart(doc,x,y,w,h,P)`: framed plot with title stating resolution; margins L36/R66/T20/B34; y-max = `niceTop(max(peak×1.06, crit×1.08, hitThr×1.06, 0.05))`; 4 grid steps with % labels; 6 x ticks with date labels (month-year when span > 400 days, day-month when > 3 days, else with time); threshold dashed lines (Warning, High, Critical, plus Policy in policy mode) with labels in the right margin nudged ≥8 pt apart; **series drawn in segments that break wherever the gap exceeds 2.5× the bucket size** (a polling outage never looks like a flat line); average (light blue, 0.8 pt) then peak (blue, 1 pt); red dots wherever bucket peak ≥ hit threshold; legend row.
- `pdfLinkReport(doc,P,{bookmark})`: first-page header (institution, title, link, generated stamp, navy rule) and continuation header on later pages; helper closures `need(h)` (page break), `h2`, `kv` (wrapped values). Sections: class/impact/site/capacity chips; **Link details** (two-column, custom fields included); **Reporting parameters** (period, hit definition, hours basis, market window, thresholds market/non-market, samples evaluated, data coverage — amber if none, **upgrade-policy status** with trigger result, exclusions, capacity review, capacity change); **Summary** (four cards: HITS, CRITICAL, DAYS WITH HITS, PEAK UTILISATION — greyed with "no samples in this period" when there is no data; band legend with counts; market vs non-market line); optional **graph**; optional **Hits by date** table (columns adapt to selected bands); optional **Hits by date-time** (first 10,000, pill-coloured band column); signature block; footnote. Tables repeat their header on every page.
- `pdfFooter`: rule plus "Institution · Confidential · Threshold Hit Report" and "Page i of n".

- `pdfCover` / `bulkCoverPages`: cover capacity 48 rows on page 1 and 60 on each next page; the **cover is rendered last and spliced to the front** so link pages could be numbered first (page numbers are final); bookmarks shift by the cover length. Columns: #, link, hits (or "no data"), critical, days, peak, policy (MET/—), start page.
- `bulkPdfFiles(preps,o,mode,onStep)`: mode `one` (and >1 link) builds one combined doc with a "cover page count mismatch" safety check; otherwise one document per link with collision-free file names `Threshold_Hits_<dev>_<intf>_<from>-<to>` (non-word characters → _, duplicates suffixed _2…).

14.10 bulkXlsx, bulkPanel, bulkGenerate

- `bulkXlsx`: sections 7.6; helper functions `xlSerial` (wall-clock → Excel serial using UTC fields, so cells show the same clock time), `xlDaySerial`, `xlDT`, `xlName` (sheet-name sanitiser, unique case-insensitively), `xlRef` (absolute reference), `xlHdr`. Summary filter range `A10:…`; frozen at row 10. Per-link line chart: series Peak, Average, "Peak at/above hit threshold" (marker-only), Warning/High/Critical (dashed) and Policy; x tick skip = `ceil(n/14)`; chart anchored beside the table; no chart if fewer than 2 points.
- `bulkPanel(links)`: link picker rows (checkbox, name, class·capacity·p95, ⚠️ upgrade tag) with `data-q` search text and `data-flag`; options as in 6.3. Helpers `mlCount`, `mlFilter`, `mlSel(all|flag|none)` (acts only on visible rows), `mlPeriod` (shows/hides custom range).
- `bulkGenerate(kind)`: validates selection and window; builds `o`; **yields to the UI between links** (`setTimeout 0`) while preparing; PDF: separate downloads are spaced 400 ms apart and capped at 25 (above that → zip, noted in the toast); single PDF downloads directly; zip uses `zipStore(..., "application/zip")` of uncompressed PDF bytes. Excel: `xlsxBlobZ(bulkXlsx(...))`. Status line warns about links without data; audit entry records every option.

15. Complete code reference — part F: handlers, glossary, boot

15.1 Interaction handlers (lines 4439–4545)

Function	Behaviour
`saveIfMeta(ifid)`	Reads the link-settings form: class, impact, tags (comma list), speed, site, business thresholds (`clamp(n/100,0,2)`), off-hours thresholds (only non-empty), custom fields. Stores `S.ifmeta[ifid]`, audit, clear analytics, toast.
`resetIfMeta`	Deletes overrides.
`addComment(id)`	Appends `{ts, who, txt}` to `S.comments[id]`; saves; audits.
`addMaint`, `delMaint`	See 13.9.
`saveThresholds`	Writes global bands (percent → fraction), off-hours set (or null if all blank), flag level (`clamp(mu/100, 0.5, 10)`).
`addCustomField` / `removeCustomField`	Validates name; key `cf_…`; refuses duplicates and clashes with built-in keys; removal keeps stored values.
`savePlanning`	Business start/end **hour** (minutes untouched), planning horizon, lead time.
`saveTimezone`	Saves zone and input mode; warns that already-ingested data needs re-upload to change interpretation.
`saveForecastCfg`	Metric, direction, `provisionFactor = max(1, n 1.5), peak basis, CI (clamp(x/100, .5, .99)), min-day samples (max(1, n|0)), window days (max(0, n|0)`.
`setForecastHours(v)`	`bh` or `all`; audit; clear analytics.
`saveClassThresholds`	Collects the 10-class × 3-band table, only non-empty entries.
`saveCompliance` / `setMarketHours`	Compliance fields (threshold clamp 10–100 %, retention default 180). Market preset sets `{9,15,15,30,[1–5]}`.
`saveUpgrade`	`utilPct` clamp 1–100; `days` ≥ 1; `lookbackDays` ≥ 0; BH-only checkbox; mode.
`remap(field,idx)` / `reStage()`	Mapping dropdown handler: sets map entry, confidence 1, marks manual, rebuilds the staged samples and validation with the new mapping.
`removeBatch(id)`	Confirm; removes samples of that batch, record, IDB slice; audit; re-analyse.
`resetAll()`	Confirm; deletes every `spl:` slice and legacy `samples`; clears samples, batches, maintenance, comments; (config and audit are kept despite the confirm text).
`handleFiles(files)`	Sets the upload queue and stages the first file. (`handleFilesOLD` is a dead, unused earlier single-file handler.)

15.2 Demo data genDemo (lines 4544–4573)

Creates 14 days of hourly data for 2 sites × 3 devices × 8 interfaces (Core, Trading, Internet, Po10, Backup, Storage, Customer, mgmt) with class-specific speeds (100G/10G/40G/1G), base load and growth (trading/internet/customer grow 35–85 %), business-hours multiplier 1.35 (+0.35 for Trading), off-hours ×0.5, weekend backup bursts (80–95 %), injected Sunday spikes on Core links, noise, and peak columns. A

deterministic LCG (`seed=42`) is used. Rows are fed through the **real** `resolveColumns` and `buildSamples`, so the demo exercises the true ingestion path. Batch id `b<now>` (note: not the `_rand` form).

15.3 Router and presentation helpers (lines 4575–4727)

- `VIEWS` maps view ids to functions (`site` redirects to the twin). `go(view,scope)` sets view/scope and renders; `navTo(obj)` interprets `data-nav` JSON (`twin` object, or `{view,id}`).
- `staggerReveal()` adds the `reveal` class with delay `min(i×45, 520) ms` to panels, KPIs, verdicts and tiles; `animateCounters()` counts KPI integers/percent up over 760 ms with a cubic ease-out. Both are skipped under reduced motion.
- **Glossary** (`GLOSSARY`, ~60 entries ordered specific→general; first key contained in the label wins) and `annotateInfo()`: after every render, adds an ⓘ with `data-tip` to table headers, KPI labels, section titles and distribution rows; a document-level `mouseover` shows the `#gtip` tooltip clamped inside the viewport (below the element, or above if it would overflow). Elements can opt out with `data-noinfo`.
- `renderWsPick()` fills the workspace selector with a final " + New workspace…" option; the handler resets the select to the current value before acting.
- `paintStorage()`: async; shows used / quota, percentage, workspace count, total samples stored, "🔒 persistent" or "best-effort", a bar coloured by level (≥90 critical, ≥75 high, ≥50 warning) and a warning at ≥75 %.
- `paintIcons()` fills `.ico[data-ic]` slots. `render()`: icons → workspace picker → nav → **clear `CHARTS`** → call the view inside try/catch (a view that throws shows a "Render error" panel instead of a blank page) → re-trigger the swap animation → reveal, counters, info marks → `wireDynamic()` (dropzone and file input events) → `paintStorage()`.

15.4 Global events and boot (lines 4738–4781)

- One delegated `click` handler: heatmap cell selection (shows its breakdown), chart zoom buttons (`[data-cz]`, ×0.6 in, ×1.7 out, reset), sidebar item (`[data-view]`), `data-nav` and `data-twin` navigation.
- Role select (audited), Demo, wall mode (`requestFullscreen`), theme toggle (class `light` on `<html>`, remembered; re-renders so SVG colours refresh), ⌘/Ctrl+K focuses search, Enter runs `handleQuery`.
- `handleQuery(q)`: **grounded, rule-based** search: link whose device **and** interface appear in the text → open it and toast its verdict and p95; else device only; else interface only; "critical/worst/risk" → highest-risk link; "capacity/forecast/upgrade/saturat" → Capacity view; "anomal" → Anomalies; otherwise a hint. No external AI.
- **Boot IIFE**: apply saved theme (try/catch) → open default DB → read active workspace id and the registry (fall back to default if they disagree) → `loadAll(act)` → render.
- Final line exposes the functions referenced by inline `onclick` attributes on `window` (genDemo, go, commitStaged, genReport, exportReport, exportAll, askRCA, addComment, addMaint, delMaint, saveThresholds, saveIfMeta, resetIfMeta, saveRCA, resetRCA, setAiMode, exportLink, genUpgradePolicyXlsx, genRegReport, saveCompliance, setMarketHours, saveForecastCfg, setForecastHours, saveClassThresholds, exportTableCSV, setWatchWin, watchAdd/AddAll/Remove/Clear, focusAnomaly, focusAnomalyIntf, applyTimeFilter, clearTimeFilter, lmSet, lmSetTh, lmSetThOff, removeBatch, savePlanning, saveUpgrade, doMergeVariants, setMergeAuto, mergeDevicesManual, mergeLinksManual, wsCreate/Switch/Rename/Delete, cancelStaged, remap, resetAll) and `S`. Note: some handlers used from inline attributes (e.g. `exportProcessed`, `bulkGenerate`, `mlSel`) are plain top-level `function` declarations and are therefore global anyway in a classic script.

16. Complete code reference — part G: server port, tests, scripts, deployment

16.1 server/ — the C# analytics port

File	Content and behaviour
`NetPulse.sln`	Solution tying the library and test project together.
`src/NetPulse.Analytics/NetPulse.Analytics.csproj`	.NET 8 class library (no external packages).
`Stats.cs`	`Clamp`; `Percentile` (sorted copy, linear interpolation, identical to JS — empty → 0); `Mean`; `Max`/`Min` (0 when empty, mirroring `amax`/`amin`); `RollingMedian` (half-window `(w-1)>>1`, clipped, median via `Percentile`). Deliberately literal translations: where an idiomatic C# form could change a rounding boundary the JS shape is kept.
`Forecast.cs`	Records `DailyPoint(DateMs,P95)`, `TheilSenFit(Slope,Intercept,SlopeLo,SlopeHi)`, `ForecastConfig(Ci=0.8, UpgradeLeadDays=42)`; class `ForecastResult` (slope, current, t80/90/95 with Lo/Hi, growth, confidence, projections at 30/60/90/180 with Lo/Hi, window, R^2, CI, slope bounds, model note). `Forecast.TheilSen` (stride `max(1,floor(n/45))`, pairwise slopes where dx>0, median slope, median intercept, CI percentiles), `MeanAbsError`, `Forecast.From` (steps in 4.2; `SlopeCeiling=0.04`, `FcMax=1.5`). The busy-weekday branch is intentionally not ported (see `ARCHITECTURE.md`). Rounding uses `MidpointRounding.AwayFromZero` where the JS used `Math.round` (the C# default would round half to even).
`Risk.cs`	`RiskWeights` record, `RiskInput`, `RiskResult`; `Risk.Score` (six factors, curve `clamp^1.4`, impact multipliers 1.25/1.05/0.85/0.65, throws on an unknown impact); `Thresholds`, `UtilState`, `States.Of` (inclusive at the lower edge). **Note:** the record's default weights (0.30, 0.15, 0.20, 0.15, 0.12, 0.08) are the values pinned in the golden fixtures and **differ from the browser's shipped defaults** (0.9, 0.7, 0.8, 0.7, 1.0, 0.6). The score depends only on the weight *ratios* being consistent with the caller's configuration, so a real service must pass the configured weights explicitly.
`tests/…/Golden.cs`	Loads fixtures from `golden/` (copied to the output directory), tolerance `1e-9`, helper `OptDouble` for nullable JSON numbers.
`tests/…/ParityTests.cs`	xUnit: `Percentile` at ten quantiles; mean/max/min; rolling median for windows 3, 7, 15; Theil–Sen at CI 0.5/0.8/0.95; **one theory case per forecast fixture** (25 cases: flat, linear growth, decline, busy weekday, noisy growth, step change, saturated, too short, single spike, two years, clamp boundaries, weekday ratio boundaries, near-zero, exactly 5 and 8 days, over 100 %, slope above/below ceiling, upgrade-window floor, weekly cycle, over-capacity growing/flat); one theory case per risk fixture (8 cases). Asserts slope, current, growth, confidence, R^2, model note, every t-value (null vs value), projections and bands, and the window.

File	Content and behaviour
`golden/forecast.json`, `risk.json`, `primitives.json`	Outputs captured from the browser engine by `scripts/capture-golden.mjs`; they define "correct" for the port.
`db/001-schema.sql`	SQL Server schema (never executed). Tables: `Site`, `Device`, `Interface` (internal bigint id, never `ifIndex`; class, impact, `ScopeKey` for future scoping), `InterfaceAttributeHistory` (effective-dated speed/class/impact), `UploadBatch` (Sha256 unique for committed batches, status check), `Sample` (clustered on `(InterfaceId, TsUtc)`, page compression, per-direction rates and peaks, capacity in force, `UtilPct`, `PeakUtilPct`, `QualityFlags` bit0 invalid / bit1 over-capacity), `MaintenanceWindow` (evaluated at query time, never stamped), `RollupDay` (avg/max/min/p95/p99, business-hours p95, seconds in each state, excluded count), `LinkAnalytics` (results with `EngineVersion` for reproducibility), `AuditEvent` (append-only by grant).
`ARCHITECTURE.md`	Decision record (ASP.NET Core 8 + SQL Server; thousands of links / 100M+ samples; everyone sees everything; local Identity accounts), the parity strategy, the 1.5 projection ceiling change, the dead busy-weekday branch finding, data-model notes, division of work (SQL for aggregation, C# for light statistics), security notes, status table and what was deliberately left out (partitioning, hourly tier, dirty-set rollups), open questions (retention, server-side report rendering).

16.2 Scripts

Script	Behaviour
`scripts/regression.mjs`	Playwright + Chromium (path from `$CHROMIUM` or `/opt/pw-browsers/chromium`; falls back to a global playwright install). Opens `diff/netpulse.html` in the `Asia/Kolkata` zone, captures `pageerror` and console errors, auto-accepts dialogs. **Seeds** 5 links × 200 days hourly (deterministic LCG, per-class bands, a 0.25 burst with ~1.5 % probability) directly into `S` and IndexedDB, then checks: every view renders without errors; a link page renders; exports build (policy xlsx, regulatory xlsx, Threshold Report HTML, link HTML — `dl` is stubbed during this block and restored); maintenance hides data and says so, deleting the window restores it exactly; link merge re-keys samples (moved + duplicates = 4,800); a new workspace starts empty and switching back restores data; a rejected `spl:` write rolls back and says "storage is full"; **bulk hit counts equal the Threshold Report's printed numbers for every link**; hits-by-date and date-time rows each sum to the total; one valid PDF per link plus one combined (`%PDF-` header); Excel is a zip with a chart per link plus the top-links chart; real download modes (three separate PDFs; a single .zip); over-capacity forecast never projects below its current level and the range band brackets it; a grep-style **offline check** over the HTML (no http(s) URLs other than OOXML namespaces, no fonts/CDN/`<link>`/`@import`/`fetch`/XHR/WebSocket); zero console/page errors. Exit code 1 on any failure.

Script	Behaviour
`scripts/capture-golden.mjs`	Drives the page, pins `forecastCI` 0.8, `upgradeLeadDays` 42 and the **risk weights used by the C# defaults**, generates 25 forecast cases and 8 risk cases plus primitive statistics (1,000 samples → 10 percentiles; rolling median windows 3/7/15 over the first 60; Theil–Sen over 80 points at three CI levels; mean/max/min), and writes `forecast.json`, `risk.json`, `primitives.json`. Paths are absolute (`/home/user/AI-Automation/…`, `/opt/node22/…`) — the script is environment-specific.
Mutation testing (practice)	Deliberately breaking the port (window, stride, ceiling, floor, minimum-history gate, backtest gate, window floor, risk exponent, divisors, multipliers) must fail the suite; the first fixture round missed three such mutations, which is why the boundary cases exist.

16.3 Deployment files

- **`diff/deploy/web.config`** (IIS static hosting): default document `netpulse.html`; directory browsing off; `staticContent` cleared and only `.html`, `.md`, `.ico` MIME types mapped; client cache disabled (revalidate each load); HTML compression; custom headers — remove `X-Powered-By`, `Content-Security-Policy: frame-ancestors 'none'` (the one directive a `<meta>` tag cannot set; the rest ships inside the page), `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, COOP/CORP same-origin, a restrictive Permissions-Policy; HSTS present but commented out; request filtering — `maxAllowedContentLength` 1 MB (uploads never reach the server), verbs GET/HEAD only, only `.html` and `.ico` extensions, hidden segment `deploy`; a **URL Rewrite** rule redirecting HTTP to HTTPS (needed because browsers withhold `navigator.storage` from insecure contexts); `<authentication mode="Windows"/>` with a commented authorisation example.
- **`diff/deploy/IIS-Deployment.md`**: explains that hosting is not centralising data (table of what is and is not shared), prerequisites (Static Content, Windows Authentication, HTTP Redirection, URL Rewrite), site layout, PowerShell to create the site and app pool (No Managed Code), certificate binding, **why HTTPS is mandatory** (measured: `file://` and `localhost` are secure contexts; `http://<ip>` is not and loses `navigator.storage.persist()`), locking access by AD group, app-pool hardening (read-only NTFS, ApplicationPoolIdentity), verification commands with `curl.exe`, HSTS caveats, upgrade procedure (replace the file; data survives because it is keyed to the origin), and the options for genuinely shared data (B: central workbook library; C: full server application — see section 16.1).
- **`diff/netpulse-vNN.html`**: a copy of the current release for version control; the IIS guide says to rename it to `netpulse.html` on the server.

16.4 The other application in the repository: neteng-toolkit.sh

A separate, unrelated product (the repository's original purpose). It is a **self-installing Linux web app**: `sudo bash neteng-toolkit.sh install` writes an embedded HTML application to `/opt/neteng-toolkit/index.html`, creates a hardened `systemd` unit that serves it with `python3 -m http.server 8443`, and `uninstall` removes both. It shares no code with NetPulse.

Part	Details
Installer	Bash with `set -euo pipefail`; `require_root`; checks `python3`; heredoc-writes the HTML; unit runs as `nobody:nogroup` with `NoNewPrivileges`, `PrivateTmp`, `ProtectSystem=full`, `ProtectHome`, `Restart=on-failure`; commands `install` / `uninstall` / usage (exit 2).

Part	Details
Authentication	Client-side gate: default `admin` / `admin123`; PBKDF2-SHA-256 (150,000 iterations, random 16-byte salt) via Web Crypto; hash stored in `localStorage`; change-password form (min 8 characters). It is a local convenience lock, not network security (the README says so).
Storage	IndexedDB database `neteng-toolkit` (`workspace` store) for notes, tasks, inventory, KB, OUI map, templates, diagram; `localStorage` for theme/session/auth. Backup export/import as JSON; optional File System Access API sync.
Themes	dark, light, sepia, high contrast (CSS variables on `body`).
28 tools	**Addressing:** IPv4 subnet calculator (network, mask, wildcard, first/last host, broadcast, usable hosts), wildcard/mask converter, VLSM planner (largest-first allocation, "No room" reporting), IPv6 calculator (expand, prefix, documentation-prefix check), MAC/OUI lookup and format converter with custom OUI import/export. **Crypto/convert:** Base64, hex, binary, URL encoders; SHA-256/384/512; password generator (rejection-free modulo over a 62+symbols set). **Config:** reusable templates, CSV-to-config, Jinja-like `{{var}}` playground, line diff, ACL line builder, CDP/LLDP neighbour extractor, prefix-list/route tester (CIDR match with `le`), Cisco cheatsheet, local KB. **Operations:** port reference, bandwidth/transfer-time calculator, MTU/MSS, PoE budget, change-window time-zone planner, draggable diagram, scratchpad, task tracker, inventory, regex tester, JSON/XML viewer.
Observations	The default password is published in the page and README; the HTTP server binds `0.0.0.0` without TLS; `cidrSize` uses `Math.ceil(log2(hosts+2))`; the "Jinja" tool only does variable substitution; the YAML branch of the viewer only numbers lines. These are properties of a convenience toolkit, noted so nobody mistakes it for a hardened service.

16.5 Documents and generators in the repository

Item	Content
`README.md` (root)	Describes NetEng Toolkit: install/operate/uninstall, default credentials, storage model, backup/restore, tool list, security notes, troubleshooting, development checks (`bash -n`, extract the script and `node --check`).
`diff/README.md`	Explains the `diff/` workspace (kept separate from the toolkit), what `netpulse.html` does, and that the heavier enterprise pieces of the spec (server store, LLM assistant, SSO/RBAC, streaming) are the client-server evolution; AI summaries are the deterministic fallback.
`diff/CLAUDE.md`	Working rules for contributors/AI assistants: the offline invariant, forbidden constructs, allowed alternatives, the grep to run, and the test commands.
`diff/NetPulse-CHANGELOG.md`	Capability summary plus a version-by-version change list and "key issues fixed (root causes)".

Item	Content
`diff/NetPulse-Capability-Summary.html/.pdf`	Management-facing summary (business outcomes, data-integrity issues resolved, capabilities, architecture/security/delivery, next steps); older than v67.
`diff/network-analytics-platform/00…21-*.md`	The 22-document product specification (summarised in section 16.6).
`…/SOP-netpulse-operations.md`	Operations manual covering every knob: uploads and validation console, Admin (thresholds, hours, Upgrade Watch, upgrade policy, forecast, class policies, CSV export, device variants, data management), Link Manager, link page tools, RCA editing, AI summary modes, maintenance, reports and audit, theme, common workflows, data-scale guidance.
`…/NetPulse-Dashboard-Logic.pptx`, `NetPulse-Plain-English.pptx`	Two slide decks (a technical "dashboard logic & formulas" deck with a dark palette, and a plain-English deck with a light palette).
`…/deck/build_deck.js`, `build_deck2.js`	Node scripts using `pptxgenjs` that generate those decks: helper functions (`card`, `circle`, `title`, `formula`, `body`, `bullets`, `chip`, `foot`) and one block per slide. They are build tooling only and are not part of the app.
`.gitkeep`, `server/.gitignore`	Housekeeping.

16.6 The specification set (docs 01–21)

These are planning documents for the full product. The offline build implements a subset (marked below); the rest describes the intended server-based evolution.

Doc	Topic	Key content	Implemented in v67?
01	Vision, personas, value	Problem/response table; positioning between collectors and visualisation tools; six personas (architect, NOC lead, NOC engineer, executive, platform admin, auditor); non-goals (no config push, no packet forensics, no sub-second alerting).	n/a
02	Information architecture	Three axes (topology, analytical, operational); dashboard hierarchy Executive → NOC → Site → Device → Interface; role-based landing; flows; deep-linking; ⌘K command bar; consistent page anatomy.	Partly (role landing and URLs are not implemented)
03	Data model	Entities, interface as atomic unit, effective-dated attributes, canonical sample, rollup tiers (raw 90 d, 5 m 13 mo, 1 h 3 y, 1 d forever), batches and provenance, analytics tables, governance tables, overlays, integrity rules.	Concepts yes; storage is IndexedDB

Doc	Topic	Key content	Implemented in v67?
04	Ingestion and validation	Adapter contract; semantic column resolver; unit normalisation; validation pipeline (schema, duplicates, gaps, corrupt values, maintenance overlap); review and commit; idempotency; future sources.	Yes for XLSX/CSV (no counter-to-rate conversion, no saved mapping profiles)
05	Screen layouts	Per-screen layouts for 18 screens.	Most screens
06	User journeys	Seven journeys (03:00 congestion, budget defence, new month ingest, board risk, walking the data centre, policy without code, natural-language question).	n/a
07	Rule engine	Declarative rules (scope/when/if/compare/for/then), threshold sets, per-rule state machine, precedence, no-code builder, dry-run.	No — thresholds and the upgrade rule are fixed-form settings
08	Analytics engine	Rollups, seasonal baselines, derived metrics, "smart pie" distribution, incremental computation, data-quality handling.	Baselines, distribution and metrics yes; rollups no
09	Verdict engine	Verdict structure and catalogue; prioritised decision procedure; reason/evidence/comparison; confidence; recommended action; AI summary grounding; hysteresis; rollup verdicts.	Yes (13 rules in `verdictFor`; no hysteresis/history)
10	Anomaly detection	Eight anomaly types, ensemble detectors, scoring, suppression/mute, explanations.	Three types (spike, weekend traffic, imbalance)
11	Capacity forecasting	Outputs, layered models, time-to-threshold, upgrade window, backtesting, aggregation by scope.	Yes for per-link (Theil–Sen + backtest); no scope aggregation
12	Risk scoring	Factors, weighted transparent model, explainability, aggregation, stability.	Yes (six factors); no per-class weights, no smoothing
13	AI assistant	Retrieval-augmented, tool-calling, grounded, scope-enforced assistant.	Only the deterministic summaries and rule-based search
14	RCA assistant	Standard questionnaire, hypothesis engine, correlation, workspace.	Questionnaire and editable cause/steps yes

Doc	Topic	Key content	Implemented in v67?
15	Reports and exports	Catalogue, anatomy, scheduling, formats.	Yes for on-demand reports; no scheduling
16	Digital twin	Hierarchy, LED health colour, interaction, front panel realism, scale.	Site → device → port (no rooms/racks)
17	Design system	Language, colour, type, components, visualisation catalogue, interaction rules.	Yes (see section 10.2)
18	Security and RBAC	Seven roles, scoped permissions, SSO/MFA, append-only audit, encryption, AI security, hardening.	Role label and audit log only
19	Scalability	100k-interface strategy: rollups, partitioning, incremental compute, frontend performance, SRE.	Browser-scale mitigations only
20	Integrations and roadmap	Adapters (SNMP, NX-API/NDFC/ACI, SolarWinds, NetFlow, gNMI, Prometheus), four phases, future AI features, success metrics.	No
21	Operational modules	Classification, business impact, historical baselines, maintenance awareness, timeline, knowledge base.	Classification, impact, baselines, maintenance, timeline, comments; no KB

17. Code-review observations, dead code and known inconsistencies

Collected while reading every line. None of these changes a published number; they are listed so a maintainer is not surprised.

#	Observation	Detail / consequence
1	`resetAll()` confirmation over-promises	Text says config and audit are deleted; code clears samples, batches, maintenance and comments only.
2	`handleFilesOLD` is dead code	Earlier single-file handler, never called.
3	`busy-weekday` branch in `forecastFrom` is unreachable	Proved in `server/ARCHITECTURE.md`; kept in the browser build to avoid changing a validated file.
4	`forecastFrom`'s `cls` parameter unused	Accepted for historical reasons.
5	`riskWeights.impact` (1.2) is stored but not read	Impact enters through the fixed multipliers 1.25/1.05/0.85/0.65.
6	`ANOM_COLOR` lists anomaly types the detector never emits	"Unusual traffic time", "Sudden traffic reduction", "Long-duration congestion", "Post-maintenance behavior change" are spec items without detectors.
7	`save()` has a redundant ternary	Both branches read `S[k]`.
8	"Export all watched to Excel" exports **all** links	It calls `genUpgradePolicyXlsx('all')`.
9	`sparkline` uses `Math.max(...ys)`	Safe for a few hundred days; would overflow the argument limit at very large counts (the rest of the code uses `amax`).
10	Zip writer limits	No ZIP64, no UTF-8 name flag (file names must be ASCII); `parseZip` likewise lacks ZIP64.
11	Exports honour the filter's dates but not its Hours choice	`activeTF` returns only from/to.
12	Policy Excel report vs verdict	The report counts the whole loaded history (or the filter); the verdict uses the rolling look-back. Both say so on the sheet.
13	Time-zone setting is a label for reports	It does not move data; only the "input is UTC" option converts, and only at parse time.
14	C# `RiskWeights` defaults differ from browser defaults	See 16.1; callers must pass configured weights.
15	Demo batch id lacks the random suffix	`b<now>` versus `b<now>_<rand>` for real uploads; harmless.
16	Capability Summary document is dated	It predates v60–v67 (peak-based hit charts, multi-link pack, separate-file PDFs).
17	`neteng-toolkit.sh` publishes default credentials	By design for a local convenience tool.

18. Line-by-line annotated reference: analytics and export path

This section goes **line by line** through the part of `netpulse.html` (v67) that turns an uploaded spreadsheet into numbers and then into reports. Each table row is one source line (verbatim, whitespace trimmed) with its explanation beside it. A grey row is a comment in the source. Where the explanation begins with "…" the statement continues from the row above. The document build refuses to finish if any non-blank line in a covered range lacks an explanation.

Not covered line by line (explained at function level in sections 10–16): the HTML templates inside the views, the PDF drawing code, the Excel file-format writer internals, the single-link Threshold Report HTML, and the CSS.

18.1 Turning raw cells into numbers: unit, speed, time and rate parsing

Line	Source (verbatim)	What it does
1077	`const norm=s=>String(s==null?'':s).toLowerCase().replace(/[^a-z0-9]/g,'');`	`norm(s)`: lower-cases and removes everything except a–z and 0–9. `null` becomes "". Used to compare headers regardless of spaces, case or punctuation ("Transmit (Mbps)" → "transmitmbps").
1078	`const UNIT_RE=/(\d+(?:\.\d+)?)\s*(t|g|m|k)?\s*(b(?:it|ps|\/s)?|bytes?|o(?:ctet)?s?|bps)/i;`	`UNIT_RE`: finds `number [T|G|M|K] unit` inside a cell. Group 1 = the number, 2 = optional magnitude letter, 3 = unit word (bit, bps, b/s, byte(s), octet(s)). Case-insensitive.
1080	`function unitMultiplierFromToken(tok){ // returns {mult, isByte} bits-per-second multiplier or null`	Start of `unitMultiplierFromToken`: converts a unit word such as "Mbps" or "GB" into a multiplier to bits per second. Returns null if it is not a unit.
1081	`if(!tok)return null; const t=tok.toLowerCase();`	Empty token → null. Otherwise lower-case it into `t`.
1082	`const isByte=/byte|octet|bytes|\bb(?![ip])/.test(t)&&!/bit|bps/.test(t)||/[gmk]?b\b/.test(t)&&/bytes|octet/.test(t);`	Computes `isByte` (an older, partly redundant test; the value actually returned later uses a separate `byte` variable). It is true for words containing byte/octet or a lone "b" not followed by i/p, unless they also contain bit/bps.
1083	`let scale=1;`	`scale` = 1 (plain bits) until a magnitude letter is found.
1084	`if(/\bt/.test(t)||/tbps|tbit|tb\b/.test(t))scale=1e12;`	Tera: a word starting with "t", or tbps/tbit/tb → 1e12.
1085	`else if(/\bg/.test(t)||/gbps|gbit|gb\b|gig/.test(t))scale=1e9;`	Giga: starts with "g", or gbps/gbit/gb/gig → 1e9.
1086	`else if(/\bm/.test(t)||/mbps|mbit|mb\b|meg/.test(t))scale=1e6;`	Mega: starts with "m", or mbps/mbit/mb/meg → 1e6.
1087	`else if(/\bk/.test(t)||/kbps|kbit|kb\b/.test(t))scale=1e3;`	Kilo: starts with "k", or kbps/kbit/kb → 1e3. (The chain is `else if`, so the first match wins.)
1088	`const byte=/byte|octet/.test(t);`	`byte` = the token contains "byte" or "octet" → the value is in bytes and must be multiplied by 8.
1089	`if(!/b|octet|bps/.test(t))return null;`	Guard: the token must contain b, octet or bps, otherwise it is not a rate unit → null.
1090	`return {mult:scale*(byte?8:1),byte};`	Returns `{mult: scale × (8 if byte else 1), byte}`. Example: "MB" → 1e6×8 = 8e6 bits per unit.
1091	`}`	End of function.
1092	`function headerUnit(h){ // detect explicit unit inside a header string`	`headerUnit(h)`: detects an explicit unit inside a column header, e.g. "Average Transmit (Mbps)".
1093	`const hl=String(h).toLowerCase();`	Lower-case copy of the header.

Line	Source (verbatim)	What it does
1094	`// Prefix (k/m/g/t) must be ATTACHED to the unit (e.g. "mbps"), with word boundaries —`	Comment: the magnitude letter must be attached to the unit; otherwise the last letter of a word ("Transmit bps") would be misread as tera.
1095	`// otherwise the trailing letter of a word leaks in as a prefix ("Transmi-t bps" → tera).`	Comment continuation.
1096	`const m=hl.match(/\b(t|g|m|k)?(bps|bit\/s|bits?|bytes?|octets?|b\/s)\b/);`	Regex: optional prefix t/g/m/k, then bps / bit/s / bit(s) / byte(s) / octet(s) / b/s, with word boundaries. `m[1]` = prefix, `m[2]` = unit word, `m[0]` = whole match.
1097	`if(!m)return null;`	No unit word in the header → null.
1098	`if(/%|percent|util/.test(hl))return null;`	If the header mentions %, percent or util it is a percentage column, not a rate → null.
1099	`// A bare bit-rate token ("bps"/"bit"/"bits"/"b/s") with NO k/m/g/t prefix is the METRIC`	Comment: a bare "bps" is the metric name, not a scale.
1100	`// NAME ("bits per second"), not a magnitude — vendors routinely label Mbps/Gbps columns`	Comment continuation: vendors label Mbps/Gbps columns just "bps".
1101	`// just "bps". Treating it as literal bits/sec would force the wrong scale and disable the`	Comment continuation: treating it as literal bits would force a wrong scale.
1102	`// magnitude self-correction, so leave it ambiguous and let magnitude inference (value vs`	Comment continuation: so it is left ambiguous.
1103	`// capacity) pick the true scale. Prefixed units (Kbps/Mbps/Gbps) and byte units stay exact.`	Comment continuation: magnitude inference (value ÷ capacity) will pick the scale later.
1104	`const prefix=m[1],isByte=/byte|octet/.test(m[2]);`	`prefix` = k/m/g/t or undefined; `isByte` = unit word is byte/octet.
1105	`if(!prefix&&!isByte)return null;`	No prefix and not a byte unit (i.e. a bare "bps"/"bits") → return null (ambiguous, see above).
1106	`const info=unitMultiplierFromToken(m[0]); if(!info)return null;`	Convert the matched token to a multiplier; null if that fails.
1107	`return info;`	Return `{mult, byte}`.
1108	`}`	End of function.
1109	`// → bits/sec. When a capacity carries no unit, magnitude decides the scale, tuned for the`	Comment: how a capacity without a unit is read.
1110	`// DC/trading domain: ≥1e6 is already bps; 1000–1e6 is Mbps; a bare 1–1000 is read as Gbps`	Comment: tuned for data-centre links.
1111	`// (so 1/10/40/100 → 1/10/40/100 Gbps). A source that expresses capacity as a bare "100"`	Comment: ≥1e6 is bps; 1000–1e6 is Mbps; 1–1000 is Gbps.
1112	`// meaning 100 Mbps would therefore over-scale ×1000 — validateStaged flags that case (most`	Comment: the risk of a bare "100" meaning 100 Mbps.
1113	`// links reading <0.5% utilized), and a per-link Speed override or an explicit unit fixes it.`	Comment: where that risk is caught (validation warning, Speed override, explicit unit).
1114	`function parseSpeed(raw){`	`parseSpeed(raw)`: capacity → bits per second.
1115	`if(raw==null||raw==='')return null;`	Null or empty → null (no capacity).

Line	Source (verbatim)	What it does
1116	`if(typeof raw==='number'){ if(raw>=1e6)return raw; if(raw>=1000)return raw*1e6; if(raw>=1)return raw*1e9; return raw; }`	Numeric cell: ≥1e6 is taken as already bps; 1000 ≤ n < 1e6 is Mbps (×1e6); 1 ≤ n < 1000 is Gbps (×1e9); below 1 is returned unchanged.
1117	`const s=String(raw).toLowerCase().trim().replace(/,/g,'');`	String cell: lower-case, trim, remove thousands commas.
1118	`const m=s.match(/([\d.]+)\s*(t|g|m|k)?\s*(bps|bit|b|e|gig|meg)?/);`	Match `number [t|g|m|k] [bps|bit|b|e|gig|meg]`. `m[1]` the number, `m[2]` the magnitude letter.
1119	`if(!m)return null; let v=parseFloat(m[1]); const u=m[2];`	No number → null. Parse the number into `v`; `u` = magnitude letter.
1120	`if(u==='t')v*=1e12;else if(u==='g')v*=1e9;else if(u==='m')v*=1e6;else if(u==='k')v*=1e3;`	If a letter is present, apply it: t=1e12, g=1e9, m=1e6, k=1e3.
1121	`else{ if(v>=1e6){} else if(v>=1000)v*=1e6; else v*=1e9; }`	No letter: same magnitude rule as for numbers (≥1e6 unchanged, ≥1000 → Mbps, else Gbps).
1122	`return v;`	Return the bits-per-second value.
1123	`}`	End of function.
1124	`// Excel serial date → timestamp, preserving the wall-clock time exactly AS WRITTEN with`	Comment: Excel serials are timezone-naive.
1125	`// NO timezone shift. Excel serials are timezone-naive; the raw (serial-25569)*86400000`	Comment continuation.
1126	`// yields the encoded calendar fields as if UTC, so we read those fields back and rebuild`	Comment continuation: reading the calendar fields back as local time keeps the hour as written.
1127	`// them in local time — the displayed hour then equals the hour in the file (00 stays 00),`	Comment continuation.
1128	`// consistent with how string dates are parsed and how every view reads getHours().`	Comment continuation: consistent with how strings are parsed and how views call `getHours()`.
1129	`// Build a timestamp from calendar fields, honoring the Admin "input time zone" setting:`	Comment: `mkTs` honours the Admin input-time-zone setting.
1130	`// 'utc' → the fields are UTC, so anchor them in UTC (browser then shows them in local time);`	Comment continuation: "utc" anchors in UTC.
1131	`// 'aswritten' (default) → keep the exact wall clock, no shift.`	Comment continuation: "aswritten" keeps the wall clock.
1132	`function mkTs(y,mo,d,h,mi,s){`	`mkTs(y,mo,d,h,mi,s)`: builds a timestamp from calendar fields (month is 0-based).
1133	`return (S.cfg&&S.cfg.inputTz==='utc')? Date.UTC(y,mo,d,h||0,mi||0,s||0):new Date(y,mo,d,h||0,mi||0,s||0).getTime();`	If `inputTz==="utc"` use `Date.UTC(...)`; otherwise `new Date(...)` in the browser's local zone. Missing hour/minute/second default to 0.
1134	`}`	End of function.
1135	`function xlSerialToTs(serial){`	`xlSerialToTs(serial)`: Excel day number → timestamp.
1136	`const ms=Math.round((serial-25569)*86400000/1000)*1000;  // nearest second (guards Excel float error)`	25569 = days from 1899-12-30 (Excel epoch) to 1970-01-01. Multiply by 86,400,000 ms, then round to the nearest whole second to remove floating-point noise ("00:00:46" stays 46 s).

Line	Source (verbatim)	What it does
1137	`// The serial's wall clock is timezone-naive; `ms` is exactly that wall clock read as UTC.`	Comment: `ms` is the wall clock read as if it were UTC.
1138	`if(S.cfg&&S.cfg.inputTz==='utc')return ms; // treat as UTC instant → local display converts`	In UTC mode return `ms` itself (it is the UTC instant).
1139	`const d=new Date(ms); // else rebuild the same wall clock in local time (as written)`	As-written mode: take the UTC calendar fields of `ms`…
1140	`return new Date(d.getUTCFullYear(),d.getUTCMonth(),d.getUTCDate(),d.getUTCHours(),d.getUTCMinutes(),d.getUTCSeconds(),0).getTime();`	…and rebuild them as the same fields in local time. Result: the hour in the file equals the hour shown.
1141	`}`	End of function.
1142	`function parseTs(raw){`	`parseTs(raw)`: any timestamp cell → epoch milliseconds, or null.
1143	`if(raw==null||raw==='')return null;`	Empty → null.
1144	`if(typeof raw==='number') { if(raw>20000&&raw<80000)return xlSerialToTs(raw); if(raw>1e11)return raw; if(raw>1e9)return raw*1000; }`	Numbers: 20,000–80,000 are Excel serials (years 1954–2118); >1e11 is already milliseconds; >1e9 is seconds (×1000).
1145	`const s=String(raw).trim();`	String form, trimmed.
1146	`if(/^\d+(\.\d+)?$/.test(s)){const n=parseFloat(s);if(n>20000&&n<80000)return xlSerialToTs(n);if(n>1e11)return n;if(n>1e9)return n*1000;}`	A string that is purely a number gets the same three numeric rules.
1147	`// day-first dates: 22-06-2026 00:30 (common in exports; JS Date.parse chokes on these)`	Comment: day-first dates.
1148	`const dm=s.match(/^(\d{1,2})[-\/.](\d{1,2})[-\/.](\d{4})(?:[ T](\d{1,2}):(\d{2})(?::(\d{2}))?)?$/);`	Regex for `dd-mm-yyyy` (separators - / .) with optional `hh:mm[:ss]`.
1149	`if(dm){const A=+dm[1],B=+dm[2];const d=A>12?A:(B>12?B:A),mo=A>12?B:(B>12?A:B);`	Disambiguation: if the first part >12 it must be the day; else if the second >12 the second is the day; otherwise the first is taken as the day. `d` = day, `mo` = month.
1150	`const t=mkTs(+dm[3],mo-1,d,+(dm[4]||0),+(dm[5]||0),+(dm[6]||0));if(!isNaN(t))return t;}`	Build the timestamp with `mkTs` (month−1). Return it unless NaN.
1151	`// ISO-style YYYY-MM-DD[ T]HH:MM[:SS] — the wall-clock fields are taken exactly as written`	Comment: ISO-style dates.
1152	`// (any trailing Z / offset is ignored) and interpreted per the Admin input-time-zone setting.`	Comment continuation: any trailing Z or offset is ignored — fields are taken as written.
1153	`const iso=s.match(/^(\d{4})-(\d{1,2})-(\d{1,2})(?:[ T](\d{1,2}):(\d{2})(?::(\d{2}))?)?/);`	Regex for `yyyy-mm-dd[ T]hh:mm[:ss]`.
1154	`if(iso){const t=mkTs(+iso[1],+iso[2]-1,+iso[3],+(iso[4]||0),+(iso[5]||0),+(iso[6]||0));if(!isNaN(t))return t;}`	Build via `mkTs` and return.
1155	`const t=Date.parse(s.replace(' ','T')); if(!isNaN(t))return t;`	Fallback 1: `Date.parse` after replacing the first space with "T".
1156	`const t2=Date.parse(s); return isNaN(t2)?null:t2;`	Fallback 2: `Date.parse` as is; if still invalid return null.
1157	`}`	End of function.
1158	`function parseRate(raw,headerInfo){ // → bits/sec; {val, viaCellUnit, unit:true if ANY explicit unit was applied}`	`parseRate(raw, headerInfo)`: a traffic cell → `{val (bps), viaCellUnit, unit}`.
1159	`if(raw==null||raw==='')return {val:null};`	Empty → `{val:null}`.

Line	Source (verbatim)	What it does
1160	`if(typeof raw==='number'){ if(headerInfo)return {val:raw*headerInfo.mult,unit:true}; return {val:raw}; }`	Numeric cell: with a header unit multiply by its multiplier and mark `unit:true`; without, return the number as is (unit unknown, may be inferred later).
1161	`const s=String(raw).trim().replace(/,/g,''); // strip thousand separators BEFORE unit match ("1,700.00 Mbps")`	String cell: remove thousands commas BEFORE matching ("1,700.00 Mbps").
1162	`const m=s.match(UNIT_RE);`	Look for a number+unit inside the cell.
1163	`if(m){const info=unitMultiplierFromToken(m[0]);if(info)return {val:parseFloat(m[1])*info.mult,viaCellUnit:true,unit:true};}`	If found, the **cell unit wins**: value × multiplier, flagged `viaCellUnit` and `unit`.
1164	`const n=parseFloat(s.replace(/ /g,'')); if(isNaN(n))return {val:null};`	Otherwise strip spaces and parse a plain number; non-numeric → `{val:null}`.
1165	`return headerInfo? {val:n*headerInfo.mult,unit:true}:{val:n};`	Plain number: scale by the header unit if there is one (`unit:true`), else leave unscaled.
1166	`}`	End of function.

18.2 buildSamples: one spreadsheet row becomes one sample

Line	Source (verbatim)	What it does
1240	`function buildSamples(rows,resolved,srcName,batchId){`	Signature: `rows` (matrix of cells), `resolved` (column mapping), `srcName`, `batchId`.
1241	`const {map,headers}=resolved;`	Unpack `map` (field → column index) and `headers` (with detected units).
1242	`const hInfo=f=>map[f]!=null?(headers.find(h=>h.i===map[f])||{}).unit:null;`	Helper `hInfo(f)`: the unit detected in the header of field `f`, or null if the field is not mapped.
1243	`const units={tx:hInfo('tx'),rx:hInfo('rx'),ptx:hInfo('ptx'),prx:hInfo('prx')};`	Per-rate-field header units: average transmit/receive and peak transmit/receive.
1244	`const out=[],issues={corrupt:0,noTs:0,noSpeed:0,utilRecalc:0,cellUnit:0};`	`out` = samples to return; `issues` = counters shown in the validation console.
1245	`const CFS=cfList();`	`CFS` = the list of custom fields.
1246	`let scaleGuessNote=null;`	Will hold a note if magnitude auto-scaling is applied.
1247	`const val=(r,f)=>map[f]!=null?r[map[f]]:null;`	Helper `val(r,f)`: raw cell of field `f` in row `r`, or null.
1248	`const num=(r,f)=>{const v=val(r,f);if(v==null||v==='')return null;const n=parseFloat(String(v).replace(/[^\d.eE+-]/g,''));return isNaN(n)?null:n;};`	Helper `num(r,f)`: cell → plain number (strips everything except digits . e E + −); null if blank or not a number. Used for percentage columns.
1249	`const raw=[];`	`raw` = first-pass rows, before capacity-dependent scaling.
1250	`const info={};`	`info` = per-link attributes found in the file (site, IP, link name, custom fields).
1251	`for(const r of rows){`	Loop over every row of the file.
1252	`  const dev=String(val(r,'device')??'').trim(),intf=String(val(r,'interface')??'').trim();`	Device and interface text, trimmed. A row missing either is skipped (line 1253), so it never creates a link.
1253	`  if(!dev||!intf)continue;`	`continue` if device or interface is empty.
1254	`  const st=String(val(r,'site')??'').trim(),ipv=String(val(r,'ip')??'').trim(),ln=String(val(r,'linkname')??'').trim();`	Optional site, IP and link name for this row.
1255	`  if(st||ipv||ln){const o=info[dev+'|'+intf]=info[dev+'|'+intf]||{};if(st)o.site=st;if(ipv)o.ip=ipv;if(ln)o.linkname=ln;}`	If any is present, store it for the link `device|interface`; later rows overwrite earlier ones (last non-empty value wins).
1256	`  // custom fields — last non-empty value per link wins (they are link attributes, not time series)`	Comment: custom fields are link attributes, not time series.
1257	`  for(const c of CFS){const cv=String(val(r,c.key)??'').trim();`	For each custom field read the cell text.
1258	`  if(cv){const o=info[dev+'|'+intf]=info[dev+'|'+intf]||{};(o.cf=o.cf||{})[c.key]=cv;}}`	If non-empty store it under `info[link].cf[key]` (last value wins).
1259	`  const ts=parseTs(val(r,'ts'));if(ts==null){issues.noTs++;continue;}`	Parse the timestamp. A row with no valid timestamp is counted (`noTs`) and skipped.
1260	`  const tx=map.tx!=null?parseRate(val(r,'tx'),units.tx):{val:null};`	Average transmit rate, scaled by the header unit if there is one (otherwise `{val:null}` when the field is unmapped).

Line	Source (verbatim)	What it does
1261	`  const rx=map.rx!=null?parseRate(val(r,'rx'),units.rx):{val:null};`	Average receive rate (same logic).
1262	`  const ptx=map.ptx!=null?parseRate(val(r,'ptx'),units.ptx):{val:null};`	Peak transmit rate.
1263	`  const prx=map.prx!=null?parseRate(val(r,'prx'),units.prx):{val:null};`	Peak receive rate.
1264	`  if(tx.viaCellUnit||rx.viaCellUnit)issues.cellUnit++;`	Count rows where the unit came from the cell text (shown as an info finding).
1265	`  // Per-direction capacity ("Transmit/Received Bandwidth"); shared speed as fallback.`	Comment: per-direction capacity with the shared speed as fallback.
1266	`  const sp=map.speed!=null?parseSpeed(val(r,'speed')):null;`	`sp` = the shared capacity column parsed to bps (or null).
1267	`  const spTx=(map.speedtx!=null?parseSpeed(val(r,'speedtx')):null)||sp;`	Transmit capacity = its own column if mapped and parseable, else the shared capacity.
1268	`  const spRx=(map.speedrx!=null?parseSpeed(val(r,'speedrx')):null)||sp;`	Receive capacity, same rule.
1269	`raw.push({dev,intf,ts,tx:tx.val,rx:rx.val,ptx:ptx.val,prx:prx.val,spTx,spRx,`	Push the raw row: identity, time, four rates, two capacities, flags saying whether each rate had an explicit unit (`tU,rU,pTU,pRU`)…
1270	`tU:tx.unit,rU:rx.unit,pTU:ptx.unit,pRU:prx.unit,`	…(continued) the flags.
1271	`txp:num(r,'txpct'),rxp:num(r,'rxpct'),ptxp:num(r,'ptxpct'),prxp:num(r,'prxpct'),utilCol:num(r,'util')});`	…and the percentage columns (`txp,rxp,ptxp,prxp`) plus a supplied overall utilisation (`utilCol`).
1272	`}`	End of the row loop.
1273	`// Magnitude inference — per column, and ONLY for values that carried no explicit`	Comment: magnitude inference, per column.
1274	`// unit (neither in the cell text nor the header). A value that arrived as`	Comment: only for values with no explicit unit.
1275	`// "25.41 Mbps" is already correct bps and must NEVER be rescaled — a file that`	Comment: a value like "25.41 Mbps" is already correct and must never be rescaled.
1276	`// mixes text cells and plain numeric cells would otherwise get the unit-tagged`	Comment: a mixed file would otherwise be blown up.
1277	`// values blown up by the correction meant for the unit-less ones.`	Comment: continuation.
1278	`{`	Block start (scoped variables).
1279	`  const inferCols=[['tx','tU'],['rx','rU'],['ptx','pTU'],['prx','pRU']];`	The four rate columns paired with their "had an explicit unit" flag.
1280	`  const scaled=[];`	`scaled` lists the corrections applied, for the note.
1281	`  for(const [col,flag] of inferCols){`	For each rate column…
1282	`    const un=raw.filter(x=>x[col]!=null&&x[col]>0&&!x[flag]&&(x.spTx>0||x.spRx>0));`	`un` = rows with a positive value, no explicit unit, and a known capacity.
1283	`    if(un.length<10)continue;`	Need at least 10 such rows, otherwise skip (not enough evidence).
1284	`    const med=percentile(un.map(x=>x[col]/Math.max(x.spTx||0,x.spRx||0)),0.5);`	`med` = median of value ÷ the larger direction capacity. This ratio is what utilisation would be if the value were already bps.
1285	`    let factor=1;`	`factor` starts at 1 (no change).

Line	Source (verbatim)	What it does
1286	`if(med>1.5&&med<=3000)factor=1e-6;`	If the median is >1.5 and ≤3000 the values are 1.5–3000× LARGER than the link capacity — impossible if they were bps — so they are assumed to be expressed in a unit a million times finer than the capacity scale and are multiplied by 1e-6.
1287	`else if(med>3000)factor=1e-9;`	If the median is >3000 the discrepancy is larger still → multiplied by 1e-9.
1288	`else if(med>0&&med<0.0005)factor=1e6;`	If the median is positive but below 0.0005 (0.05%) the values are far too small to be bps — typically a file that reports Mbps (e.g. 500 on a 1 Gbps link) → multiplied by 1e6.
1289	`if(factor!==1){for(const x of raw)if(x[col]!=null&&!x[flag])x[col]*=factor;scaled.push(`${col} ×${factor}`);}`	Apply the factor to every unit-less value of that column and record it.
1290	`}`	End of column loop.
1291	`if(scaled.length)scaleGuessNote=`Auto-scaled unit-less values (${scaled.join(', ')}) from magnitude vs interface bandwidth; values carrying an explicit unit were left untouched.`;`	If anything was scaled, write the note shown in the validation console.
1292	`}`	End of block.
1293	`for(const x of raw){`	Second pass: turn each raw row into a final sample.
1294	`const spT=x.spTx,spR=x.spRx;`	`spT`, `spR` = transmit and receive capacities.
1295	`let tx=x.tx,rx=x.rx,ptx=x.ptx,prx=x.prx;`	Copy the four rates so they can be filled in.
1296	`// Derive missing rates from their percentage columns (% of the direction's capacity)`	Comment: derive missing rates from percentage columns.
1297	`if(tx==null&&x.txp!=null&&spT)tx=x.txp/100*spT;`	If transmit rate is missing but "Transmit %" and capacity exist: rate = % ÷ 100 × capacity.
1298	`if(rx==null&&x.rxp!=null&&spR)rx=x.rxp/100*spR;`	Same for receive.
1299	`if(ptx==null&&x.ptxp!=null&&spT)ptx=x.ptxp/100*spT;`	Same for peak transmit.
1300	`if(prx==null&&x.prxp!=null&&spR)prx=x.prxp/100*spR;`	Same for peak receive.
1301	`const speed=Math.max(spT||0,spR||0)||null;`	Sample capacity = the larger of the two directions (0 → null).
1302	`if(!speed)issues.noSpeed++;`	No capacity at all → count `noSpeed`.
1303	`// Average utilization: each direction against its OWN capacity (asymmetric links exist)`	Comment: each direction is measured against its own capacity.
1304	`const uT=spT&&tx!=null?tx/spT:(x.txp!=null?x.txp/100:null);`	`uT` = transmit utilisation: rate ÷ transmit capacity; else the supplied % ÷ 100; else null.
1305	`const uR=spR&&rx!=null?rx/spR:(x.rxp!=null?x.rxp/100:null);`	`uR` = receive utilisation, same rule.
1306	`let util=(uT!=null||uR!=null)?Math.max(uT??0,uR??0):(x.utilCol!=null?x.utilCol/100:null);`	**Headline utilisation** = max(uT, uR) (a missing side counts as 0); if both are null, use the supplied overall util column ÷ 100; else null.
1307	`if(util!=null&&x.utilCol!=null&&Math.abs(x.utilCol/100-util)>0.15)issues.utilRecalc++;`	If a supplied util differs from the computed one by more than 15 points, count `utilRecalc` (information only; the computed value is used).

Line	Source (verbatim)	What it does
1308	`  // Peak utilization from the peak columns — catches microbursts the averages hide`	Comment: peak utilisation.
1309	`  const pT=spT&&ptx!=null?ptx/spT:(x.ptxp!=null?x.ptxp/100:null);`	`pT` = peak transmit ÷ transmit capacity (or peak %).
1310	`  const pR=spR&&prx!=null?prx/spR:(x.prxp!=null?x.prxp/100:null);`	`pR` = peak receive utilisation.
1311	`  let pu=(pT!=null||pR!=null)?Math.max(pT??0,pR??0):null;`	`pu` = max(pT, pR) or null when there is no peak data.
1312	`  // Samples are NEVER dropped for high utilization. A utilization above 100% almost`	Comment: samples are NEVER dropped for high utilisation.
1313	`  // always means the capacity in the source is stale / the link was upgraded — that is`	Comment: >100% usually means a stale capacity.
1314	`  // surfaced to the user in the Upgrade tab for review, not hidden. Only a genuinely`	Comment: it is surfaced for review.
1315	`  // invalid value (negative) is flagged as bad. `over` marks a >100% (or Admin flag`	Comment: only a negative value is invalid.
1316	`  // threshold) sample so links needing a capacity check can be found.`	Comment: `over` marks a reading above the flag level.
1317	`  const flag=(S.cfg&&S.cfg.flagUtilAbove)||1.0;`	`flag` = Admin "flag utilisation above" (default 1.0 = 100%).
1318	`  let bad=0;`	`bad` starts 0.
1319	`  if(util!=null&&util<0)bad=1;`	A negative utilisation is the only thing marked `bad`.
1320	`  if(bad)issues.corrupt++;`	Count it in `issues.corrupt`.
1321	`  const over=(util!=null&&util>flag)?1:undefined;`	`over` = 1 if utilisation exceeds the flag level, else undefined (keeps stored objects small).
1322	`  if(over)issues.overCap=(issues.overCap||0)+1;`	Count `issues.overCap`.
1323	`  const dv=intern(x.dev),it=intern(x.intf);`	Intern the strings (device, interface) to save memory.
1324	`out.push({ifid:intern(dv+'|'+it),dev:dv,intf:it,ts:x.ts,tx,rx,ptx,prx,pu,`	Push the sample: `ifid` (device|interface), names, time, the four rates, peak utilisation `pu`…
1325	`    speed,speedTx:spT||null,speedRx:spR||null,util,bad:bad||undefined,over,batch:batchId});`	…capacity (`speed`, `speedTx`, `speedRx`), `util`, `bad`, `over`, `batch`.
1326	`}`	End of the second loop.
1327	`const extremes=[...out].filter(x=>x.util!=null).sort((a,b)=>(b.util||0)-(a.util||0)).slice(0,5);`	`extremes` = the five highest-utilisation samples, shown in the "parse inspector" so a human can check the numbers.
1328	`return {samples:out,issues,scaleGuessNote,info,extremes};`	Return `{samples, issues, scaleGuessNote, info, extremes}`.
1329	`}`	End of function.

18.3 inMaintWin and computeAnalytics: samples become link records

Line	Source (verbatim)	What it does
1710	`// Does this sample fall inside a declared maintenance window right now? Evaluated live`	Comment: asks whether a sample falls in a maintenance window right now.
1711	`// so edits to the Maintenance tab apply to already-uploaded data in both directions.`	Comment: evaluated live, so edits apply to already-uploaded data in both directions.
1712	`function inMaintWin(x){`	`inMaintWin(x)`: x is one sample.
1713	`const M=S.maintenance;if(!M||!M.length)return false;`	`M` = the list of windows. If none, return false immediately (the common fast path).
1714	`for(const m of M)if(m.start<=x.ts&&m.end>=x.ts&&(m.scope==='all'||m.target===x.ifid||m.target===x.dev))return true;`	For each window: inside the time range (inclusive both ends) AND (scope "all", or target equals this sample's link id, or target equals its device) → true.
1715	`return false;`	No window matched → false.
1716	`}`	End of function.
1717	`// `win` (optional {from,to}) restricts the input samples to a time window. It runs the`	Comment: optional `win` ({from,to}) restricts the input.
1718	`// exact same pipeline on a narrower input — no separate maths — and the windowed result`	Comment: same pipeline, narrower input, no separate maths.
1719	`// is returned WITHOUT touching the cached full-history analytics.`	Comment: the windowed result does not replace the cached full-history analytics.
1720	`function computeAnalytics(win){`	`computeAnalytics(win)`.
1721	`const groups={};`	`groups` = samples bucketed by link id.
1722	`for(const x of S.samples){`	Loop over all samples.
1723	`  if(win&&(x.ts<win.from||x.ts>win.to))continue;`	If a window was given, skip samples outside it.
1724	`  (groups[x.ifid]=groups[x.ifid]||[]).push(x);}`	Append the sample to its link's list.
1725	`const ifs=[];`	`ifs` = link records to return.
1726	`for(const ifid in groups){`	Loop over each link.
1727	`  // Ingested span BEFORE any analytics exclusion — used to reconcile "the export has`	Comment: the ingested span before any exclusion.
1728	`  // rows the chart doesn't". Anything removed below is counted and surfaced, never`	Comment: used to reconcile "the export has rows the chart does not".
1729	`  // silently dropped.`	Comment: nothing removed below is dropped silently.
1730	`  const gAll=groups[ifid].sort((a,b)=>a.ts-b.ts);`	Sort this link's samples by time into `gAll`.
1731	`  const rawCount=gAll.length,rawStart=gAll[0].ts,rawEnd=gAll.at(-1).ts;`	Remember how many samples there are and the first and last timestamps (`rawCount, rawStart, rawEnd`) before anything is removed.
1732	`  // Maintenance is evaluated LIVE against the current windows (it used to be stamped`	Comment: maintenance is evaluated live.
1733	`  // onto the sample at upload time, so a window added later did nothing and a window`	Comment: it used to be stamped at upload.
1734	`  // deleted later left those samples excluded from the chart permanently).`	Comment: a later-added window did nothing.
1735	`  const maintS=S.maintenance.length?gAll.filter(inMaintWin):[];`	`maintS` = samples inside any window (only computed if windows exist).

Line	Source (verbatim)	What it does
1736	`  let s=maintS.length?gAll.filter(x=>!inMaintWin(x)):gAll;`	`s` = the samples that remain after removing maintenance (or all if none).
1737	`  if(!s.length)continue;`	If nothing remains, the link is skipped entirely.
1738	`  const dev=s[0].dev,intf=s[0].intf;`	`dev`, `intf` from the first remaining sample.
1739	`  let {cls,impact}=classify(ifid,intf);`	Automatic class and impact from the interface name (`classify`).
1740	`  const ov=S.ifmeta[ifid]||{};  // user-set overrides beat every automatic guess`	`ov` = this link's user overrides; they beat every automatic guess (applied on the next line).
1741	`  if(ov.cls)cls=ov.cls; if(ov.impact)impact=ov.impact;`	Apply the class and impact overrides if set.
1742	`  const tags=ov.tags||[];`	User tags.
1743	`  const info=S.ifinfo[ifid]||{};`	`info` = link attributes from the file (site, IP, link name, custom fields).
1744	`  const site=ov.site||info.site||siteOf(dev);`	Site = override, else from file, else guessed from the hostname.
1745	`  const tRes=Object.assign({},S.cfg.thresholds,S.cfg.classThresholds[cls]||{},ov.th||{}); // per-link thresholds win`	Business-hours thresholds: global ← class ← per-link (later wins).
1746	`  const tOffRes=Object.assign({},tRes,S.cfg.thresholdsOff||{},ov.thOff||{});  // off-hours bands (fall back to business)`	Off-hours thresholds: the business set ← global off-hours ← per-link off-hours.
1747	`  const hasOffBands=!!(S.cfg.thresholdsOff||ov.thOff);`	True if any off-hours band exists (global or per-link).
1748	`  const tForS=x=>(!hasOffBands||isBH(x.ts))?tRes:tOffRes;`	`tForS(x)`: the band set that applies to one sample — business set if there is no off-hours band or the sample is in market hours; else the off-hours set.
1749	`  const spOv=ov.speed?parseSpeed(ov.speed):null; // per-link speed override`	Per-link speed override parsed to bps (or null).
1750	`  const flag=S.cfg.flagUtilAbove||1.0; // >this util → flag for capacity review`	Flag level for "over capacity".
1751	`  if(spOv)s=s.map(x=>{const u=(x.tx!=null||x.rx!=null)?Math.max(x.tx||0,x.rx||0)/spOv:x.util;`	If a speed override exists, rebuild each sample: utilisation = max(tx,rx) ÷ override (or keep the old util if there are no rates)…
1752	`    const pk=(x.ptx!=null||x.prx!=null)?Math.max(x.ptx||0,x.prx||0)/spOv:x.pu;`	…peak utilisation = max(ptx,prx) ÷ override…
1753	`    return {...x,speed:spOv,speedTx:spOv,speedRx:spOv,util:u,pu:pk,bad:(u!=null&&u<0)?1:undefined,over:(u!=null&&u>flag)?1:undefined};});`	…and return a copy with new speed fields, `util`, `pu`, `bad` (negative) and `over` (above the flag).
1754	`  const speed=spOv||s.map(x=>x.speed).filter(Boolean).pop()||null;`	Link capacity = override, else the most recent non-null sample capacity (`.pop()` takes the last).
1755	`  // Detect a capacity change within the link's own history (an upgrade/downgrade in the`	Comment: detect a capacity change within the history.
1756	`  // source data). Utilization is already computed per-sample against each sample's own`	Comment: utilisation is already per-sample, so results stay correct.
1757	`  // capacity, so hits/verdicts are correct either way — this just surfaces it to the user.`	Comment: this only surfaces the change.
1758	`  let capChange=null;`	`capChange` starts null.

Line	Source (verbatim)	What it does
1759	`  if(!spOv){const seq=s.map(x=>x.speed).filter(Boolean);const distinct=[...new Set(seq)];`	Only without an override: collect the sequence of capacities and the distinct values.
1760	`    if(distinct.length>1){const first=seq[0],last=seq.at(-1);`	If more than one distinct value: remember the first and last.
1761	`      let atTs=null,fromSp=first;for(const x of s){if(x.speed&&x.speed!==fromSp){atTs=x.ts;break;}}`	Find the first sample whose capacity differs from the first one (the change time).
1762	`      capChange={distinct:distinct.sort((a,b)=>a-b),first,last,at:atTs,up:last>first};}}`	Store `{distinct (sorted), first, last, at, up}` where `up` = last > first.
1763	`  const badS=s.filter(x=>x.bad),badCount=badS.length;`	`badS` = negative-utilisation samples; `badCount` = how many.
1764	`  s=s.filter(x=>!x.bad);`	Remove them from `s`.
1765	`  if(!s.length)continue;`	If nothing is left, skip the link.
1766	`  // Coverage reconciliation: what was ingested vs what the charts/analytics show.`	Comment: coverage reconciliation.
1767	`  // If the raw data runs past the analyzed end (a maintenance window covering the`	Comment: raw data may run past the analysed end.
1768	`  // tail, or invalid rows), the link page states it rather than the chart just stopping.`	Comment: the page states it rather than just stopping.
1769	`  const exSpan=a=>a.length?{n:a.length,first:a[0].ts,last:a.at(-1).ts}:null;`	`exSpan(a)`: summary `{n, first, last}` of an excluded list, or null.
1770	`  const excluded={rawCount,rawStart,rawEnd,maint:exSpan(maintS),bad:exSpan(badS),`	Build `excluded`: raw count and span, the maintenance and bad summaries, total excluded `n`…
1771	`n:maintS.length+badCount,tailGap:rawEnd>s.at(-1).ts?rawEnd:null,headGap:rawStart<s[0].ts?rawStart:null};`	…`tailGap` (raw data ends later than analysed data) and `headGap` (starts earlier).
1772	`  // Over-capacity flag: samples whose utilization exceeds 100% (or the Admin flag level)`	Comment: over-capacity flag.
1773	`  // — kept in all analytics, but the link is flagged so the user can verify/override the`	Comment: kept in all analytics.
1774	`  // capacity in the Upgrade tab (a >100% reading usually means the source capacity is stale).`	Comment: the link is flagged for review.
1775	`  const overS=s.filter(x=>x.util!=null&&x.util>flag);   // derive live so the Admin flag level applies without re-upload`	`overS` = samples above the flag level, derived live so changing the Admin level applies without re-upload.
1776	`  let overCap=null;`	`overCap` starts null.
1777	`  if(overS.length){let mxu=0,mxTs=null;for(const x of overS){if(x.util>mxu){mxu=x.util;mxTs=x.ts;}}`	If any: find the maximum utilisation and when it happened…
1778	`overCap={count:overS.length,pct:overS.length/s.length,max:mxu,maxTs:mxTs,lastTs:overS.at(-1).ts,threshold:flag};}`	…store `{count, pct (share of samples), max, maxTs, lastTs, threshold}`.
1779	`  const speedTx=s.map(x=>x.speedTx).filter(Boolean).pop()||speed;`	Latest transmit capacity (falls back to the link capacity).
1780	`  const speedRx=s.map(x=>x.speedRx).filter(Boolean).pop()||speed;`	Latest receive capacity.
1781	`  const utils=s.map(x=>x.util).filter(v=>v!=null&&v>=0);`	`utils` = all non-negative utilisations.
1782	`  const p95=percentile(utils,0.95),p99=percentile(utils,0.99),avg=mean(utils),mx=amax(utils,0);`	Headline statistics: p95, p99, mean, maximum (`mx`).

Line	Source (verbatim)	What it does
1783	`  // peak-column analytics (microbursts the averages hide)`	Comment: peak-column analytics.
1784	`  const pus=s.map(x=>x.pu).filter(v=>v!=null&&v>=0);`	`pus` = peak-column utilisations.
1785	`  const pkP95=pus.length?percentile(pus,0.95):null,pkMax=pus.length?amax(pus):null;`	p95 and max of the peak column (null if no peak data).
1786	`  // #3 "peak" used to SIZE capacity — a selectable high percentile (robust to a single glitch,`	Comment: the "peak" used to size capacity.
1787	`  // yet closer to true peak load than p95), measured over the trailing projection window`	Comment continuation: a selectable high percentile.
1788	`  // (default 180 days) so it reflects the *recent* peak the way capacity rules intend.`	Comment continuation: over the trailing window.
1789	`  const pkB=S.cfg.peakBasis||'p999';const pkQ=pkB==='p99'?0.99:pkB==='max'?1:0.999;`	Peak basis: p99.9 default, p99, or max → quantile 0.999 / 0.99 / 1.
1790	`  const fwDays=S.cfg.forecastWindowDays!=null?S.cfg.forecastWindowDays:180;`	Trailing window in days (default 180; 0 means all history).
1791	`  const fwStart=fwDays>0?s.at(-1).ts-fwDays*86400000:-Infinity;`	Window start = last sample time − window (or −∞ if 0).
1792	`  const pusW=s.filter(x=>x.ts>=fwStart).map(x=>x.pu).filter(v=>v!=null&&v>=0);`	Peak-column values inside the window.
1793	`  const utilsW=s.filter(x=>x.ts>=fwStart).map(x=>x.util).filter(v=>v!=null&&v>=0);`	Utilisation values inside the window.
1794	`  const provisionPeak=pusW.length?percentile(pusW,pkQ):(utilsW.length?percentile(utilsW,pkQ):(pus.length?percentile(pus,pkQ):mx));`	`provisionPeak` = percentile (quantile above) of peak values in the window; else of utilisation in the window; else of all peak values; else the maximum. This is the "peak" used by provisioning and regulatory sizing.
1795	`  // distribution (time-in-state)`	Comment: time-in-state distribution.
1796	`  const dist={normal:0,warning:0,high:0,critical:0};let distN=0;`	Counters for the four states and their total.
1797	`  for(const x of s){if(x.util==null||x.util<0)continue;dist[stateOfT(x.util,tForS(x))]++;distN++;}`	For each valid sample classify it against ITS band set (market or off-hours) and count.
1798	`  for(const k in dist)dist[k]/=(distN||1);`	Convert counts to fractions.
1799	`  // business-hour breaches`	Comment: business-hour breaches.
1800	`  const t=tRes;const breachesBH=s.filter(x=>x.util>=t.crit&&isBH(x.ts)).length;`	`t` = business thresholds. `breachesBH` = market-hours samples at/above Critical.
1801	`  const bhDays=new Set(s.filter(x=>x.util>=t.high&&isBH(x.ts)).map(x=>new Date(x.ts).toDateString())).size;`	`bhDays` = distinct calendar days with a market-hours sample at/above High.
1802	`  // Configurable link-upgrade policy (util > X% [in business hours] on Y distinct days)`	Comment: upgrade policy.
1803	`  const ur=Object.assign({},S.cfg.upgradeRule,ov.upgrade||{});`	`ur` = global rule overlaid by per-link overrides.
1804	`  // Evaluate the policy over a trailing window (default 90d) ending at the link's latest`	Comment: evaluated over a trailing look-back window.
1805	`  // sample — NOT the whole loaded history — so an old breach cannot latch the verdict`	Comment: so an old breach cannot latch the verdict.
1806	`  // forever and this stays consistent with the windowed Upgrade Watch view.`	Comment continuation.
1807	`  const upLook=(ur.lookbackDays||0)>0?ur.lookbackDays*86400000:Infinity;`	Look-back in ms (0 or absent → infinite).

Line	Source (verbatim)	What it does
1808	`  const upWinStart=s.at(-1).ts-upLook;`	Window start = last sample − look-back.
1809	`  const upgradeDays=new Set(s.filter(x=>x.ts>=upWinStart&&x.util!=null&&x.util>=ur.utilPct/100&&(!ur.businessHoursOnly||isBH(x.ts))).map(x=>new Date(x.ts).toDateString())).size;`	`upgradeDays` = number of distinct days with a sample in the window at/above the policy % (market hours only if ticked).
1810	`  const upgradeNeeded=upgradeDays>=ur.days;`	`upgradeNeeded` = `upgradeDays ≥ required days`.
1811	`  // baseline dow×hour`	Comment: weekday × hour baseline.
1812	`  const base={};for(const x of s){if(x.util==null)continue;const k=new Date(x.ts).getDay()+'-'+new Date(x.ts).getHours();(base[k]=base[k]||[]).push(x.util);}`	`base[k]` = list of utilisations per slot `weekday-hour`.
1813	`  const baseStat={};for(const k in base)baseStat[k]={m:mean(base[k]),sd:std(base[k]),n:base[k].length};`	`baseStat[k]` = mean, standard deviation and count per slot.
1814	`  // heatmap dow×hour avg`	Comment: heatmap average.
1815	`  const heat={};for(const k in base)heat[k]=mean(base[k]);`	`heat[k]` = mean utilisation per slot (the heatmap data).
1816	`  // anomalies`	Comment: anomalies.
1817	`  const anomalies=[];`	`anomalies` list.
1818	`  // Baseline (dow×hour) z-score, guarded three ways so a thin/noisy baseline`	Comment: baseline z-score with three guards.
1819	`  // can't manufacture a "spike": (1) at least 5 historical points for that slot,`	Comment: at least 5 historical points for the slot.
1820	`  // (2) the absolute jump above the baseline must itself be meaningful (>=8pp),`	Comment: the jump above baseline must be ≥8 points.
1821	`  // not just many small stddevs on a very tightly-clustered baseline,`	Comment: not just many standard deviations on a tight baseline.
1822	`  // (3) still requires the raw value to be operationally non-trivial (>15%).`	Comment: the value must also be >15%.
1823	`  for(const x of s){if(x.util==null)continue;const k=new Date(x.ts).getDay()+'-'+new Date(x.ts).getHours();const b=baseStat[k];`	For each sample: its slot and baseline `b`…
1824	`    if(b&&b.n>=5&&b.sd>0.001){const z=(x.util-b.m)/b.sd;const jump=x.util-b.m;`	…if the slot has ≥5 points and sd > 0.001: z = (value − mean)/sd and jump = value − mean…
1825	`if(z>3.2&&jump>=0.08&&x.util>0.15)anomalies.push({ts:x.ts,type:'Traffic spike',z,util:x.util,exp:b.m,jump});}}`	…flag a **Traffic spike** when z > 3.2 AND jump ≥ 0.08 AND value > 0.15; store time, z, util, expected mean, jump.
1826	`  // weekend traffic on business link`	Comment: weekend traffic on a business link.
1827	`  // Weekend traffic on a link that should idle on weekends — flagged only when the`	Comment continuation.
1828	`  // weekend median is BOTH operationally significant (>20%) AND a large fraction of the`	Comment continuation: weekend median significant (>20%).
1829	`  // weekday median (i.e. the weekend genuinely resembles a working day), so a link that`	Comment continuation: and ≥60% of the weekday median.
1830	`  // is legitimately busy 7 days a week does not trip a permanent false positive.`	Comment continuation: so a 7-day-busy link does not trip a permanent false positive.
1831	`if(['Core','Trading','Internet','Customer'].includes(cls)){`	Only for classes Core, Trading, Internet, Customer.
1832	`    const we=s.filter(x=>[0,6].includes(new Date(x.ts).getDay())&&x.util!=null);`	`we` = weekend samples (Sun=0, Sat=6) with utilisation.

Line	Source (verbatim)	What it does
1833	`    const wd=s.filter(x=>![0,6].includes(new Date(x.ts).getDay())&&x.util!=null);`	`wd` = weekday samples.
1834	`    if(we.length>5&&wd.length>5){`	Need more than 5 of each.
1835	`      const wem=percentile(we.map(x=>x.util),0.5),wdm=percentile(wd.map(x=>x.util),0.5);`	Weekend and weekday medians.
1836	`      if(wem>0.2&&wem>=wdm*0.6){const at=we.find(x=>x.util>=wem)||we[0];anomalies.push({ts:at.ts,type:'Weekend traffic on business link',util:wem,exp:wdm});}`	If weekend median >0.2 and ≥ 0.6 × weekday median: add the anomaly (anchored at the first weekend sample at/above the median).
1837	`    }`	End of the inner if.
1838	`  }`	End of the class if.
1839	`  // rx/tx imbalance`	Comment: RX/TX imbalance.
1840	`  const withDir=s.filter(x=>x.tx>0&&x.rx>0);if(withDir.length>10){const ratios=withDir.map(x=>Math.max(x.tx,x.rx)/Math.min(x.tx,x.rx));const mr=percentile(ratios,0.5);if(mr>6)anomalies.push({ts:withDir[0].ts,type:'RX/TX imbalance',ratio:mr});}`	Samples with both rates positive; if more than 10, take max÷min ratios; median ratio > 6 → anomaly **RX/TX imbalance**.
1841	`  // daily p95 series + forecast`	Comment: daily series + forecast.
1842	`  // daily series for forecasting — metric (p95/p99/max) and direction (combined / TX & RX split)`	Comment: metric and direction.
1843	`  const fm=S.cfg.forecastMetric||'p95';const pctl=fm==='p99'?0.99:fm==='max'?1:0.95;`	Forecast metric (p95 default) → quantile 0.95/0.99/1.
1844	`  const bhOnlyFc=(S.cfg.forecastHours==='bh');const minDay=S.cfg.forecastMinDaySamples||1;`	Market-hours-only flag and the minimum samples per day.
1845	`  // #5 drop thin days (few samples → noisy daily figure) unless that leaves too little history.`	Comment: thin days.
1846	`  // #6 optionally restrict the basis to market hours only.`	Comment: market-hours basis.
1847	`  // Projection window: fit the trend to the trailing N days only (default 180) — matches the`	Comment: projection window.
1848	`  // "project the peak from the last 180 days" basis, and keeps the trend recent.`	Comment: matches the "project from the last 180 days" basis.
1849	`  const dseries=(valFn)=>{const bd={};for(const x of s){if(x.ts<fwStart)continue;if(bhOnlyFc&&!isBH(x.ts))continue;const v=valFn(x);if(v==null)continue;const d=new Date(x.ts);d.setHours(0,0,0,0);(bd[+d]=bd[+d]||[]).push(v);}`	`dseries(valFn)`: bucket the chosen value per local day, skipping samples before the window start and (optionally) outside market hours; values that are null are skipped.
1850	`    const arr=Object.keys(bd).map(Number).sort((a,b)=>a-b).map(d=>({d,p95:percentile(bd[d],pctl),n:bd[d].length}));`	For each day (sorted): `{d (day start), p95 (metric percentile of the day), n (sample count)}`.
1851	`    const filt=arr.filter(o=>o.n>=minDay);return filt.length>=5?filt:arr;};`	Drop days with too few samples, unless fewer than 5 days would remain (then keep all).
1852	`  let daily,forecast,fnote=(fm!=='p95'?fm:'');`	`daily`, `forecast`, and `fnote` (metric label when not p95).
1853	`  if((S.cfg.forecastDirection||'combined')==='split'){`	Split mode: forecast TX and RX separately.
1854	`    const txD=dseries(x=>speedTx&&x.tx!=null?x.tx/speedTx:null),rxD=dseries(x=>speedRx&&x.rx!=null?x.rx/speedRx:null);`	`txD`/`rxD` = daily series of transmit ÷ its capacity and receive ÷ its capacity.
1855	`    const fTx=txD.length?forecastFrom(txD,percentile(txD.map(d=>d.p95),0.95),cls):null;`	Forecast for transmit (if it has data).
1856	`    const fRx=rxD.length?forecastFrom(rxD,percentile(rxD.map(d=>d.p95),0.95),cls):null;`	Forecast for receive.

Line	Source (verbatim)	What it does
1857	`    if(fTx||fRx){`	If at least one exists…
1858	`      // the binding direction is the one that saturates first (smallest days-to-90%)`	Comment: the binding direction is the one that saturates first.
1859	`      const sc=f=>f&&f.t90!=null&&f.t90>=0?f.t90:1e9;`	`sc(f)`: days to 90% (if non-negative) else a huge number (1e9).
1860	`      const txBind=!!(fTx&&(!fRx||sc(fTx)<=sc(fRx)));`	Transmit binds if receive is missing or its score ≤ receive's.
1861	`      forecast=txBind?fTx:fRx;`	Use that direction's forecast.
1862	`      daily=(txBind?txD:rxD).length?(txBind?txD:rxD):dseries(x=>x.util);`	Chart series = that direction's daily data (or combined if empty).
1863	`forecast.dir=txBind?'TX':'RX';forecast.otherDir=txBind?'RX':'TX';`	Label the direction (`dir`) and the other (`otherDir`).
1864	`      const other=txBind?fRx:fTx;forecast.otherCur=other?.current;forecast.otherT90=other?.t90;`	Attach the other direction's current level and days-to-90%.
1865	`      fnote=`${forecast.dir}-bound${fm!=='p95'?' · '+fm:''}`;`	Note text, e.g. "TX-bound · p99".
1866	`    }else{ // neither direction had usable per-direction data — fall back to combined, unlabeled`	Comment: neither direction had data.
1867	`daily=dseries(x=>x.util);forecast=forecastFrom(daily,p95,cls);`	Fall back to the combined series and forecast.
1868	`    }`	End of the split branch.
1869	`  }else{daily=dseries(x=>x.util);forecast=forecastFrom(daily,p95,cls);}`	Combined mode: series of `util`; forecast with the link's p95.
1870	`if(fnote)forecast.modelNote=(forecast.modelNote?forecast.modelNote+' · ':'')+fnote;`	Append the note to the forecast's model note.
1871	`  // risk`	Comment: risk.
1872	`  const growth=forecast.growthMoM;`	`growth` = forecast monthly growth.
1873	`  const risk=riskScore({p95,avg,pk:pkP95,slope:forecast.slopePerDay,growth,breachesBH:bhDays,anomalies:anomalies.length,impact,speed,cls});`	Risk score from p95, average, peak-p95, slope, growth, business-day breaches, anomaly count, impact, speed, class.
1874	`  const rec={ifid,dev,intf,site,cls,impact,tags,speed,speedTx,speedRx,capChange,overCap,th:tRes,thOff:hasOffBands?tOffRes:null,ip:info.ip,linkname:info.linkname,cf:info.cf||{},badCount,excluded,upgradeDays,upgradeNeeded,upgradeRule:ur,samples:s,count:s.length,`	Build the link record: identity, class, impact, capacity, flags, thresholds, policy result, the sample list…
1875	`    start:s[0].ts,end:s.at(-1).ts,avg,p95,p99,mx,pkP95,pkMax,provisionPeak,dist,breachesBH,bhDays,baseStat,heat,anomalies,daily,forecast,risk};`	…`count, start, end, avg, p95, p99, mx, pkP95, pkMax, provisionPeak, dist, breachesBH, bhDays, baseStat, heat, anomalies, daily, forecast, risk`.
1876	`  rec.verdict=verdictFor(rec);`	Attach the verdict (needs the record).
1877	`  rec.ai=aiSummary(rec);`	Attach the deterministic AI summary text.
1878	`  ifs.push(rec);`	Add to the output list.
1879	`}`	End of the per-link loop.
1880	`// aggregates`	Comment: aggregates.
1881	`const byDev={},bySite={};`	`byDev`, `bySite` buckets.

Line	Source (verbatim)	What it does
1882	`for(const r of ifs){(byDev[r.dev]=byDev[r.dev]||[]).push(r);(bySite[r.site]=bySite[r.site]||[]).push(r);}`	Fill them.
1883	`const impactW={Critical:1,High:0.7,Medium:0.4,Low:0.2};`	Impact weights for aggregation: Critical 1, High 0.7, Medium 0.4, Low 0.2.
1884	`const aggRisk=list=>{let sw=0,s=0,mxr=0;for(const r of list){const w=impactW[r.impact];sw+=w;s+=r.risk.score*w;mxr=Math.max(mxr,r.risk.score);}return Math.round(clamp((s/(sw||1))*0.6+mxr*0.4,0,100));};`	`aggRisk(list)`: 60% × impact-weighted mean risk + 40% × worst risk, rounded and clamped to 0–100.
1885	`const devices=Object.entries(byDev).map(([dev,list])=>({dev,site:list[0].site,list,risk:aggRisk(list),critical:list.filter(r=>r.verdict.level==='critical').length}));`	Device records with risk and critical-link counts.
1886	`const sites=Object.entries(bySite).map(([site,list])=>({site,list,risk:aggRisk(list),devices:new Set(list.map(r=>r.dev)).size,critical:list.filter(r=>r.verdict.level==='critical').length}));`	Site records with risk, device count and critical-link count.
1887	`const infraRisk=aggRisk(ifs);`	Fleet risk.
1888	`const out={ifs,devices,sites,infraRisk,byDev,bySite};`	Result object.
1889	`if(win)return out;  // windowed view: never replaces the cache`	If a window was given, return it without touching the cache.
1890	`S.analytics=out;`	Otherwise store it in `S.analytics`…
1891	`return S.analytics;`	…and return it.
1892	`}`	End of function.

18.4 Forecast, risk score and verdict

Line	Source (verbatim)	What it does
1901	`// #2 centered rolling median — removes the weekday/weekend sawtooth before trending.`	Comment: centred rolling median — removes the weekday/weekend saw-tooth before trending.
1902	`function rollingMedian(ys,w){const h=(w-1)>>1,out=[];for(let i=0;i<ys.length;i++){const a=Math.max(0,i-h),b=Math.min(ys.length,i+h+1);out.push(percentile(ys.slice(a,b),0.5));}return out;}`	`rollingMedian(ys,w)`: half-window `h=(w−1)>>1`; for each index i take the slice from max(0,i−h) to min(n,i+h+1) (clipped at both ends) and return its median via `percentile(…,0.5)`.
1903	`// Theil–Sen with a slope confidence interval — slopeLo/slopeHi are the CI-percentiles of the`	Comment: Theil–Sen with a slope confidence interval — slopeLo/slopeHi are CI percentiles of the pairwise slopes.
1904	`// pairwise slopes (the textbook Theil–Sen interval), used to project a days-to-threshold range.`	Comment continuation: the textbook Theil–Sen interval, used to project a days-to-threshold range.
1905	`function theilSen(pts,ci){`	`theilSen(pts,ci)`: `pts` are `{x (days), y (utilisation)}`.
1906	`const n=pts.length;if(n<2)return null;`	Fewer than 2 points → null (no slope).
1907	`const step=Math.max(1,Math.floor(n/45)),slopes=[];`	`step` = max(1, floor(n/45)): with many points only every step-th point is paired, bounding the pair count (≈45² pairs); `slopes` collects the results.
1908	`for(let i=0;i<n;i+=step)for(let j=i+1;j<n;j+=step){const dx=pts[j].x-pts[i].x;if(dx>0)slopes.push((pts[j].y-pts[i].y)/dx);}`	Double loop over point pairs (i<j, both stepping by `step`); `dx` = x distance; if positive, push the slope $(y_2-y_1)/dx$.
1909	`if(!slopes.length)return null;`	No slopes (all dx ≤ 0) → null.
1910	`const slope=percentile(slopes,0.5),a=(1-(ci||0.8))/2;`	`slope` = median of all pairwise slopes; `a` = (1−ci)/2 = the lower tail probability (ci defaults to 0.8 → 0.1).
1911	`return {slope,inter:percentile(pts.map(p=>p.y-slope*p.x),0.5),slopeLo:percentile(slopes,a),slopeHi:percentile(slopes,1-a)};`	Return `{slope, inter (median of y − slope·x over ALL points), slopeLo (percentile a of slopes), slopeHi (percentile 1−a)}`.
1912	`}`	End of function.
1913	`const maeOf=(fit,pts)=>pts.length?mean(pts.map(p=>Math.abs(p.y-(fit.slope*p.x+fit.inter)))):Infinity;`	`maeOf(fit,pts)`: mean absolute error of the line `slope·x+inter` against the points; Infinity when there are none (so an empty test set never wins).
1914	`// Ceiling for the fitted level AND every projection. They used to differ (level 1.5, projections`	Comment: one ceiling for the fitted level AND every projection.
1915	`// 1.2), so a link already running above 120% had its range band start at 150% and "fall" to 120%`	Comment continuation: they used to differ (level 1.5, projections 1.2).
1916	`// over 30 days and run flat — a decline made entirely by the clamps.`	Comment continuation: a decline made entirely by the clamps.
1917	`const FC_MAX=1.5;`	`FC_MAX = 1.5` (150%) — the single ceiling.

Line	Source (verbatim)	What it does
1918	`function forecastFrom(daily,p95,cls){`	`forecastFrom(daily,p95,cls)`: `daily` = array of `{d (day start ms), p95 (daily metric), n}`; `p95` = the link's overall p95; `cls` is accepted but unused.
1919	`const out={slopePerDay:0,current:p95,t80:null,t90:null,t95:null,t80Lo:null,t80Hi:null,t90Lo:null,t90Hi:null,t95Lo:null,t95Hi:null,growthMoM:0,conf:0.2,proj:{30:p95,60:p95,90:p95,180:p95},projLo:{},projHi:{},window:null,r2:0,ci:(S.cfg.forecastCI||0.8),modelNote:''};`	Default result `out`: slope 0, current = p95, all time-to-threshold null (with Lo/Hi variants), growth 0, confidence 0.2, flat projections at 30/60/90/180 days, empty low/high bands, no window, R^2 0, CI from settings, empty note.
1920	`if(daily.length<5){out.modelNote='not enough history for a trend';return out;}`	Fewer than 5 daily points → note "not enough history for a trend" and return the default.
1921	`const d0=daily[0].d;`	`d0` = first day; the x axis is elapsed real days from it.
1922	`let pts=daily.map(p=>({x:(p.d-d0)/86400000,y:clamp(p.p95,0,1.5),dow:new Date(p.d).getDay()}));`	Build points: x = (day − d0)/86,400,000; y = daily value clamped to 0–1.5; also the weekday number. Real day spacing means polling gaps do not distort the slope.
1923	`// #2 deseasonalize the weekly cycle before fitting the trend`	Comment: deseasonalise the weekly cycle before fitting.
1924	`const sm=rollingMedian(pts.map(p=>p.y),7);pts=pts.map((p,i)=>({...p,y:sm[i]}));`	`sm` = 7-day rolling median of y; replace every point's y with it.
1925	`const wd=pts.filter(p=>p.dow>=1&&p.dow<=5),we=pts.filter(p=>p.dow===0||p.dow===6);`	`wd` = Mon–Fri points, `we` = Sat/Sun points (after smoothing).
1926	`if(wd.length>=5&&we.length>=2&&percentile(wd.map(p=>p.y),0.5)>percentile(we.map(p=>p.y),0.5)*1.35+0.02){`	Busy-weekday test: ≥5 weekday and ≥2 weekend points and weekday median > 1.35 × weekend median + 0.02.
1927	`  pts=wd;out.modelNote='busy-weekday';`	If true, trend only the weekday points and tag the note "busy-weekday". (Unreachable in practice: the 7-day median has already equalised weekday and weekend values — see server/ARCHITECTURE.md.)
1928	`}`	End of branch.
1929	`const ci=S.cfg.forecastCI||0.8;`	`ci` = confidence level for the slope interval (default 0.8).
1930	`const rec=pts.slice(-Math.max(7,Math.floor(pts.length/2)));`	`rec` = the most recent window: the last max(7, half of all points).
1931	`const cands=[{name:'full-window trend',pts},{name:'recent trend',pts:rec}]`	Two candidate models: "full-window trend" (all points) and "recent trend" (`rec`)…
1932	`  .map(c=>({...c,fit:theilSen(c.pts,ci)})).filter(c=>c.fit);`	…each fitted with Theil–Sen; a candidate that cannot be fitted is removed.
1933	`if(!cands.length){out.modelNote+=' · no fit';return out;}`	If neither fits, append " · no fit" to the note and return the default.
1934	`let chosen=cands[0],relErr=0.35;`	Default choice = first candidate; default relative error 0.35 (used when there are too few points to backtest).
1935	`if(pts.length>=8){`	Backtest only with ≥8 points.

Line	Source (verbatim)	What it does
1936	`  const k=Math.max(3,Math.round(pts.length*0.2));`	`k` = number of held-out days = max(3, round(20% of the points)).
1937	`  const cut=pts[pts.length-k-1].x,test=pts.slice(-k);`	`cut` = x of the last training point; `test` = the last k points.
1938	`  let best=Infinity;`	`best` = lowest test error so far (starts at infinity).
1939	`  for(const c of cands){`	For each candidate model…
1940	`    const tf=theilSen(c.pts.filter(p=>p.x<=cut),ci);`	…refit it using only its points up to the cut (no peeking at the test days).
1941	`    if(!tf)continue;`	If it cannot be refitted, skip it.
1942	`    const err=maeOf(tf,test);`	Mean absolute error of that refit on the held-out days.
1943	`    if(err<best){best=err;chosen=c;}`	Keep the candidate with the smallest error as `chosen`.
1944	`  }`	End of candidate loop.
1945	`if(best<Infinity)relErr=clamp(best/Math.max(0.05,mean(test.map(p=>p.y))),0,1);`	If at least one candidate was scored: relative error = best error ÷ max(0.05, mean of the held-out y), clamped 0–1. This is what lowers confidence for erratic links.
1946	`}`	End of backtest block.
1947	`const fit=chosen.fit;`	`fit` = the chosen model's fit on ALL of its points.
1948	`const slope=clamp(fit.slope,-0.04,0.04); // ±4%/day plausibility ceiling`	Slope clamped to ±0.04 per day (±4 utilisation points per day) — a plausibility ceiling.
1949	`const sHi=clamp(fit.slopeHi,-0.04,0.04),sLo=clamp(fit.slopeLo,-0.04,0.04);`	The upper and lower interval slopes clamped the same way (`sHi`, `sLo`).
1950	`const lastX=pts.at(-1).x;`	`lastX` = x of the most recent point.
1951	`const cur=clamp(slope*lastX+fit.inter,0,FC_MAX);`	**Current level** = slope × lastX + intercept (the fitted line evaluated today), clamped to 0–FC_MAX. This is the number all time-to-threshold results start from.
1952	`out.slopePerDay=slope;out.current=cur;out.r2=clamp(1-relErr,0,1);out.slopeLo=sLo;out.slopeHi=sHi;`	Store slope, current, `r2 = 1 − relative error` (clamped), and the clamped slope bounds.
1953	`// #1 days-to-threshold as a RANGE: faster slope (sHi) → reaches sooner (Lo days); slower → Hi days.`	Comment: days-to-threshold as a range: the faster slope reaches the threshold sooner.
1954	`const ttFor=(T,sl)=>cur>=T?0:(sl>1e-5?(T-cur)/sl:null);`	`ttFor(T,sl)`: 0 if current ≥ T; else (T − current) ÷ sl when sl > 0.00001 per day; else null (never reached).
1955	`for(const T of [80,90,95]){const t=T/100;out['t'+T]=ttFor(t,slope);out['t'+T+'Lo']=ttFor(t,sHi);out['t'+T+'Hi']=ttFor(t,sLo);}`	For each threshold 80/90/95 %: `tNN` with the central slope; `tNNLo` with the fastest slope (earliest date); `tNNHi` with the slowest slope (latest date, null if that slope is ≤0).
1956	`for(const dd of [30,60,90,180]){out.proj[dd]=clamp(cur+slope*dd,0,FC_MAX);out.projLo[dd]=clamp(cur+sLo*dd,0,FC_MAX);out.projHi[dd]=clamp(cur+sHi*dd,0,FC_MAX);}`	Projections at 30, 60, 90, 180 days = current + slope × days, plus low and high band versions with `sLo` and `sHi`; every value clamped to 0–FC_MAX.

Line	Source (verbatim)	What it does
1957	`// #4 growth taken from the SAME fitted slope (relative monthly rate) — no separate estimate to disagree`	Comment: growth is taken from the SAME slope, so it can never disagree with the trend.
1958	`out.growthMoM=clamp(slope*30/Math.max(cur,0.03),-3,3);`	`growthMoM` = slope × 30 ÷ max(current, 0.03), clamped to −3…+3 (the 0.03 floor stops tiny links showing absurd percentages).
1959	`const agree=cands.length===2&&Math.sign(cands[0].fit.slope)===Math.sign(cands[1].fit.slope);`	`agree` = both candidate models exist and their slopes have the same sign.
1960	`const bandDays=(out.t90Hi!=null&&out.t90Lo!=null)?Math.abs(out.t90Hi-out.t90Lo):null;`	`bandDays` = width in days of the 90% threshold range (|t90Hi − t90Lo|), if both bounds exist.
1961	`const tight=bandDays!=null?clamp(1-bandDays/120,0,1):0.35; // tighter band → higher confidence`	`tight` = 1 − bandDays/120 clamped 0–1 (narrow range → high), or 0.35 when unknown.
1962	`out.conf=clamp(0.15+clamp((lastX-pts[0].x)/45,0,1)*0.25+(1-relErr)*0.3+(agree?0.1:0)+tight*0.15,0,0.95);`	**Confidence** = 0.15 + history span (45 days = full credit) × 0.25 + (1 − relative error) × 0.3 + 0.1 if the two models agree + `tight` × 0.15; clamped 0–0.95.
1963	`out.modelNote=(out.modelNote?out.modelNote+' · ':'')+chosen.name+` ` · ${Math.round(ci*100)}% band`;`	Model note = any existing note + " · " + chosen model name + " · NN% band".
1964	`if(out.t90!=null&&out.t90>0){const lead=S.cfg.upgradeLeadDays;const c0=Math.max(0,out.t90-lead);out.window=[Math.round(c0),Math.round(Math.max(c0+15,out.t90))];}`	If t90 is positive: lead time from Admin; window start = max(0, t90 − lead); window = [start, max(start+15, t90)] rounded to whole days.
1965	`return out;`	Return the forecast object.
1966	`}`	End of function.
1967	`function riskScore(f){`	`riskScore(f)`.
1968	`const W=S.cfg.riskWeights;const impactMul={Critical:1.25,High:1.05,Medium:0.85,Low:0.65}[f.impact];`	`W` = weights; impact multiplier Critical 1.25, High 1.05, Medium 0.85, Low 0.65.
1969	`const nl=v=>clamp(v,0,1)**1.4; // non-linear utilization curve`	`nl(v)` = clamp(v,0,1)^1.4 (non-linear curve).
1970	`const factors={`	`factors` object begins.
1971	`  'Peak util':nl(f.pk!=null?f.pk:f.p95)*W.peak,`	Peak util = curve(peak p95, else p95) × weight.
1972	`  'Avg util':nl(f.avg)*W.avg,`	Avg util = curve(avg) × weight.
1973	`  'Trend':clamp(f.slope*60/0.5,0,1)*W.trend,`	Trend = clamp(slope × 60 ÷ 0.5, 0, 1) × weight (60 days × slope reaching +50 points = 1).
1974	`  'Growth':clamp(f.growth/0.3,0,1)*W.growth,`	Growth = clamp(growth ÷ 0.3, 0, 1) × weight.
1975	`'Violations':clamp(f.breachesBH/14,0,1)*W.violations,`	Violations = clamp(business-day breaches ÷ 14, 0, 1) × weight.
1976	`  'Anomalies':clamp(f.anomalies/4,0,1)*W.anomaly`	Anomalies = clamp(count ÷ 4, 0, 1) × weight.
1977	`};`	End of factors.
1978	`const sumW=W.peak+W.avg+W.trend+W.growth+W.violations+W.anomaly;`	Sum of the six weights.
1979	`let raw=Object.values(factors).reduce((a,b)=>a+b,0)/sumW;`	`raw` = sum of weighted factors ÷ sum of weights (0–1).
1980	`const score=Math.round(clamp(raw*impactMul,0,1)*100);`	Score = round(clamp(raw × impact multiplier, 0, 1) × 100).

Line	Source (verbatim)	What it does
1981	`return {score,factors,impactMul};`	Return `{score, factors, impactMul}`.
1982	`}`	End of function.
1983	`function verdictFor(r){`	`verdictFor(r)`: first matching rule wins.
1984	`const t=r.th||th(r.cls);const f=r.forecast;`	`t` = the link's thresholds; `f` = forecast.
1985	`const critFrac=r.dist.critical,highFrac=r.dist.high;`	Shares of time in Critical and High.
1986	`const growing=f.growthMoM>0.05&&f.slopePerDay>0;`	`growing` = monthly growth >5% and positive slope.
1987	`const planSoon=f.t90!=null&&f.t90<=S.cfg.planHorizonDays;`	`planSoon` = t90 exists and ≤ planning horizon.
1988	`const wk=r.anomalies.find(a=>a.type==='Weekend traffic on business link');`	Weekend anomaly, if any.
1989	`const imb=r.anomalies.find(a=>a.type==='RX/TX imbalance');`	Imbalance anomaly, if any.
1990	`let level='normal',title='Healthy',reason='Utilization within normal bounds across the observed period.',action='No action required.';`	Default verdict: normal / Healthy / no action.
1991	`if(r.badCount>0&&r.badCount>=r.count*0.3){level='warning';title='Data Quality Issue';reason=`${r.badCount} of ${r.count+r.badCount} samples are physically impossible (rate above link capacity) — likely counter-wrap artifacts or a wrong bandwidth column.`;action='Verify the bandwidth mapping or set a Speed override in Link Manager; artifacts are excluded from analytics.';}`	Rule 1 — **Data Quality Issue**: bad samples ≥ 30% of samples.
1992	`else if(r.count<8){level='warning';title='Insufficient Data';reason='Not enough history for confident analysis.';action='Upload more polling history.';}`	Rule 2 — **Insufficient Data**: fewer than 8 samples.
1993	`else if(r.upgradeNeeded){const ur=r.upgradeRule;level='critical';title='Link Upgrade Recommended';`	Rule 3 — **Link Upgrade Recommended** (critical): `upgradeNeeded`.
1994	`reason=`Utilization crossed ${ur.utilPct}%${ur.businessHoursOnly?' during business hours':''} on ${r.upgradeDays} day(s) — meeting the ${ur.days}-day upgrade trigger. p95 ${pct1(r.p95)}${growing?`, +${(f.growthMoM*100).toFixed(0)}% MoM`:''}.`;`	(rule 3 reason text with the policy numbers and growth).
1995	`action=`${ur.mode==='auto'?'Auto-flagged for upgrade':'Upgrade recommended'}${f.window?` — plan within the ${f.window[0]}–${f.window[1]} day window`:''}. Tune the trigger in Admin → Upgrade policy.`;}`	(rule 3 action text with the upgrade window).
1996	`else if(r.bhDays>=5&&(critFrac>0.02||highFrac>0.1)){level='critical';title='Recurring Business-Hour Congestion'+(growing?' — Capacity Planning Required':'');`	Rule 4 — **Recurring Business-Hour Congestion** (critical): ≥5 business days at High and (Critical time >2% or High time >10%).
1997	`reason=`Utilization reached the High/Critical band on ${r.bhDays} business day(s); p95 ${pct1(r.p95)}${growing?`, up ${(f.growthMoM*100).toFixed(0)}% MoM`:''}.`;`	(rule 4 reason).
1998	`action=planSoon?`Plan a capacity upgrade within the ${f.window?f.window[0]+'–'+f.window[1]:'~60'} day window.`:'Confirm organic growth vs incident; schedule capacity review.';}`	(rule 4 action: plan within the window if `planSoon`, else review).
1999	`else if(critFrac>0.03){level='critical';title='Critical Congestion';reason=`Spent ${pct1(critFrac)} of intervals in the Critical band (p99 ${pct1(r.p99)}).`;action='Investigate now; check peer/uplinks; consider QoS; open RCA.';}`	Rule 5 — **Critical Congestion**: Critical time >3%.

Line	Source (verbatim)	What it does
2000	`else if(planSoon&&growing) {level='high';title='Capacity Planning Required';reason=`Forecast reaches 90% in ~${Math.round(f.t90)} days at current growth (+${(f.growthMoM*100).toFixed(0)}% MoM).`;action=`Plan upgrade in the ${f.window?f.window[0]+'–'+f.window[1]:'recommended'} day window.`;}`	Rule 6 — **Capacity Planning Required** (high): `planSoon` and growing.
2001	`else if(wk){level='warning';title='Weekend Backup Traffic';reason=`Weekend traffic (~${pct1(wk.util)}) on a business-classified link normally idle on weekends.`;action='Confirm backup schedule is intended; reclassify if mislabeled.';}`	Rule 7 — **Weekend Backup Traffic**.
2002	`else if(imb){level='warning';title='RX/TX Imbalance';reason=`Sustained directional imbalance (~${imb.ratio.toFixed(1)}× RX/TX).`;action='Check asymmetric routing / unidirectional flows / SPAN config.';}`	Rule 8 — **RX/TX Imbalance**.
2003	`else if(r.pkP95!=null&&r.pkP95>=t.crit&&r.p95<t.high) {level='warning';title='Bursty Traffic (Microbursts)';reason=`Peaks reach ${pct1(r.pkP95)} (p95 of peak samples) while the average stays at ${pct1(r.p95)} — short bursts saturate the link between polls.`;action='Review QoS/buffer tuning; keep burst headroom for latency-sensitive traffic.';}`	Rule 9 — **Bursty Traffic**: peak p95 ≥ Critical while p95 < High.
2004	`else if(r.anomalies.length) {level='warning';title='Traffic Pattern Changed';reason=`${r.anomalies.length} behavioral anomaly(ies) detected vs baseline.`;action='Compare to change/maintenance events; verify expected vs anomalous.';}`	Rule 10 — **Traffic Pattern Changed**: any anomaly.
2005	`else if(r.p95<0.03){level='normal';title='Idle / Under-utilized';reason=`p95 utilization only ${pct1(r.p95)} — consolidation candidate.`;action='Verify still in service; candidate for reclaim.';}`	Rule 11 — **Idle / Under-utilized**: p95 < 3%.
2006	`else if(r.dist.high+r.dist.warning>0.15||growing) {level='warning';title='Monitor';reason=`Elevated but sub-critical (p95 ${pct1(r.p95)}${growing?`, +${(f.growthMoM*100).toFixed(0)}% MoM`:''}).`;action='Keep under observation.';}`	Rule 12 — **Monitor**: High+Warning time >15% or growing.
2007	`const conf=clamp(0.4+clamp(r.count/300,0,1)*0.35+f.r2*0.2+clamp(r.bhDays/7,0,0.1),0,0.97);`	Confidence = 0.4 + min(count/300,1)×0.35 + forecast R^2×0.2 + min(bhDays/7, 0.1); clamp 0–0.97.
2008	`const comparison=`p95 ${pct1(r.p95)} vs baseline avg ${pct1(r.avg)} · ${r.count} samples over ${Math.round(days(r.start,r.end))||1}d`;`	Comparison text.
2009	`return {level,title,reason,action,confidence:conf,comparison};`	Return `{level, title, reason, action, confidence, comparison}`.
2010	`}`	End of function.

18.5 The shared hit definition and the multi-link preparation

Line	Source (verbatim)	What it does
3726	`/* ---- Threshold hit calculation — the ONE definition of "a hit" ------------------`	Comment: heading — the ONE definition of "a hit".
3727	` Used by the Threshold Report below and by the multi-link PDF / Excel pack, so they can`	Comment: used by the Threshold Report and the multi-link PDF/Excel pack.
3728	` never disagree. `bands` is 'warn' | 'high' | 'crit' (severity bands, per-link thresholds,`	Comment: so they can never disagree.
3729	` market / off-hours bands honoured) or 'policy' (util at or above the link's Upgrade`	Comment: `bands` is warn | high | crit (severity bands) or policy.
3730	` Policy percentage, whatever band that lands in). Window and hours are applied here;`	Comment: market and off-hours bands are honoured.
3731	` the caller decides what the window is. Pure: reads r, returns numbers. */`	Comment: the caller decides the window; this function is pure.
3732	`function hitCalc(r,{winStart,winEnd,hours,bands}) {`	`hitCalc(r, {winStart, winEnd, hours, bands})`: `r` is a link record from `computeAnalytics`.
3733	`const BANDSETS={warn: ['warning','high','critical'],high: ['high','critical'],crit:['critical'],policy: ['normal','warning','high','critical']};`	`BANDSETS`: which states count as a hit for each choice — warn = warning+high+critical; high = high+critical; crit = critical; policy = all four (the policy test is a utilisation test, not a band test).
3734	`const policy=bands==='policy',pol=policy? r.upgradeRule.utilPct/100:null;`	`policy` = true when bands is "policy"; `pol` = the link's upgrade-policy fraction (utilPct/100), else null.
3735	`const sel=new Set(BANDSETS[bands]|| BANDSETS.warn);`	`sel` = the set of states that count (defaults to the warn set for an unknown value).
3736	`const hourOk=x=>hours==='bh'? isBH(x.ts):hours==='off'?!isBH(x.ts):true;`	`hourOk(x)`: passes if hours is "all"; market hours only ("bh") requires `isBH`; non-market ("off") requires `!isBH`.
3737	`const thFor=x=>isBH(x.ts)?r.th:(r.thOff||r.th);`	`thFor(x)`: the band set for a sample — business thresholds in market hours; otherwise the off-hours set if the link has one, else the business set.
3738	`const p2=n=>String(n).padStart(2,'0');`	Helper `p2` pads numbers to two digits.
3739	`const fmtD=ts=>{const d=new Date(ts);return `${d.getFullYear()}-${p2(d.getMonth()+1)}-${p2(d.getDate())}`;};`	`fmtD(ts)`: local date key `yyyy-mm-dd` used for per-day grouping.
3740	`const samples=r.samples.filter(x=>x.util!=null&&x.ts>=winStart&&x.ts<=winEnd&&hourOk(x));`	`samples` = the link's analysed samples that have a utilisation, lie inside [winStart, winEnd] (inclusive) and pass the hours filter. This is the population every count below is taken from.
3741	`const breaches=[]; let cW=0,cH=0,cC=0,cN=0; const dW=new Set(),dC=new Set(); let peak=0,peakTs=null; const byDay={};`	Accumulators: hit list, counts by band (`cW` warning, `cH` high, `cC` critical, `cN` below-warning), distinct hit days (`dW` any, `dC` critical), peak value and time, and `byDay`.
3742	`let mWarn=0,mCrit=0,nWarn=0,nCrit=0;  // market vs non-market splits (all hits / critical)`	Comment: market vs non-market splits.

Line	Source (verbatim)	What it does
3743	`for(const x of samples){const t=thFor(x);const st=stateOfT(x.util,t);const bh=isBH(x.ts);`	Loop over the population. For each sample: `t` = its band set; `st` = its state (normal/warning/high/critical); `bh` = in market hours.
3744	`  if(x.util>peak){peak=x.util;peakTs=x.ts;}`	Update the peak (over ALL samples in the population, hits or not) and its time.
3745	`  if(policy?x.util<pol:(st==='normal'||!sel.has(st)))continue;   // not a hit under this definition`	**The hit test.** Policy mode: a sample below the policy percentage is skipped. Severity mode: skipped if its state is "normal" or not in the selected set. Everything after this line is a hit.
3746	`  const dk=fmtD(x.ts);const o=byDay[dk]=byDay[dk]||{w:0,h:0,c:0,n:0,m:0,peak:0,peakTs:null,first:x.ts,last:x.ts};`	Get (or create) this day's record: warnings, highs, criticals, total `n`, market count `m`, peak, peak time, first and last time.
3747	`  if(st==='warning')cW++; else if(st==='high')cH++; else if(st==='critical'){cC++;dC.add(dk);} else cN++;`	Count by band: warning/high/critical go to `cW/cH/cC` (critical also adds the day to `dC`); a policy hit that is below warning goes to `cN`.
3748	`  dW.add(dk);`	Add the day to the set of days with any hit.
3749	`  if(bh){mWarn++; if(st==='critical')mCrit++; o.m++;} else {nWarn++; if(st==='critical')nCrit++;}`	Market hits increment `mWarn` (and `mCrit` if critical) and the day's market counter `o.m`; non-market hits increment `nWarn` (and `nCrit`).
3750	`  if(st==='warning')o.w++; else if(st==='high')o.h++; else if(st==='critical')o.c++;`	Increment the day's per-band counter.
3751	`  o.n++;`	Increment the day's total.
3752	`  if(x.util>o.peak){o.peak=x.util;o.peakTs=x.ts;} if(x.ts<o.first)o.first=x.ts; if(x.ts>o.last)o.last=x.ts;`	Update the day's peak, peak time, first and last hit times.
3753	`breaches.push({ts:x.ts,util:x.util,tx:x.tx,rx:x.rx,st,bh});`	Push the hit `{ts, util, tx, rx, st, bh}` to the list.
3754	`}`	End of the loop.
3755	`return {sel,samples,breaches,cW,cH,cC,cN,dW,dC,peak,peakTs,byDay,mWarn,mCrit,nWarn,nCrit,pol};`	Return everything: the selected set, the population, the hits, counts, distinct days, peak, per-day records, market/non-market splits and the policy fraction.
3756	`}`	End of function.
4004	`// One window for every selected link, anchored on the newest sample among them, so a pack`	Comment: one window for every selected link, anchored on the newest sample.
4005	`// compares like with like. Each link's own coverage inside it is reported separately.`	Comment: so the pack compares like with like.
4006	`function bulkWindow(list,period,cFrom,cTo){`	`bulkWindow(list, period, cFrom, cTo)`.
4007	`const end=amax(list.map(r=>r.end)),start0=amin(list.map(r=>r.start)),DAY=86400000;`	`end` = latest sample time among the selected links; `start0` = earliest; `DAY` = 86,400,000 ms.
4008	`if(period==='custom')return {from:cFrom?new Date(cFrom).getTime():start0,to:cTo?new Date(cTo).getTime():end};`	Custom range: from/to inputs (as local time), falling back to the data bounds if blank.
4009	`if(period==='all')return {from:start0,to:end};`	"All": the full span.

Line	Source (verbatim)	What it does
4010	`return {from:Math.max(start0,end-(+period)*DAY),to:end};`	Otherwise N days back from `end`, but not earlier than the earliest data.
4011	`}`	End of function.
4012	`// Chart resolution follows the window length so the plot is readable without hiding hits:`	Comment: chart resolution follows the window length.
4013	`// hourly up to ~2 weeks, 6-hourly up to ~100 days, daily beyond. Each point keeps the PEAK, so a`	Comment: hourly ≤ ~2 weeks, 6-hourly ≤ ~100 days, daily beyond.
4014	`// single hit is never averaged away, plus the average for context.`	Comment: each point keeps the PEAK so a hit is never averaged away.
4015	`function hitSeries(r,{winStart,winEnd,hours}){`	`hitSeries(r, {winStart, winEnd, hours})`.
4016	`const span=(winEnd-winStart)/86400000,step=span<=16?'h':span<=100?'6h':'d';`	`span` in days; `step` = "h" if ≤16, "6h" if ≤100, else "d".
4017	`const ms={h:3600000,'6h':21600000,d:86400000}[step];`	Bucket length in ms.
4018	`const key=ts=>{const d=new Date(ts);if(step==='h')d.setMinutes(0,0,0);else if(step==='6h')d.setHours(Math.floor(d.getHours()/6)*6,0,0,0);else d.setHours(0,0,0,0);return +d;};`	`key(ts)`: floor a timestamp to the start of its hour / 6-hour block / local day.
4019	`const bk=new Map();`	`bk` = map from bucket start to its accumulator.
4020	`for(const x of r.samples){`	Loop over the link's samples.
4021	`  if(x.util==null||x.ts<winStart||x.ts>winEnd)continue;`	Skip samples without utilisation or outside the window.
4022	`  if(hours==='bh'?!isBH(x.ts):hours==='off'?isBH(x.ts):false)continue;`	Apply the hours filter (skip market samples when hours="off", non-market when hours="bh").
4023	`  const k=key(x.ts);let o=bk.get(k);`	Bucket key and its accumulator.
4024	`  if(!o){o={t:k,peak:x.util,sum:x.util,n:1};bk.set(k,o);}else{if(x.util>o.peak)o.peak=x.util;o.sum+=x.util;o.n++;}`	First sample of a bucket: `{t, peak, sum, n=1}`; otherwise update peak, add to the sum, increment n.
4025	`}`	End of the loop.
4026	`return {step,ms,pts:[...bk.values()].sort((a,b)=>a.t-b.t).map(o=>({t:o.t,peak:o.peak,avg:o.sum/o.n,n:o.n}))};`	Return the step, its length, and the points sorted by time as `{t, peak, avg (sum/n), n}`.
4027	`}`	End of function.
4028	`function bulkHit(r,o){`	`bulkHit(r, o)`: describes the hit definition for a link.
4029	`if(o.bands==='policy'){const t=r.upgradeRule.utilPct/100;return {name:'Policy',thr:t,text:`Upgrade policy threshold — utilisation at or above ${bPc(t)}`};}`	Policy: threshold = the link's policy fraction; name "Policy"; text states it.
4030	`const k=o.bands,nm={warn:'Warning',high:'High',crit:'Critical'}[k],t=r.th[k];`	Severity: the band name and the link's own threshold for that band (`r.th[k]`, i.e. the market-hours value) plus a description.
4031	`return {name:nm,thr:t,text:`${nm}${k==='crit'?' only':' and above'} — utilisation at or above ${bPc(t)} (per-link thresholds)`};`	Return `{name, thr, text}`.
4032	`}`	End of function.

Line	Source (verbatim)	What it does
4033	`function bulkPrep(r,o){`	`bulkPrep(r, o)`: everything needed to render one link.
4034	`const H=hitCalc(r, {winStart:o.from,winEnd:o.to,hours:o.hours,bands: o.bands});`	Run `hitCalc` with the pack's window, hours and band choice.
4035	`let covN=0,covFirst=null,covLast=null;`	Coverage counters start empty.
4036	`for(const x of r.samples){if(x.ts<o.from|| x.ts>o.to)continue;covN++;if(covFirst==null|| x.ts<covFirst)covFirst=x.ts;if(covLast==null|| x.ts>covLast)covLast=x.ts;}`	Loop over ALL samples in the window (ignoring the hours filter) counting them and tracking first/last time — so "no data" truly means no samples, not "no samples in these hours".
4037	`return {r,o,H,hit:bulkHit(r,o),total:H.breaches.length,c ovN,covFirst,covLast,`	Return the link, options, `H` (hit results), the hit description, `total` = number of hits, coverage `covN/covFirst/covLast`…
4038	`ser:o.graph?hitSeries(r, {winStart:o.from,winEnd:o.to,hours:o.hours}):null };`	…and `ser` = the bucketed series if graphs were requested, else null.
4039	`}`	End of function.

18.6 Upgrade Policy Hits workbook: genUpgradePolicyXlsx

Line	Source (verbatim)	What it does
3356	`function genUpgradePolicyXlsx(scope){`	`genUpgradePolicyXlsx(scope)`: `scope` is "all" or a link id.
3357	`if(!S.samples.length){toast('No data yet');return;}`	No data loaded → toast and stop.
3358	`const a=A();const list=scope==='all'?a.ifs:a.ifs.filter(r=>r.ifid===scope);`	`a` = analytics; `list` = all links, or just the one whose id equals `scope`.
3359	`if(!list.length){toast('Link not found');return;}`	No such link → toast and stop.
3360	`// Single-link export: if the SAME interface exists under a device-name variant`	Comment: sibling check for single-link exports.
3361	`// (one hostname is a dotted prefix of the other — short name vs FQDN) that holds far`	Comment: the same interface under a device-name variant (short name vs FQDN).
3362	`// more data, the user probably picked a fragment. Flag the busier twin in the report.`	Comment: if that twin holds far more data, the user probably picked a fragment.
3363	`let siblingNote=null;`	`siblingNote` starts null.
3364	`if(scope!=='all'){const r0=list[0];`	Single-link export only: `r0` = the chosen link.
3365	`  const twin=a.ifs.filter(x=>x.ifid!==r0.ifid&&x.intf===r0.intf&&(x.dev.startsWith(r0.dev+'.')||r0.dev.startsWith(x.dev+'.'))&&x.count>Math.max(8,r0.count*2))`	Look for a **twin**: a different link with the same interface name, whose device name is a dotted-prefix variant of this one in either direction, and which has more than max(8, 2× this link's samples)…
3366	`    .sort((x,y)=>y.count-x.count)[0];`	…sort the twins by sample count (descending) and take the first, i.e. the busiest.
3367	`  if(twin)siblingNote=`⚠ Likely wrong link selected. The same interface exists under a device-name variant "${twin.dev} ${twin.intf}" with ${twin.count.toLocaleString()} samples (p95 ${(twin.p95*100).toFixed(0)}%, ${twin.upgradeNeeded?'upgrade trigger MET':'no upgrade'}) — this export is the smaller "${r0.dev} ${r0.intf}" (${r0.count.toLocaleString()} samples). Re-export with the larger one selected, or use "All links".`;`	If a twin exists, write the warning text: twin name, its sample count, its p95, whether its upgrade trigger is met, and the size of the chosen link.
3368	`}`	End of the sibling block.
3369	`const p2=n=>String(n).padStart(2,'0');`	Helper `p2` for two-digit padding.
3370	`const CFS=cfList();`	`CFS` = custom fields (columns in every sheet).
3371	`const fmtTs=ts=>{const d=new Date(ts);return `${d.getFullYear()}-${p2(d.getMonth()+1)}-${p2(d.getDate())} ${p2(d.getHours())}:${p2(d.getMinutes())} ${tzLabel()}`;};`	`fmtTs(ts)`: "yyyy-mm-dd hh:mm TZ" using the wall-clock fields and the zone label.
3372	`// A From/To filter set on the link's page also bounds this export, so the workbook and`	Comment: an on-screen From/To filter also bounds this export.
3373	`// the on-screen window always describe the same period. Pure scoping — unchanged maths.`	Comment: the workbook and the screen describe the same period.
3374	`const tfP=(scope!=='all')?activeTF(scope):null;`	`tfP` = the active filter for a single link, else null (all-links exports ignore the filter).
3375	`const inTF=x=>!tfP||(x.ts>=tfP.from&&x.ts<=tfP.to);`	`inTF(x)`: true for every sample if there is no filter, else only samples inside it.
3376	`const cStart=tfP?Math.max(amin(list.map(r=>r.start)),tfP.from):amin(list.map(r=>r.start));`	`cStart` = coverage start: earliest link start (or the filter start if later).

Line	Source (verbatim)	What it does
3377	`const cEnd=tfP? Math.min(amax(list.map(r=>r.end)),tfP.to):amax(list.map(r=>r.end));`	`cEnd` = coverage end: latest link end (or the filter end if earlier).
3378	`const covDays=Math.max(1,Math.round(days(cStart,cEnd)));`	`covDays` = whole days between them (at least 1).
3379	`// The report is the complete evidence log over ALL loaded history, so the summary`	Comment: the report is the complete evidence log over ALL loaded history…
3380	`// counts (days hit, total hits, trigger met) are computed over exactly the same set of`	Comment: …so summary counts and detail rows are computed over exactly the same set.
3381	`// rows as the Policy Hits detail — the two sheets always agree. (The on-screen verdict`	Comment: the two sheets always agree.
3382	`// instead uses the rolling look-back window from Admin → Link upgrade policy.)`	Comment: the on-screen verdict uses the rolling look-back instead.
3383	`const summary=[`	Start of the `summary` rows array.
3384	`['NetPulse — Link Upgrade Policy Report'],`	Title row.
3385	`[`Generated: ${fmtIST(Date.now())}`,'',`Scope: ${scope==='all'?'all links':list[0].dev+' '+list[0].intf}`],`	Row with the generated time and the scope text (all links, or device + interface).
3386	`[`Coverage: ${fmtIST(cStart)} to ${fmtIST(cEnd)} (${covDays} days, ${tfP?'filtered range — matches the From/To filter set on screen':'entire loaded history'})`],`	Coverage text: from/to, days, and whether it is the filtered range or the entire loaded history.
3387	`['Counts below cover the full loaded history; the live "Upgrade Recommended" verdict uses the rolling '+(S.cfg.upgradeRule.lookbackDays?S.cfg.upgradeRule.lookbackDays+'-day':'full-history')+' look-back window.'],`	Note that counts cover the full loaded history while the live verdict uses the rolling look-back window (days from the setting).
3388	`['NOTE: Every threshold crossing is listed and tagged Market hours vs Non-market hours. '+(list.some(r=>r.upgradeRule.businessHoursOnly)?'Policy = "Market hours only": only Market-hr hits count toward the upgrade trigger (Days threshold hit / Upgrade trigger met); Non-market-hr hits (e.g. overnight/backup peaks) are shown for visibility but do NOT trigger. Change it in Admin → Link upgrade policy.':'Policy = "All hours": both Market-hr and Non-market-hr hits count toward the trigger.')],`	Policy note: states whether only market-hour hits count toward the trigger or all hours do (taken from the policy of the listed links).
3389	`...(list.some(r=>r.capChange)?[['NOTE: '+list.filter(r=>r.capChange).map(r=>`${r.dev} ${r.intf} capacity changed ${r.capChange.distinct.map(v=>(v/1e6).toFixed(0)+' Mbps').join('→')}`).join('; ')+'. Utilization is measured per-sample against the capacity in force at that time; the "Capacity (Mbps)" column shows the latest.']]:[]),`	If any listed link had a capacity change, a note lists each link and its capacities (Mbps).
3390	`...(siblingNote?[[siblingNote]]:[]),`	If a sibling note exists, add it.
3391	`[],`	Blank row.
3392	`['Device','Interface','Link name','Site',...CFS.map(c=>c.label),'Class','Impact','Capacity (Mbps)','Policy threshold %','Policy days','Policy counts','Days threshold hit','Market-hr hits','Non-market-hr hits','Total crossings','Last hit','Data through','Upgrade trigger met','Busy p95 %','Verdict']];`	Header row of the summary table: device, interface, link name, site, custom fields, class, impact, capacity, policy %, policy days, policy counts, days hit, market hits, non-market hits, total crossings, last hit, data through, trigger met, p95, verdict.
3393	`// Detail log lists EVERY crossing of the threshold, tagged Market vs Non-market hours,`	Comment: the detail log lists EVERY crossing.
3394	`// so overnight/backup peaks are visible even when the policy only counts market hours.`	Comment: tagged market vs non-market so overnight peaks are visible.

Line	Source (verbatim)	What it does
3395	`const hits=[['Device','Interface','Site',...CFS.map(c=>c.label),'Timestamp','Period','Utilization %','Transmit (Mbps)','Received (Mbps)','Capacity (Mbps)','Counts toward policy','Policy threshold %']];`	`hits` sheet header: device, interface, site, custom fields, timestamp, period, utilisation, transmit, received, capacity, counts toward policy, policy %.
3396	`// Per-day roll-up: one row per day, market-hour and non-market-hour hits split out.`	Comment: per-day roll-up.
3397	`const perday=[['Device','Interface','Site',...CFS.map(c=>c.label),'Date','Day of week','Market-hr hits','Non-market-hr hits','Total hits','Peak util %','First hit','Last hit','Policy threshold %']];`	`perday` sheet header: device, interface, site, custom fields, date, weekday, market hits, non-market hits, total, peak, first hit, last hit, policy %.
3398	`const hm=ts=>{const d=new Date(ts);return `${p2(d.getHours())}:${p2(d.getMinutes())}`;};`	`hm(ts)`: "hh:mm" string.
3399	`const MAXH=50000;let truncated=false,totalHits=0;`	`MAXH` = 50,000 detail rows; `truncated` flag; `totalHits` counter.
3400	`// Compute each link's crossings first, then sort so links that DID cross the threshold`	Comment: compute crossings first, then sort.
3401	`// appear at the top — otherwise a quiet link can sit in row 1 and the report looks`	Comment: links that crossed appear at the top.
3402	`// empty even when busy links further down have hundreds of crossings.`	Comment: otherwise a quiet link could sit in row 1 and the report look empty.
3403	`const recs=list.map(r=>{const ur=r.upgradeRule;const thr=ur.utilPct/100;`	`recs` = for each link: `ur` its policy and `thr` = utilPct/100…
3404	`  const allCross=r.samples.filter(x=>x.util!=null&&x.util>=thr&&inTF(x));`	…`allCross` = analysed samples at/above the threshold and inside the filter…
3405	`  const mkt=allCross.filter(x=>isBH(x.ts)),nonMkt=allCross.filter(x=>!isBH(x.ts));`	…`mkt` and `nonMkt` split by market hours…
3406	`  const policyHits=ur.businessHoursOnly?mkt:allCross;  // what actually drives the trigger`	…`policyHits` = `mkt` if the policy counts only business hours, else all crossings (this is what drives the trigger)…
3407	`  const daysHit=new Set(policyHits.map(x=>new Date(x.ts).toDateString())).size;`	…`daysHit` = distinct calendar days (`toDateString`) in `policyHits`…
3408	`  return {r,ur,allCross,mkt,nonMkt,policyHits,daysHit,met:daysHit>=ur.days};});`	…and `met` = `daysHit ≥ required days`.
3409	`recs.sort((a,b)=>b.allCross.length-a.allCross.length||b.r.p95-a.r.p95);`	Sort links by number of crossings (descending), ties by p95.
3410	`const linksWithHits=recs.filter(x=>x.allCross.length>0).length;`	Number of links with at least one crossing (for the toast).
3411	`for(const {r,ur,allCross,mkt,nonMkt,policyHits,daysHit,met} of recs){`	Loop over the sorted links.
3412	`  totalHits+=allCross.length;`	Add this link's crossings to the grand total.
3413	`  // "Last hit" vs "Data through" sit side by side so a link whose crossings stopped`	Comment: "Last hit" vs "Data through" side by side.
3414	`  // months ago is obviously NOT a link whose data stops there — coverage still runs`	Comment: a link whose crossings stopped months ago is not a link whose data stops.
3415	`  // to the latest sample. Prevents reading a quiet recent stretch as missing data.`	Comment: avoids reading a quiet recent stretch as missing data.
3416	`  const lastHitTs=policyHits.length?policyHits.reduce((m,x)=>x.ts>m?x.ts:m,0):null;`	`lastHitTs` = latest time among `policyHits` (null if none).

Line	Source (verbatim)	What it does
3417	`summary.push([r.dev,r.intf,r.linkname||'',r.site,...CFS.map(c=>cfVal(r,c.key)),r.cls,r.impact,r.speed?+(r.speed/1e6).toFixed(0):'',ur.utilPct,ur.days,ur.businessHoursOnly?'Market hours only':'All hours',daysHit,mkt.length,nonMkt.length,allCross.length,lastHitTs?fmtTs(lastHitTs):'—',fmtTs(tfP?Math.min(r.end,tfP.to):r.end),met?'YES':'no',+(r.p95*100).toFixed(1),r.verdict.title]);`	Push the summary row: identity, class, impact, capacity (Mbps, rounded), policy %, required days, "Market hours only"/"All hours", days hit, market hits, non-market hits, total, last hit, data-through (or filter end), YES/no, p95 % (1 decimal), verdict title.
3418	`  for(const x of allCross){`	Loop over this link's crossings to build detail rows.
3419	`    if(hits.length>MAXH){truncated=true;break;}`	Stop at the 50,000-row cap and set `truncated`.
3420	`    const bh=isBH(x.ts);`	`bh` = this crossing is in market hours.
3421	`hits.push([r.dev,r.intf,r.site,...CFS.map(c=>cfVal(r,c.key)),fmtTs(x.ts),bh?'Market hours':'Non-market hours',+((x.util*100).toFixed(2)),x.tx!=null?+((x.tx/1e6).toFixed(1)):'',x.rx!=null?+((x.rx/1e6).toFixed(1)):'',r.speed?+((r.speed/1e6).toFixed(0)):'',(!ur.businessHoursOnly||bh)?'Yes':'No',ur.utilPct]);`	Push the detail row: identity, timestamp, Market/Non-market hours, utilisation % (2 decimals), transmit and received Mbps (1 decimal), capacity Mbps, whether it counts toward the policy (Yes if the policy is all-hours or it is a market-hour hit), policy %.
3422	`  }`	End of the detail loop.
3423	`  // roll the crossings up per calendar day, market/non-market split, chronologically`	Comment: roll crossings up per calendar day.
3424	`  const byDay=new Map();`	`byDay` map from date key to day record.
3425	`  for(const x of allCross){const d=new Date(x.ts);const key=`${d.getFullYear()}-${p2(d.getMonth()+1)}-${p2(d.getDate())}`;`	For each crossing: build the local date key.
3426	`    let o=byDay.get(key);if(!o){o={key,dow:DOW[d.getDay()],mkt:0,non:0,peak:0,first:x.ts,last:x.ts};byDay.set(key,o);}`	Create the day record if new: weekday, market count, non-market count, peak, first and last time.
3427	`    if(isBH(x.ts))o.mkt++;else o.non++;if(x.util>o.peak)o.peak=x.util;if(x.ts<o.first)o.first=x.ts;if(x.ts>o.last)o.last=x.ts;}`	Increment market or non-market count; update peak, first and last.
3428	`  for(const o of [...byDay.values()].sort((p,q)=>p.key.localeCompare(q.key)))`	Sort days chronologically…
3429	`perday.push([r.dev,r.intf,r.site,...CFS.map(c=>cfVal(r,c.key)),o.key,o.dow,o.mkt,o.non,o.mkt+o.non,+((o.peak*100).toFixed(1)),hm(o.first),hm(o.last),ur.utilPct]);`	…and push a per-day row: identity, date, weekday, market hits, non-market hits, total, peak % (1 decimal), first and last hit time, policy %.
3430	`  if(truncated)break;`	If the cap was reached stop processing further links.
3431	`}`	End of the link loop.
3432	`if(truncated)hits.push(['NOTE: hit list truncated at 50,000 rows — narrow the scope (single link or shorter window) for full detail']);`	If truncated, append a note row to the detail sheet.
3433	`// Make a zero-hit result unambiguous instead of looking like a broken export.`	Comment: make a zero-hit result unambiguous.
3434	`if(totalHits===0){`	If no crossings at all…
3435	`  const ur0=list[0].upgradeRule;`	…take the first link's policy for the wording…
3436	`  const why=`No link ${scope==='all'?'in this export':'in scope'} crossed ${ur0.utilPct}% in ANY hours (market or non-market) over the ${covDays}-day coverage — utilization stayed below the policy threshold.`;`	…compose the explanation: no link crossed the threshold in any hours over the coverage days.
3437	`  summary.push([]); summary.push(['— No threshold crossings —',why]);`	Add an explicit "— No threshold crossings —" row to the summary…

Line	Source (verbatim)	What it does
3438	`  if(scope!=='all')summary.push(['Tip','This is a single-link export. Choose “All links” in Reports → Upgrade Policy Hits to include links that did cross.']);`	…and a tip for single-link exports suggesting "All links".
3439	`  hits.push(['— No threshold crossings —',why]);`	Same explicit row on the detail sheet.
3440	`  perday.push(['— No threshold crossings —',why]);`	Same on the per-day sheet.
3441	`  if(siblingNote){summary.push([siblingNote]);hits.push([siblingNote]);perday.push([siblingNote]);}`	Repeat the sibling warning on all three sheets if it exists.
3442	`}`	End of the zero-hit block.
3443	`const blob=xlsxBlob([{name:'Summary',rows:summary},{name:'Hits by Day',rows:perday},{name:'Hits by Date-Time',rows:hits}]);`	Build the workbook (stored zip) with three sheets: Summary, Hits by Day, Hits by Date-Time.
3444	`const fname=`NetPulse_Upgrade_Policy_${scope==='all'?'AllLinks':(list[0].dev+'_'+list[0].intf).replace(/\W+/g,'_')}_${new Date().toISOString().slice(0,10)}.xlsx`;`	File name `NetPulse_Upgrade_Policy_<AllLinks|Dev_Intf>_<yyyy-mm-dd>.xlsx` (non-word characters → underscore).
3445	`dl(blob,fname);`	Download (the workspace prefix is added by `dl`).
3446	`logAudit('report.export.xlsx',scope==='all'?'all links':scope,`${totalHits} threshold crossings`);`	Audit-log the export with the crossing count.
3447	`toast(siblingNote?`⚠ Exported the smaller “${list[0].dev} ${list[0].intf}” (${list[0].count.toLocaleString()} samples) — a device-name twin holds far more data. See the note atop the report and re-select.`:(totalHits?`✓ Excel report downloaded — ${totalHits.toLocaleString()} threshold crossing(s) (market + non-market) across ${linksWithHits} link(s)`:`✓ Report downloaded — 0 crossings: no link crossed ${list[0].upgradeRule.utilPct}% in any hours${scope!=='all'?' (try All links)':''}`));`	Toast: a warning if the smaller twin was exported; otherwise the number of crossings and links; or a zero-crossing message.
3448	`}`	End of function.

18.7 Regulatory Capacity Report: data and Excel sheets

Line	Source (verbatim)	What it does
3456	`function timeToLevel(f,L){if(f.current>=L)return 0;return f.slopePerDay>1e-5?(L-f.current)/f.slopePerDay:null;}`	`timeToLevel(f,L)`: days until the forecast reaches level L — 0 if the forecast's current value is already ≥ L; else (L − current) ÷ slope if slope > 0.00001 per day; else null (no upward trend).
3457	`function regReportData(scope){`	`regReportData(scope)`.
3458	`const a=A();const list=scope==='all'?[...a.ifs]:a.ifs.filter(r=>r.ifid===scope);`	`a` = analytics; `list` = every link or the single chosen one.
3459	`const T=S.cfg.regThreshold;`	`T` = the regulatory threshold (Admin → Compliance, default 75%).
3460	`const dataStart=amin(list.map(r=>r.start)),dataEnd=amax(list.map(r=>r.end));`	Data span across the listed links.
3461	`let totalSamples=0,cadDeltas=[];`	`totalSamples` counter (the cadence array is declared but unused).
3462	`const rows=list.map(r=>{`	Build one row per link and sort later.
3463	`  totalSamples+=r.count;`	Add this link's sample count.
3464	`  const mkt=r.samples.filter(x=>isBH(x.ts)&&x.util!=null);`	`mkt` = market-hours samples with a utilisation.
3465	`  const overThr=r.samples.filter(x=>x.util!=null&&x.util>=T);`	`overThr` = all samples at/above T.
3466	`  const mktOver=mkt.filter(x=>x.util>=T);`	`mktOver` = market-hours samples at/above T.
3467	`  const tReg=timeToLevel(r.forecast,T);`	`tReg` = days until the forecast reaches T.
3468	`  const breachDays=new Set(overThr.map(x=>new Date(x.ts).toDateString())).size;`	`breachDays` = distinct calendar days with a sample at/above T.
3469	`  const augment=r.p95>=T||(tReg!=null&&tReg>=0&&tReg<=180);`	`augment` = augmentation required: p95 ≥ T, OR a forecast to reach T within 0–180 days.
3470	`  return {r,peak:r.provisionPeak!=null?r.provisionPeak:(r.pkP95!=null?Math.max(r.pkP95,r.mx):r.mx),p95:r.p95,avg:r.avg,`	Row fields: link; **peak** = `provisionPeak` (or the larger of peak-p95 and max, or max); p95; avg…
3471	`    pctOver:overThr.length/(r.count||1),mktPctOver:mktOver.length/(mkt.length||1),`	…`pctOver` = share of all samples ≥ T; `mktPctOver` = share of market-hour samples ≥ T…
3472	`    headThr:Math.max(0,T-r.p95),head100:Math.max(0,1-r.p95),breachDays,tReg,augment};`	…headroom to T = max(0, T − p95); headroom to 100% = max(0, 1 − p95); breach days; `tReg`; `augment`.
3473	`}).sort((x,y)=>y.p95-x.p95);`	Sort rows by p95, highest first.
3474	`const breaching=rows.filter(x=>x.p95>=T).length,augCount=rows.filter(x=>x.augment).length;`	Count links with p95 ≥ T and links needing augmentation.
3475	`return {list,rows,T,dataStart,dataEnd,totalSamples,breaching,augCount,`	Return `{list, rows, T, dataStart, dataEnd, totalSamples, breaching, augCount, …}`.
3476	`covDays:Math.max(1,Math.round(days(dataStart,dataEnd)))};`	…and `covDays` = whole days of coverage (at least 1).
3477	`}`	End of function.

Line	Source (verbatim)	What it does
3478	`function tzLabel(){const tz=S.cfg.tz||'Asia/Kolkata';const KN={'Asia/Kolkata':'IST'};if(KN[tz])return KN[tz];try{const p=new Intl.DateTimeFormat('en-US',{timeZone:tz,timeZoneName:'short'}).formatToParts(Date.now()).find(x=>x.type==='timeZoneName');return p?p.value:tz;}catch(e){return tz;}}`	`tzLabel()`: "IST" for Asia/Kolkata; otherwise the short zone name from `Intl.DateTimeFormat` (the zone id if that fails). It is a LABEL only.
3479	`// Data timestamps are wall-clock in the configured zone (naive exports carry no offset).`	Comment: data timestamps are wall-clock in the configured zone.
3480	`// Parsing, bucketing (isBH / heatmaps / anomalies) and every rendered time all sit on the`	Comment: parsing, bucketing and rendering all use the same clock.
3481	`// SAME clock, so the report cover, the breach-detail rows and the market-hours flags never`	Comment: so cover, detail rows and market flags never disagree.
3482	`// disagree across browsers. fmtIST just labels that wall-clock with the configured zone.`	Comment: `fmtIST` just labels the wall clock with the zone.
3483	`function fmtIST(ts){const d=new Date(ts),p2=n=>String(n).padStart(2,'0');const mon=['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec'][d.getMonth()];return `${d.getDate()} ${mon} ${d.getFullYear()}, ${p2(d.getHours())}:${p2(d.getMinutes())} ${tzLabel()}`;}`	`fmtIST(ts)`: "d Mon yyyy, hh:mm TZ" from the local fields.
3484	`function genRegReport(scope,fmt){`	`genRegReport(scope, fmt)`: fmt = xlsx | pdf | html.
3485	`if(!S.samples.length){toast('No data yet');return;}`	No data → toast, stop.
3486	`const D=regReportData(scope);const c=S.cfg;const bh=c.businessHours;`	`D` = report data; `c` = config; `bh` = business hours.
3487	`const p2=n=>String(n).padStart(2,'0');const bhWin=`${p2(bh.startHr)}:${p2(bh.startMin||0)}–${p2(bh.endHr)}:${p2(bh.endMin||0)}`;`	`bhWin` = "hh:mm–hh:mm" text of the market window.
3488	`const Tp=(D.T*100).toFixed(0);`	`Tp` = threshold as a whole-number percent string.
3489	`const N=S.cfg.provisionFactor||1.5;`	`N` = provisioning factor (default 1.5).
3490	`logAudit('report.regulatory',scope==='all'?'all links':scope,fmt.toUpperCase());`	Audit-log the report.
3491	`if(fmt==='xlsx'){`	Excel branch begins.
3492	`  const cover=[`	Start of the `cover` rows array.
3493	`    ['Regulatory Capacity Utilization & Augmentation Report'],`	Title row.
3494	`    ['Institution',c.institution||'(set in Admin → Compliance)'],`	Institution name (or a prompt to set it in Admin → Compliance).
3495	`    ['Report period',`${fmtIST(D.dataStart)} to ${fmtIST(D.dataEnd)} (${D.covDays} days)`],`	Report period: from, to and the number of days.
3496	`    ['Generated',fmtIST(Date.now())],`	Generated time.
3497	`    ['Scope',scope==='all'?`All monitored links (${D.list.length})`:D.list[0].dev+' '+D.list[0].intf],`	Scope: all monitored links (with count) or the chosen device + interface.
3498	`    ['Regulatory capacity threshold',`${Tp}% (configurable — set per applicable circular)`],`	Regulatory capacity threshold, stated as configurable.
3499	`    ['Provisioning target',`installed capacity ≥ ${N}× peak (${({p99:'p99',p999:'p99.9',max:'max'})[c.peakBasis||'p999']} basis)`],`	Provisioning target: installed capacity ≥ N × peak, naming the peak basis (p99 / p99.9 / max).

Line	Source (verbatim)	What it does
3500	`['Projection basis',` `${(c.forecastWindowDays!=null?c.forecastWindowDays:180)>0?'trailing '+(c.forecastWindowDays!=null?c.forecastWindowDays:180)+' days':'all loaded history'}, ${(c.forecastCI*100||80).toFixed(0)}% confidence range`],`	Projection basis: trailing window days or all loaded history, and the confidence-band percentage.
3501	`['Business / market hours',` `${bhWin}, ${bh.days.map(d=>DOW[d]).join(' ')}`],`	Business / market hours window and days.
3502	`['Log retention basis',` `${c.regRetentionDays} days`],`	Log retention basis (days) — a statement, not a deletion rule.
3503	`['Total samples analysed',D.totalSamples],`	Total samples analysed.
3504	`['Links at/above threshold (p95)',D.breaching],`	Number of links with p95 at/above the threshold.
3505	`['Links requiring augmentation',D.augCount],`	Number of links requiring augmentation.
3506	`[],`	Blank row.
3507	`['Prepared by',c.reportPreparedBy||'','','Reviewed by',c.reportReviewedBy||'','','Approved by',c.reportApprovedBy||''],`	Signatories: prepared / reviewed / approved by.
3508	`['Classification','Confidential — regulatory']`	Classification line ("Confidential — regulatory").
3509	`];`	End of the cover rows.
3510	`const ciPct=(S.cfg.forecastCI*100||80).toFixed(0);`	`ciPct` = confidence band as a whole-number percent.
3511	`const summ=[['Device','Interface','Link name','Site',...cfList().map(c=>c.label),'Class','Impact','Capacity (Mbps)',` `Peak %`,`p95 %`,`Avg %`,`% time > ${Tp}% (all)`,`% time > ${Tp}% (mkt hrs)`,`Days > ${Tp}%`,`Headroom to ${Tp}% (pp)`,`Headroom to 100% (pp)`,`Days to ${Tp}%`,`Days to ${Tp}% (${ciPct}% range)`,`Recommended capacity (${N}×peak, Mbps)`,`Meets ${N}× headroom`,'Augmentation required','Verdict']];`	`summ` header: device, interface, link name, site, custom fields, class, impact, capacity, Peak %, p95 %, Avg %, % time > T (all), % time > T (market hrs), days > T, headroom to T, headroom to 100%, days to T, days to T range, recommended capacity, meets N× headroom, augmentation required, verdict.
3512	`for(const x of D.rows){const recMbps=x.r.speed?+((x.peak*x.r.speed*N)/1e6).toFixed(0):'';`	For each row: recommended capacity (Mbps) = peak × link capacity × N ÷ 1e6.
3513	`const headroomOK=x.r.speed?(x.peak*N<=1?'YES':'NO — below '+N+'×'):'—';`	Headroom test: if the capacity is known, "YES" when peak × N ≤ 1, else "NO — below N×"; "—" if capacity unknown.
3514	`const f=x.r.forecast,dTo=sl=>f.current>=D.T?0:(sl>1e-5?(D.T-f.current)/sl:null);const tLo=dTo(f.slopeHi),tHi=dTo(f.slopeLo);`	Range of days to T: `dTo(slope)` = 0 if already ≥ T, else (T − current) ÷ slope for slope > 0.00001, else null; early bound uses the high slope, late bound the low slope.
3515	`const tRange=x.tReg==null?'no trend':(x.tReg<=0?'AT/OVER':(tLo!=null?`${Math.round(Math.max(0,tLo))}–${tHi!=null&&tHi<3650?Math.round(tHi):'>365'}`:'—'));`	Text of the range: "no trend" (no positive slope); "AT/OVER" (already there); otherwise "early–late" rounded, with ">365" when the late bound is ≥3650 days; "—" if the early bound is null.
3516	`summ.push([x.r.dev,x.r.intf,x.r.linkname||'',x.r.site,...cfList().map(c=>cfVal(x.r,c.key)),x.r.cls,x.r.impact,x.r.speed?+(x.r.speed/1e6).toFixed(0):'',`	Push the row: identity, class, impact, capacity (Mbps), then…
3517	`+(x.peak*100).toFixed(1),+(x.p95*100).toFixed(1),+(x.avg*100).toFixed(1),+(x.pctOver*100).toFixed(2),+(x.mktPctOver*100).toFixed(2),x.breachDays,`	…peak %, p95 %, avg % (1 decimal), % time over T (all, 2 decimals) and in market hours, breach days…

Line	Source (verbatim)	What it does
3518	`+(x.headThr*100).toFixed(1),+(x.head100*100).toFixed(1),x.tReg==null?'no trend':(x.tReg<=0?'AT/OVER':Math.round(x.tReg)),tRange,recMbps,headroomOK,x.augment?'YES':'No',x.r.verdict.title]);}`	…headroom to T and to 100% (percentage points), days to T (or "no trend"/"AT/OVER"), the range, recommended capacity, headroom check, YES/No augmentation, verdict title.
3519	`const breach=[['Device','Interface','Site','Timestamp','Utilization %','Transmit (Mbps)','Received (Mbps)','Capacity (Mbps)','Market hours',`Threshold %`]];`	`breach` sheet header: device, interface, site, timestamp, utilisation, transmit, received, capacity, market hours, threshold.
3520	`let n=0,trunc=false;`	Counters `n` and `trunc` for the 50,000-row cap.
3521	`for(const x of D.rows){for(const s of x.r.samples){if(s.util==null||s.util<D.T)continue;if(n++>=50000){trunc=true;break;}`	For each link row and each sample ≥ T (skipping samples without utilisation or below T), stop at the cap…
3522	`breach.push([x.r.dev,x.r.intf,x.r.site,fmtTsPlain(s.ts),+((s.util*100).toFixed(2)),s.tx!=null?+((s.tx/1e6).toFixed(1)):'',s.rx!=null?+((s.rx/1e6).toFixed(1)):'',x.r.speed?+((x.r.speed/1e6).toFixed(0)):'',isBH(s.ts)?'Yes':'No',Tp]);}`	…push a detail row (timestamp without seconds, utilisation 2 decimals, Mbps 1 decimal, market-hours Yes/No, threshold).
3523	`if(trunc)break;}`	If truncated stop the outer loop too.
3524	`if(trunc)breach.push(['NOTE: truncated at 50,000 rows — narrow scope/period for full detail']);`	If truncated append a note row.
3525	`if(breach.length===1)breach.push([`— No samples at or above the ${Tp}% regulatory threshold over the ${D.covDays}-day coverage —`,'','','Utilization stayed below the threshold; see the Capacity Summary sheet for each link's p95, peak and headroom.']);`	If no breaches, add an explicit "no samples at or above T" row pointing to the Capacity Summary.
3526	`const defs=[['Definitions & Methodology'],[],`	`defs` sheet (Definitions & Methodology): title…
3527	`['Utilization','Traffic ÷ capacity, taken as the worse of Transmit and Receive per sample.'],`	…utilisation definition (worse of transmit and receive per sample)…
3528	`['Peak %','Highest observed utilization (peak columns where available).'],`	…peak definition…
3529	`['p95 %','95th-percentile utilization — sustained busy level; the basis for capacity assessment.'],`	…p95 definition (the capacity-assessment basis)…
3530	`['Regulatory threshold',`Configured capacity ceiling (${Tp}%). Links operating at/above it on a sustained basis require augmentation.`],`	…regulatory threshold definition…
3531	`['Market hours',`${bhWin} on business days — the window in which utilization is most material for a trading system.`],`	…market hours definition…
3532	`['Days to threshold','Projected days until p95 reaches the threshold at the current trend (robust regression on daily p95).'],`	…days-to-threshold definition (robust regression on daily p95)…
3533	`['Augmentation required',`Flagged when p95 ≥ ${Tp}% or projected to reach it within 180 days.`],`	…augmentation rule (p95 ≥ T or projected within 180 days)…
3534	`[],['Regulatory alignment (indicative — validate against your current circulars)'],`	…heading for the indicative regulatory alignment…
3535	`['Capacity monitoring & augmentation','Peak utilization kept within configured limits with headroom for growth.'],`	…capacity monitoring and augmentation statement…
3536	`['IT operations & resilience','Capacity management, monitoring and reporting as part of IT operations & resilience.'],`	…IT operations and resilience statement…
3537	`['Logging & retention','Log enablement and configurable retention; synchronized time source.'],`	…logging and retention statement…

Line	Source (verbatim)	What it does
3538	`    ],['Disclaimer','NetPulse provides the underlying data and structure. The threshold value and report format must be validated by the compliance team against the current applicable regulatory circulars.']];`	…disclaimer: the threshold and format must be validated by the compliance team.
3539	`  const blob=xlsxBlob([{name:'Cover',rows:cover},{name:'Capacity Summary',rows:summ},{name:'Threshold Breaches',rows:breach},{name:'Definitions',rows:defs}]);`	Build the four-sheet workbook: Cover, Capacity Summary, Threshold Breaches, Definitions.
3540	`  dl(blob,`Regulatory_Capacity_Report_${scope==='all'?'AllLinks':(D.list[0].dev+'_'+D.list[0].intf).replace(/\W+/g,'_')}_${new Date().toISOString().slice(0,10)}.xlsx`);`	Download as `Regulatory_Capacity_Report_<AllLinks|Dev_Intf>_<date>.xlsx`.
3541	`  toast('✓ Regulatory Excel report downloaded');return;`	Toast and return (the PDF/HTML branch is skipped for xlsx).
3542	`}`	End of the xlsx branch.

18.8 Processed-data export: exportProcessed (the audit trail)

Line	Source (verbatim)	What it does
2827	`// Export the processed samples for a date range so the data can be audited end-to-end`	Comment: purpose — export processed samples for end-to-end audit against the source file.
2828	`// against the source file: raw rate, per-sample capacity, derived %, market/non-market,`	Comment: raw rate, per-sample capacity, derived %, market/non-market, policy-hit flag.
2829	`// and the policy-hit flag — all derived live from the Admin-set policy (nothing hardcoded).`	Comment: all derived live from the Admin policy — nothing hard-coded.
2830	`// Artifact rows (util above the Admin validity ceiling / rate above capacity) are included`	Comment: artifact rows are included with their exclusion reason…
2831	`// with the exclusion reason, so nothing is silently dropped from the audit.`	Comment: …so nothing is silently dropped from the audit.
2832	`function exportProcessed(fmt){`	`exportProcessed(fmt)`: fmt = "csv" or "xlsx".
2833	`if(!S.samples.length){toast('No data yet');return;}`	No data → toast and stop.
2834	`const scope=($('#pdScope')||{}).value||'all';`	`scope` = the link selected in the panel ("all" by default).
2835	`const from=($('#pdFrom')\|\|{}).value?new Date($('#pdFrom').value).getTime():-Infinity;`	`from` = From input as local time, or −∞ if blank.
2836	`const to=($('#pdTo')\|\|{}).value?new Date($('#pdTo').value).getTime():Infinity;`	`to` = To input as local time, or +∞ if blank.
2837	`const ur=S.cfg.upgradeRule,thr=ur.utilPct/100,flag=S.cfg.flagUtilAbove||1.0;`	`ur` = upgrade policy, `thr` = policy fraction, `flag` = over-capacity flag level.
2838	`const p2=n=>String(n).padStart(2,'0');`	Helper `p2`.
2839	`// seconds included so the audit reconciles to the exact source timestamp (e.g. 00:00:46)`	Comment: seconds are included so the audit reconciles to the exact source timestamp.
2840	`const fmtTs=ts=>{const d=new Date(ts);return `${d.getFullYear()}-${p2(d.getMonth()+1)}-${p2(d.getDate())} ${p2(d.getHours())}:${p2(d.getMinutes())}:${p2(d.getSeconds())} ${tzLabel()}`;};`	`fmtTs(ts)`: "yyyy-mm-dd hh:mm:ss TZ".
2841	`const sel=S.samples.filter(x=>x.ts>=from&&x.ts<=to&&(scope==='all'||x.ifid===scope));`	`sel` = ALL stored samples (`S.samples`, NOT the analysed ones) inside the time range and, if chosen, for the one link. Because it reads the raw store, maintenance and invalid rows are included.
2842	`if(!sel.length){toast('No samples in that range/link');return;}`	No rows → toast and stop.
2843	`sel.sort((a,b)=>a.ifid<b.ifid?-1:a.ifid>b.ifid?1:a.ts-b.ts);`	Sort by link id then time.
2844	`const MAX=fmt==='xlsx'?200000:1000000;let trunc=false;`	Row cap: 200,000 for Excel, 1,000,000 for CSV; `trunc` flag.
2845	`const use=sel.length>MAX?(trunc=true,sel.slice(0,MAX)):sel;`	If over the cap, keep the first MAX rows and set `trunc`.
2846	`const CFS=cfList();`	`CFS` = custom fields.
2847	`// per-link custom-field values, resolved once (override wins over file value)`	Comment: per-link custom-field values resolved once.
2848	`const cfByIf={};for(const rr of A().ifs){const o={};for(const c of CFS)o[c.key]=cfVal(rr,c.key);cfByIf[rr.ifid]=o;}`	`cfByIf` = for each link, its custom-field values (override beats file value).

Line	Source (verbatim)	What it does
2849	`const hdr=['Device','Interface',...CFS.map(c=>c.label),'Timestamp (local)','Epoch ms','Transmit (Mbps)','Received (Mbps)','Peak Tx (Mbps)','Peak Rx (Mbps)','Capacity Tx (Mbps)','Capacity Rx (Mbps)','Transmit %','Receive %','Utilization % (max)','Peak %','Period','Policy threshold %','Counts as policy hit','Capacity review','Excluded from analytics'];`	`hdr` = header row: device, interface, custom fields, timestamp, epoch ms, transmit, received, peak Tx, peak Rx, capacity Tx, capacity Rx, transmit %, receive %, utilisation % (max), peak %, period, policy threshold %, counts as policy hit, capacity review, excluded from analytics.
2850	`const rows=[hdr];`	`rows` starts with the header.
2851	`for(const x of use){`	Loop over the selected samples.
2852	`  const bad=!!x.bad,bh=isBH(x.ts),mnt=inMaintWin(x);`	`bad` = stored invalid flag; `bh` = market hours; `mnt` = inside a maintenance window **right now** (live).
2853	`  const txp=(x.tx!=null&&x.speedTx)?x.tx/x.speedTx:null;`	Transmit % = transmit rate ÷ that sample's transmit capacity (blank if unknown).
2854	`  const rxp=(x.rx!=null&&x.speedRx)?x.rx/x.speedRx:null;`	Receive % likewise.
2855	`  const hit=!bad&&x.util!=null&&x.util>=thr&&(!ur.businessHoursOnly||bh);`	`hit` = not invalid, has a utilisation ≥ policy fraction, and (market hours if the policy is business-hours-only).
2856	`rows.push([x.dev,x.intf,...CFS.map(c=>(cfByIf[x.ifid]||{})[c.key]||''),fmtTs(x.ts),x.ts,`	Push the row: device, interface, custom fields, timestamp, epoch ms…
2857	`    x.tx!=null?+(x.tx/1e6).toFixed(3):'',x.rx!=null?+(x.rx/1e6).toFixed(3):'',`	…transmit and receive in Mbps (3 decimals)…
2858	`    x.ptx!=null?+(x.ptx/1e6).toFixed(3):'',x.prx!=null?+(x.prx/1e6).toFixed(3):'',`	…peak transmit and receive in Mbps…
2859	`x.speedTx?+(x.speedTx/1e6).toFixed(0):'',x.speedRx?+(x.speedRx/1e6).toFixed(0):'',`	…capacity transmit and receive in Mbps…
2860	`    txp!=null?+(txp*100).toFixed(2):'',rxp!=null?+(rxp*100).toFixed(2):'',`	…transmit % and receive % (2 decimals)…
2861	`    x.util!=null?+(x.util*100).toFixed(2):'',x.pu!=null?+(x.pu*100).toFixed(2):'',`	…utilisation % (max) and peak % (2 decimals) …
2862	`    bh?'Market hours':'Non-market hours',ur.utilPct,bad?'invalid (negative)':(hit?'Yes':'No'),`	…Market/Non-market hours, policy %, and "invalid (negative)" / "Yes" / "No" for the hit flag…
2863	`    (x.util!=null&&x.util>flag)?`⚠ over ${Math.round(flag*100)}% — verify capacity (kept, not dropped)`:'',`	…a capacity-review message when utilisation exceeds the flag level (the sample is kept)…
2864	`    mnt?'Yes — maintenance window':(bad?'Yes — invalid (negative) reading':'No — included')]);`	…and the exclusion reason: "Yes — maintenance window", "Yes — invalid (negative) reading", or "No — included".
2865	`}`	End of the loop.
2866	`if(trunc)rows.push([`NOTE: truncated at ${MAX.toLocaleString()} rows — narrow the date range or pick a single link for the full slice.`]);`	If truncated, append a note row saying how many rows and to narrow the range.
2867	`const stamp=new Date().toISOString().slice(0,10),scopeLbl=scope==='all'?'AllLinks':scope.replace(/\W+/g,'_');`	File name parts: date stamp and a scope label.
2868	`if(fmt==='csv'){`	CSV branch: …
2869	`  const csv=rows.map(r=>r.map(c=>{const s=String(c==null?'':c);return /[",\n]/.test(s)?'"'+s.replace(/"/g,'""')+'"':s;}).join(',')).join('\n');`	…join cells, quoting any that contain a comma, quote or newline, doubling quotes…

Line	Source (verbatim)	What it does
2870	`  dl(new Blob([csv],{type:'text/csv;charset=utf-8'}),`NetPulse_ProcessedData_${scopeLbl}_${stamp}.csv`);`	…download as `NetPulse_ProcessedData_<scope>_<date>.csv` (UTF-8).
2871	`}else{`	Excel branch: …
2872	`  dl(xlsxBlob([{name:'Processed Data',rows}]),`NetPulse_ProcessedData_${scopeLbl}_${stamp}.xlsx`);`	…one-sheet workbook "Processed Data".
2873	`}`	End of the branch.
2874	`logAudit('data.export.processed',scope,`${use.length} rows${trunc?' (truncated)':''}`);`	Audit-log the export with the row count.
2875	`toast(`⬇ ${use.length.toLocaleString()} processed sample(s) exported${trunc?' — truncated, narrow the range':''}`);`	Toast with the number of rows exported (and a truncation warning).
2876	`}`	End of function.

18.9 Multi-link Excel pack: bulkXlsx and its helpers

Line	Source (verbatim)	What it does
4259	`/* ---- Excel pack: Summary + charts + hit tables, every figure from the same bulkPrep() -------- */`	Comment: the Excel pack — Summary, charts and hit tables, all from the same `bulkPrep` results.
4260	`const xlSerial=ts=>{const d=new Date(ts);return Date.UTC(d.getFullYear(),d.getMonth(),d.getDate(),d.getHours(),d.getMinutes(),d.getSeconds())/86400000+25569;};`	`xlSerial(ts)`: converts a wall-clock timestamp to an Excel date-time number: it takes the LOCAL calendar fields, rebuilds them as a UTC instant, divides by 86,400,000 and adds 25569 (days from 1899-12-30 to 1970-01-01). The cell therefore displays the same clock time as the app.
4261	`const xlDaySerial=key=>{const [y,m,d]=key.split('-').map(Number);return Date.UTC(y,m-1,d)/86400000+25569;};`	`xlDaySerial(key)`: the same for a "yyyy-mm-dd" key (date only).
4262	`const xlDT=ts=>({v:xlSerial(ts),s:'dt'});`	`xlDT(ts)`: a styled cell `{v: serial, s: "dt"}` (format yyyy-mm-dd hh:mm:ss).
4263	`function xlName(raw,used){ // Excel sheet names: ≤31 chars, none of []:*?/\ , unique ignoring case`	`xlName(raw, used)`: Excel sheet-name sanitiser (≤31 chars, none of [ ] : * ? / \, unique ignoring case).
4264	`const base=String(raw).replace(/[\[\]:*?\/\\]/g,'-').replace(/^'+|'+$/g,'').trim().slice(0,31)||'Sheet';`	Replace forbidden characters with "-", strip leading/trailing apostrophes, trim, cut to 31 characters, default "Sheet".
4265	`let n=base,k=2;while(used.has(n.toLowerCase())){const sfx=` (${k++})`;n=base.slice(0,31-sfx.length)+sfx;}`	If the lower-cased name is taken, append " (2)", " (3)"... shortening the base so the total stays ≤31.
4266	`used.add(n.toLowerCase());return n;`	Record the lower-cased name as used and return it.
4267	`}`	End of function.
4268	`const xlRef=(sheet,col,r1,r2)=>`${xlQ(sheet)}!$${colL(col)}$${r1}`+(r2?`:$${colL(col)}$${r2}`:'');`	`xlRef(sheet, col, r1, r2)`: an absolute reference such as `'Sheet'!$B$6:$B$40` for chart series (col is 0-based, rows 1-based).
4269	`const xlHdr=a=>a.map(t=>({v:t,s:'hdr'}));`	`xlHdr(a)`: turn header strings into header-styled cells.
4271	`function bulkXlsx(preps,o){`	`bulkXlsx(preps, o)`: returns the list of sheets (name, rows, widths, frozen rows, filter range, charts). Every number comes from the `bulkPrep` results.
4272	`const used=new Set(),cf=cfList(),inst=S.cfg.institution||'Network Operations';`	`used` = sheet names already taken; `cf` = custom fields; `inst` = institution name (default "Network Operations").
4273	`const sorted=[...preps].sort((a,b)=>b.total-a.total||b.H.peak-a.H.peak); // busiest links first`	`sorted` = links ordered by total hits (descending), ties by peak utilisation — busiest first. This order is used on every sheet.
4274	`const out=[],hitLbl=sorted.length?(o.bands==='policy'?'Upgrade policy threshold (each link's own policy percentage)':({warn:'Warning and above',high:'High and above',crit:'Critical only'})[o.bands]+' — each link's own thresholds'):'';`	`hitLbl` = the "Hits counted as" text: policy wording, or the severity wording followed by "— each link's own thresholds"; `out` is unused.

Line	Source (verbatim)	What it does
4275	`const sumName=xlName('Summary',used);`	Reserve the sheet name "Summary".
4277	`// ---- Summary ----------...`	Comment: Summary sheet.
4278	`const head=['Device','Interface','Link name','Site',...cf.map(c=>c.label),'Class','Impact','Capacity (Mbps)','Data from','Data to','Samples in period',`	`head` = Summary column headers: device, interface, link name, site, custom fields, class, impact, capacity (Mbps), data from, data to, samples in period…
4279	`'Total hits','Warning','High','Critical','Market-hr hits','Non-market hits','Distinct hit days','Peak utilization','Peak time','First hit','Last hit','Policy %','Policy days hit','Upgrade trigger met'];`	…total hits, warning, high, critical, market-hr hits, non-market hits, distinct hit days, peak utilisation, peak time, first hit, last hit, policy %, policy days hit, upgrade trigger met.
4280	`const totalHits=sorted.reduce((a,P)=>a+P.total,0);`	`totalHits` = sum of all links' hit counts.
4281	`const sum=[`	Start of the `sum` rows (the nine preamble rows come first).
4282	`[{v:'Threshold Hit Report — Summary',s:'title'}],`	Row 1: title "Threshold Hit Report — Summary".
4283	`[{v:inst,s:'bold'}],`	Row 2: institution (bold).
4284	`[{v:`Generated ${bT(Date.now())} ${tzLabel()}`,s:'note'}],`	Row 3: generated time and zone label (note style).
4285	`[{v:'Reporting period',s:'bold'},`${bD(o.from)} to ${bD(o.to)} (${Math.max(1,Math.round((o.to-o.from)/86400000))} days)`],`	Row 4: reporting period and the number of days (at least 1).
4286	`[{v:'Hits counted as',s:'bold'},hitLbl],`	Row 5: hits counted as…
4287	`[{v:'Hours basis',s:'bold'},HOURS_LBL[o.hours]],`	Row 6: hours basis (all / market / non-market).
4288	`[{v:'Links / total hits',s:'bold'},`${sorted.length.toLocaleString()} link(s) · ${totalHits.toLocaleString()} hit(s)`],`	Row 7: number of links and total hits.
4289	`...(sorted.some(P=>!P.covN)?[[{v:`${sorted.filter(P=>!P.covN).length} link(s) have no samples in this period (see "Samples in period" = 0). Their hit counts are 0 only because there is no data to measure.`,s:'bold'}]]:[[]]),`	Row 8: if any link has no samples in the period, a bold warning that their zero hits mean "no data"; otherwise an empty row (keeps the header on row 10).
4290	`[{v:'Utilization = busier of Transmit / Receive, measured per sample against the capacity in force at that time. Timestamps are '+tzLabel()+'. Maintenance windows and invalid readings are excluded; they remain in the source data.',s:'note'}],`	Row 9: note — utilisation = busier of transmit/receive per sample against the capacity in force; times in the zone; maintenance and invalid readings excluded but still in the source.
4291	`xlHdr(head)];`	Row 10: the header row.
4292	`for(const P of sorted){`	Loop over links, busiest first.
4293	`const {r,H}=P,ur=r.upgradeRule,first=H.breaches[0],last=H.breaches.at(-1);`	`r` = link, `H` = hit results, `ur` = its policy, `first`/`last` = first and last hit.
4294	`sum.push([r.dev,r.intf,r.linkname||'',r.site,...cf.map(c=>cfVal(r,c.key)),r.cls,r.impact,r.speed?+(r.speed/1e6).toFixed(0):'',`	Summary row, part 1: device, interface, link name, site, custom fields, class, impact, capacity in Mbps (whole number, blank if unknown)…
4295	`P.covFirst!=null?xlDT(P.covFirst):'—',P.covLast!=null?xlDT(P.covLast):'—',{v:P.covN,s:'int'},`	…part 2: coverage first and last time (date-time cells or "—") and samples in the period (integer format)…

Line	Source (verbatim)	What it does
4296	`{v:P.total,s:'int'},H.cW,H.cH,H.cC,H.mWarn,H.nWarn,H.dW.size,{v:H.peak,s:'pct'},H.peakTs!=null?xlDT(H.peakTs):'—',`	...part 3: total hits (integer format), warning, high, critical, market-hour hits, non-market hits, distinct hit days, peak (percent cell) and peak time...
4297	`first?xlDT(first.ts):'—',last?xlDT(last.ts):'—',ur.utilPct,r.upgradeDays,r.upgradeNeeded?'YES':'no']);`	...part 4: first hit and last hit times, policy %, **policy days hit** (`r.upgradeDays` — the rolling look-back count, which can differ from the hit days shown in the same row), and "YES"/"no" for the upgrade trigger.
4298	`}`	End of the loop.
4299	`if(!sorted.length)sum.push(['— No links selected —']);`	If no links were selected, a placeholder row.
4300	`const sheets=[{name:sumName,rows:sum,freeze:10,filter:`A10:${colL(head.length-1)}${10+Math.max(1,sorted.length)}`,`	Sheet definition 1: the Summary — frozen at row 10, autofilter over the table (`A10:` to the last column and row), ...
4301	`cols:[24,22,22,10,...cf.map(()=>16),11,10,14,18,18,14,11,10,8,9,14,15,14,16,18,18,18,10,14,16]}];`	...and column widths (custom-field columns 16 wide).
4303	`// ---- Top-links bar chart ----------...`	Comment: top-links bar chart.
4304	`if(o.graph){`	Only if "Utilisation graph" was ticked.
4305	`const nm=xlName('Top Links Chart',used),top=sorted.filter(P=>P.total>0).slice(0,25),pol=o.bands==='policy';`	Sheet "Top Links Chart"; `top` = the first 25 links that have at least one hit; `pol` = policy mode.
4306	`const rows=[xlHdr(['Link','Warning','High','Critical',...(pol?['Below warning']:[])])];`	Data table header: link, warning, high, critical (plus "Below warning" in policy mode).
4307	`top.forEach(P=>rows.push([`${P.r.dev} ${P.r.intf}`,P.H.cW,P.H.cH,P.H.cC,...(pol?[P.H.cN]:[])]));`	One row per top link: "device interface" and its band counts.
4308	`if(!top.length)rows.push(['— No hits in this period —']);`	If there are none, a placeholder row.
4309	`const bands=[['Warning','#B45309'],['High','#C2410C'],['Critical','#E11D48'],...(pol?[['Below warning','#64748B']]:[])];`	Band list and colours: Warning amber, High orange, Critical red, Below warning grey (policy mode only).
4310	`sheets.push({name:nm,rows,cols:[34,11,11,11,14],freeze:1,`	Sheet definition 2: name, rows, widths, frozen header...
4311	`charts:top.length?[{type:'col',stacked:true,title:`Hits per link — ${top.length>=25?'top 25 of '+sorted.filter(P=>P.total>0).length:top.length+' link(s) with hits'}`,`	...a native **stacked column chart** (only if there are hit links) titled "Hits per link — top 25 of N" or "n link(s) with hits"...
4312	`cats:xlRef(nm,0,2,1+top.length),series:bands.map(([t,c],i)=>({name:xlRef(nm,1+i,1),val:xlRef(nm,1+i,2,1+top.length),color:c})),anchor:{c0:6,r0:1,c1:20,r1:24}}]:[]});`	...categories = the link-name column; one series per band, named and valued by cell references into the data table...
4313	`}`	End of the graph block (chart anchor 6–20 columns, rows 1–24 is on the line above).
4315	`// ---- Hits by date ----------...`	Comment: hits by date.
4316	`if(o.byDate){`	Only if "Hits by date" was ticked.
4317	`const rows=[xlHdr(['Device','Interface','Date','Day','Warning','High','Critical','Total hits','Market-hr hits','Non-market hits','Peak utilization','Peak time','First hit','Last hit'])];`	Header: device, interface, date, day, warning, high, critical, total hits, market-hr hits, non-market hits, peak utilisation, peak time, first hit, last hit.

Line	Source (verbatim)	What it does
4318	`  for(const P of sorted)for(const dk of Object.keys(P.H.byDay).sort()){const d=P.H.byDay[dk];`	For each link and each hit day (sorted keys): `d` = that day's record from `hitCalc`…
4319	`    rows.push([P.r.dev,P.r.intf,{v:xlDaySerial(dk),s:'date'},DOW[new Date(d.first).getDay()],d.w||'',d.h||'',d.c||'',{v:d.n,s:'int'},d.m,d.n-d.m,{v:d.peak,s:'pct'},bHM(d.peakTs),bHM(d.first),bHM(d.last)]);}`	…push a row: device, interface, date (date-format cell), weekday of the first hit, warning/high/critical counts (blank when zero), total (integer), market count, non-market = total − market, peak (percent), and peak/first/last times as "hh:mm:ss".
4320	`  if(rows.length===1)rows.push(['— No hits in this period —']);`	If no rows were added, a placeholder row.
4321	`  sheets.push({name:xlName('Hits by Date',used),rows,freeze:1,filter:`A1:N${rows.length}`,cols:[24,22,12,7,10,8,9,11,14,15,16,11,11,11]});`	Sheet definition 3: "Hits by Date", frozen header, filter `A1:N<rows>`, widths.
4322	`}`	End of the by-date block.
4324	`// ---- Hits by date-time (split across sheets well under Excel's 1,048,576-row limit) ------`	Comment: hits by date-time, split across sheets well under Excel's 1,048,576-row limit.
4325	`if(o.byDateTime){`	Only if "Hits by date-time" was ticked.
4326	`  const CH=200000,hdr=xlHdr(['Device','Interface','Timestamp','Band','Utilization','Transmit (Mbps)','Receive (Mbps)','Capacity (Mbps)','Period']);`	`CH` = 200,000 rows per sheet; header: device, interface, timestamp, band, utilisation, transmit (Mbps), receive (Mbps), capacity (Mbps), period.
4327	`  let rows=[hdr],part=1;`	`rows` starts with the header; `part` counts sheets.
4328	`  const flush=()=>{if(rows.length===1)rows.push(['— No hits in this period —']);`	`flush()` begins: if the sheet has only the header, add a placeholder row…
4329	`    sheets.push({name:xlName(part===1?'Hits by Date-Time':`Hits by Date-Time (${part})`,used),rows,freeze:1,filter:`A1:I${rows.length}`,cols:[24,22,19,12,12,15,15,15,13]});};`	…then add the sheet named "Hits by Date-Time" (or "(2)", "(3)"…) with frozen header, filter and widths.
4330	`  for(const P of sorted)for(const x of P.H.breaches){`	For every link (busiest first) and every hit of that link…
4331	`    if(rows.length>CH){flush();rows=[hdr];part++;}`	…if the current sheet is full, close it, start a new one and increment the part number…
4332	`rows.push([P.r.dev,P.r.intf,xlDT(x.ts),PB.band[x.st].l,{v:x.util,s:'pct2'},x.tx!=null?{v:+(x.tx/1e6).toFixed(3),s:'num1'}:'',x.rx!=null?{v:+(x.rx/1e6).toFixed(3),s:'num1'}:'',P.r.speed?+(P.r.speed/1e6).toFixed(0):'',x.bh?'Market':'Non-market']);`	…push a row: device, interface, timestamp (date-time cell), band name, utilisation (0.00%), transmit and receive in Mbps (1 decimal, blank if missing), capacity Mbps, "Market"/"Non-market".
4333	`  }`	End of the hit loop.
4334	`  flush();`	Close the last sheet.
4335	`}`	End of the date-time block.
4337	`// ---- One chart sheet per link ------------------------------------------------------`	Comment: one chart sheet per link.
4338	`if(o.graph)for(const P of sorted){`	Only if graphs were ticked: loop over links, busiest first.
4339	`  const {r,ser,hit}=P,nm=xlName(`${r.dev} ${r.intf}`,used),pol=o.bands==='policy',pts=ser?ser.pts:[];`	`r` = link, `ser` = bucketed series, `hit` = hit definition; sheet name = "device interface" made unique; `pol` = policy mode; `pts` = series points (empty if none).

Line	Source (verbatim)	What it does
4340	`const stepLbl={h:'hourly','6h':'6-hourly',d:'daily'}[ser?ser.step:'d'];`	`stepLbl` = "hourly", "6-hourly" or "daily" for the note.
4341	`const hdr=['Time','Peak','Average','Peak at/above hit threshold','Warning','High','Critical',...(pol?['Policy']:[]),'Samples'];`	Header: time, peak, average, peak at/above hit threshold, warning, high, critical, (policy), samples.
4342	`const rows=[[{v:`${r.dev} · ${r.intf}${r.linkname?' · '+r.linkname:''}`,s:'title'}],`	Preamble row 1: title "device · interface · link name".
4343	`[{v:`Period ${bD(o.from)} to ${bD(o.to)} · ${HOURS_LBL[o.hours]} · ${P.total.toLocaleString()} hit(s) counted as: ${hit.text}`,s:'note'}],`	Preamble row 2: period, hours basis, hit count and the hit definition.
4344	`[{v:`Chart resolution ${stepLbl}: each point is the highest / mean utilization in that bucket. Threshold lines are the market-hours bands${r.thOff?' (this link has separate non-market bands)':''}.`,s:'note'}],`	Preamble row 3: chart-resolution note (each point is the highest / mean in its bucket; thresholds are the market-hours bands, noting separate non-market bands if the link has them).
4345	`[],xlHdr(hdr)];`	Preamble row 4 empty, then the header row (row 5).
4346	`const fmtT=step=>ts=>step==='d'?bD(ts):bT(ts).slice(0,16);`	`fmtT(step)`: date only for daily buckets, "yyyy-mm-dd hh:mm" otherwise.
4347	`const lab=fmtT(ser?ser.step:'d'),thCols=[r.th.warn,r.th.high,r.th.crit,...(pol?[hit.thr]:[])];`	`lab` = the formatter for this link's step; `thCols` = threshold values drawn as lines (warning, high, critical, plus the policy fraction in policy mode).
4348	`pts.forEach(p=>rows.push([lab(p.t),{v:p.peak,s:'pct'},{v:p.avg,s:'pct'},p.peak>=hit.thr?{v:p.peak,s:'pct'}:'',`	For each point push: time label, peak (percent), average (percent), the peak again only when it is at/above the hit threshold (this column is drawn as red markers)…
4349	`...thCols.map(v=>({v,s:'pct'})),p.n]));`	…the threshold values (percent) and the sample count.
4350	`if(!pts.length)rows.push(['— No samples in this period —']);`	If there are no points, a placeholder row.
4351	`const n=pts.length,last=5+n,series=[`	`n` = points; `last` = last data row (data starts on row 6, so `5+n`); the series list begins: Peak (blue)…
4352	`{name:xlRef(nm,1,5),val:xlRef(nm,1,6,last),color:'#0369A1'},{name:xlRef(nm,2,5),val:xlRef(nm,2,6,last),color:'#93C5E8'},`	…Average (light blue)…
4353	`{name:xlRef(nm,3,5),val:xlRef(nm,3,6,last),color:'#E11D48',marker:true},`	…hit markers (red, markers only, no line)…
4354	`{name:xlRef(nm,4,5),val:xlRef(nm,4,6,last),color:'#B45309',dash:true},{name:xlRef(nm,5,5),val:xlRef(nm,5,6,last),color:'#C2410C',dash:true},`	…Warning and High (dashed)…
4355	`{name:xlRef(nm,6,5),val:xlRef(nm,6,6,last),color:'#E11D48',dash:true},...(pol?[{name:xlRef(nm,7,5),val:xlRef(nm,7,6,last),color:'#475569',dash:true}]:[])];`	…Critical (dashed) and, in policy mode, Policy (dashed grey).
4356	`sheets.push({name:nm,rows,freeze:5,cols:[17,10,10,16,10,10,10,...(pol?[10]:[]),10],`	Sheet definition: frozen at row 5 (header stays visible), column widths.

Line	Source (verbatim)	What it does
4357	`    charts:n>1?[{type:'line',title:`${r.dev} · ${r.intf} — utilization ${bD(o.from)} to ${bD(o.to)}`,yFmt:'0%',skip:Math.max(1,Math.ceil(n/14)),`	Native **line chart**, only if more than one point: title with the link and period, y-axis format 0%, x-label skip = ceil(n/14) so labels stay readable…
4358	`      cats:xlRef(nm,0,6,last),series,anchor:{c0:pol?10:9,r0:3,c1:(pol?10:9)+12,r1:25}}]:[]});`	…categories = the time column; series as above; anchored to the right of the table.
4359	`}`	End of the per-link loop.
4360	`return sheets;`	Return the sheet list (the caller builds the workbook with `xlsxBlobZ`).
4361	`}`	End of function.

18.10 Worked example: one sample from file to report

Settings: bands 60 / 75 / 90 %, upgrade policy 80 %, 5 days, business hours only; market hours 09:00–17:00 Mon–Fri; link capacity 1 Gbps both directions; time zone "as written".

1. The file row is `core-1 | Gi0/1 | 2026-03-02 10:00 | 620 Mbps | 310 Mbps` with headers "Transmit (Mbps)" and "Receive (Mbps)". `headerUnit` returns `{mult:1e6}` for both; `parseTs` (ISO branch → `mkTs`) gives 10:00 on 2 March as written (a Monday).
2. `parseRate` multiplies by 1e6: tx = 620,000,000 bps, rx = 310,000,000 bps; both are flagged as having an explicit unit, so magnitude inference never touches them.
3. `buildSamples`: `uT = 0.62`, `uR = 0.31`, `util = max = 0.62`. Not negative and not above 1.0, so `bad` and `over` stay unset.
4. `computeAnalytics`: no maintenance window, so the sample stays. It contributes to p95, to the Monday-10:00 baseline slot and to the daily series. `stateOfT(0.62, {0.60, 0.75, 0.90})` = **warning** (62 ≥ 60, < 75).
5. `isBH` is true (Monday 10:00 is inside 09:00–17:00). It is **not** at or above the 80 % policy, so it adds nothing to `upgradeDays`.
6. Threshold Report, "Warning and above": `hitCalc` with `bands="warn"` keeps it (state warning is in the set) → `cW` +1, `mWarn` +1, that day's record `w` +1, and it appears in the detail log as "Warning · 62.00 % · 620 Mbps / 310 Mbps · Market".
7. Multi-link pack with "Upgrade policy threshold": `hitCalc` with `bands="policy"` skips it (0.62 < 0.80). The same sample is a hit under one definition and not the other, which is why every report states its definition.
8. Processed-data export: the row appears with utilisation 62.00 %, period "Market hours", "Counts as policy hit" = No, "Excluded from analytics" = "No — included".
9. Regulatory report (threshold 75 %): it is below 75 %, so it adds to neither "% time > T" nor "Days > T"; it does contribute to p95, the average and the market-hours population in the denominator.

> If a maintenance window covering 10:00 were added, steps 4–9 change immediately: the sample leaves every analytic and report number, but the processed-data export still lists it, marked "Yes — maintenance window". Deleting the window brings it back.